CSCE 3701 SOFTWARE ENGINEERING · THE AMERICAN UNIVERSITY IN CAIRO

Software Design Document for Closet

Version 1.2 · revision 4 (Milestone 2 final) · application build v1.5

Prepared by	Habiba Khashaba, Salma, Bashar, Karim and Youssef (Team Closet)
Instructor	Dr. Sherif Aly
Date	10/28/2026
How to read it	Pages headed “v1.2 (rev. 4)” describe the v1.5 design: new diagrams D18 – D23, the security pipeline, the shared Wardrobe Insights layer, repositories and the offline demo, diverse recommendations, Ask Closet routing, notifications, migration 006 and the API additions. The v1.1 revision 3 SDD follows unchanged as the baseline. Where a v1.5 section supersedes a baseline section, R4.0 says so.

R4.0 What changed in the design

Baseline (v1.1 rev 3)	Now (v1.2 rev 4, build v1.5)	Section
3.1 component diagram (D2)	D18 adds the Proactive service, Wardrobe Insights, repositories and the security pipeline as a visible layer	R4.1
5.4 top-3 by score	Diversity-aware selection: any two looks share ≤ 1 major garment when the closet allows	R4.5
5.8 Ask Closet: 14 functions	31 whitelisted functions; questions never change state; follow-up context	R4.6
5.5 notifications without levels	One Notification Center: type, level HIGH / MEDIUM / LOW, stable key, de-duplication	R4.7
4.1 data model (D3)	D20: price, rehoming, snoozes, outfit memory, planned weather (migration 006)	R4.8
Demo: recorded responses	Offline demo runs the real services on in-memory repositories	R4.4
5.19 planned designs	Seasonal rotation, laundry risk, monitor, borrow tracker, trip capsule, purchase impact, cost per wear and donation are now built (Sprint 3 preview)	R4.3

R4.1 Architecture · component view (D18)

Closet stays a **modular monolith**: one Node/Express process, one PostgreSQL database with a schema per module, a React single-page app. v1.5 adds no new process. The proactive features are one service (`intelligence/proactive.js`) that only *composes* the existing modules through their services, plus one pure rule module (`intelligence/insights.js`) with no I/O.

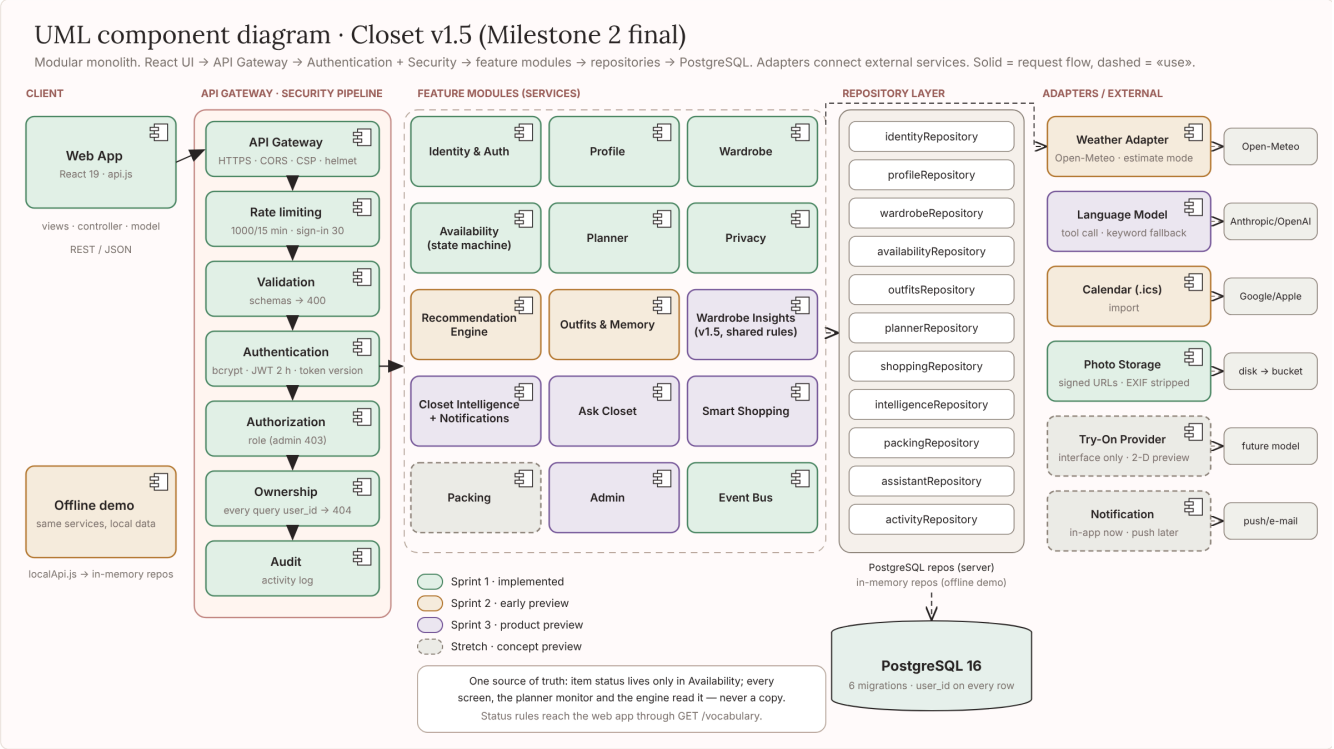


Figure R4-1 (D18). Component diagram v1.5. Every request passes the security pipeline before any module. New in v1.5: Proactive service, Wardrobe Insights, repositories, offline demo host.

R4.2 Security pipeline (applies to every route, old and new)

Order	Stage	Implementation	Failure	Evidence
1	Headers and CORS	helmet (CSP: self, blob: images, Google sign-in only), CORS allow-list	—	SEC unit tests
2	Rate limit	1,000 / 15 min general (photos excluded, D-04); 30 / 15 min sign-in	429	API-AU
3	Validation	shared/validate.js schemas (types incl. number, ranges, enums, max lengths)	400 VALIDATION	UT-V*, API-PR50
4	Authentication	Bearer JWT, short expiry; offline demo uses its own local session token only	401	API-AU, OF-02
5	Role	user / admin; admin routes only accounts and health — no wardrobe data (FR-77)	403	API-AU
6	Ownership	every query scoped by user id; another user's piece, plan, trip or look	404 (not 403 — does not reveal existence)	API-PR01, API-PR03
7	Audit	sign-in, deletes, role changes written to the audit log	—	API-AU

R4.3 Wardrobe Insights — one rule layer, many surfaces

`insights.js` is pure: every function takes plain data (pieces, statuses, wears, weather, events, snoozes, today) and returns a decision with its reasons. The same function feeds Home, Planner, Analytics, Ask Closet and the Notification Center, so a rule cannot say one thing on one screen and another elsewhere.

Function	Rule (thresholds are constants in the file)	Used by	Requirement
seasonFromWeather / seasonFromMonth	≥ 27 °C Summer, ≤ 17 °C Winter, else by month	seasonal, forgotten, trip	FR-84
forgottenCandidates	Available, not rehoming, added ≥ 30 days, in season, unworn 90 days (180 if dress codes ≥ 4 only), not snoozed	Home, Analytics, Ask, notifications	FR-81
seasonalRotation	off-season Available pieces → pack; stored in-season pieces → bring back	Analytics, Ask, notifications	FR-84
laundryRisk	2 of the user's 3 most-worn pieces in Laundry before a planned event	Home, Ask, notifications	FR-85
planDays	n days without repeats; repeats only when the closet runs out, with a note	Ask "plan my week"	FR-86
repeatInsight / similarLook	same set worn recently / near-identical look	Outfit card, Ask	FR-87
memoryMatches / memorySignals	past wears with event, rating, weather; boost ≤ +3 within ±4 °C	Ask, ranker	FR-88
weatherShift	forecast differs from the planned snapshot by ≥ 5 °C or rain appears	Monitor	FR-89
costPerWear / valueInsights	price ÷ wears; only priced pieces; never invented	Piece, Analytics, Ask	FR-97
gapConfidence	HIGH / MEDIUM / LOW from evidence count, with reasons and limits	Shopping, Ask	FR-93
purchaseImpact	distinct major-garment sets the new piece enables; twin → "Probably unnecessary"	Shopping, Ask	FR-94
tripCapsule	carry-on: 8 clothing / 2 shoes / 3 extras; each piece ≤ 2 (3 with laundry) wears	Packing, Ask	FR-95
explainHealth	what raises / lowers the score, from the same parts as the score	Analytics	FR-91

Proactive service

`proactiveService({repo, wardrobe, availability, outfits, planner, weather, recommendation})` loads data through the module services, calls the rules, and performs the few writes the user confirms (snooze, pack for season, adjust one planned piece, remind later). Snoozes are one table with typed keys (`keep:`, `season:`, `borrow:`, `notice:`) and an expiry, so "Not now" works the same everywhere.

Design decision · rehoming is not a status

"Ready to donate / give / sell" is stored in `wardrobe.rehoming`, not as a sixth availability status. Reason: the status machine describes where a piece *is* (Available, Laundry, Borrowed, Repair, Stored); rehoming is an *intention* that can coexist with a status and that ends in a delete with a reason (Donated / Given away / Sold / rehomed). Keeping it separate leaves the tested state machine (28 tests) unchanged. Rehoming pieces are excluded from recommendations.

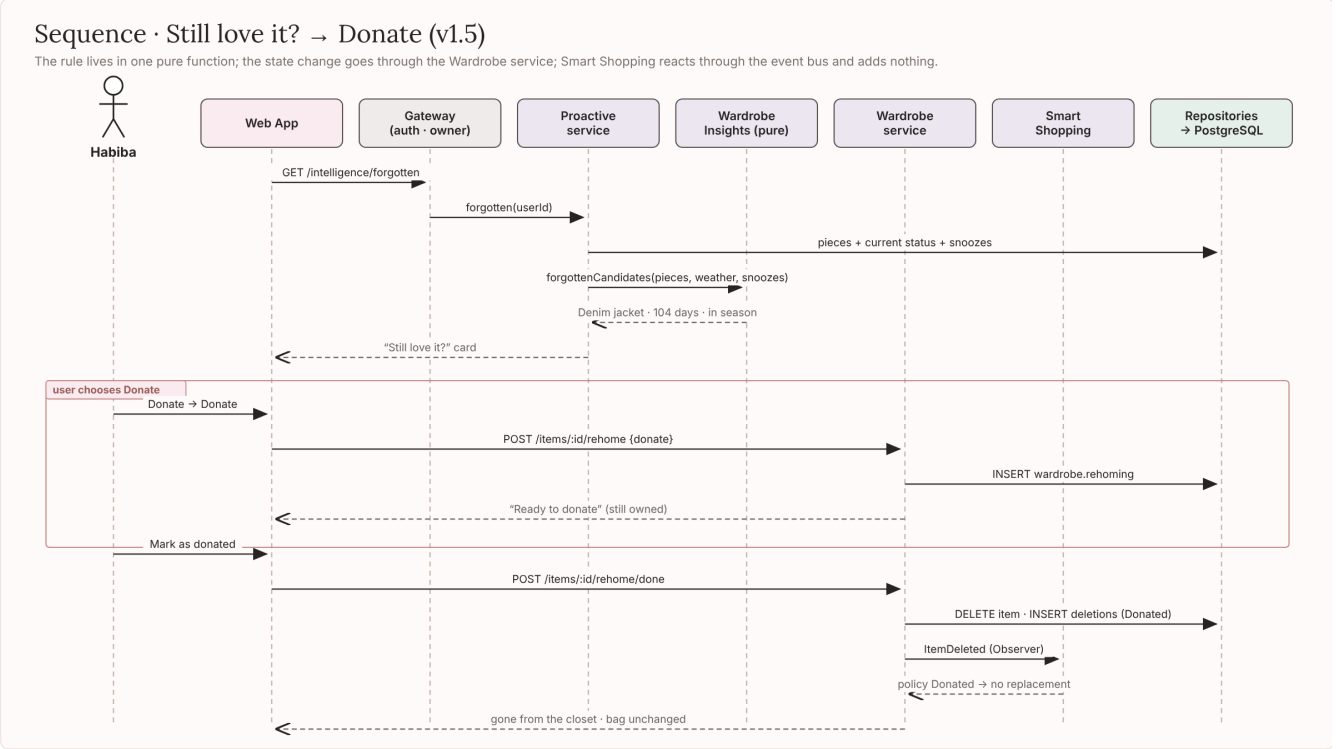


Figure R4-2 (D22). Sequence · Still love it? → Donate → Confirm it has left.

R4.4 Repositories and the offline demo

Every module now receives its repository through `buildModules(deps)` in `app.js`. The server passes PostgreSQL repositories; the offline demo passes `memoryRepositories.js`. Both run the **same services, rules and validation**; only storage differs. This replaced the v1.4 recorded-response demo (defect D-13), where a change on one screen did not reach the others.

Concern	Server	Offline demo (Closet_Demo.html)
Services, rules, validation	same code	same code (bundled with esbuild)
Storage	PostgreSQL 16	in-memory store, saved to sessionStorage; Reset restores the seed
Seed	seedData.js	seedData.js (same pieces, wears, prices, events)
Auth	JWT	local demo session only; requests without it → 401 (OF-02)
Weather / AI	adapter (live or estimate)	estimate mode; keyword parser (no network)
Verified by	API tests, stateful E2E (live)	OF-01 – OF-09, stateful E2E (offline), 15 checkpoints

R4.5 Recommendation · diversity-aware selection

The ranker (pipe-and-filter, unchanged) scores candidate looks. `pickDiverse` then chooses three:

1. Take the best look. For each next pick, penalise overlap with looks already chosen: `score − 6 × weightedOverlap` (accessories weigh 0.5), +3 for a different silhouette (dress vs separates).
2. Accept a look only if it shares at most **1** major garment (top, bottom, dress, outerwear, shoes) with each chosen look.
3. If fewer than three qualify, relax the limit step by step (2, 3, 4) — small closets still get three looks, and the relaxation is visible in tests.
4. With a fixed piece (“style this piece”, forgotten piece) the fixed piece is excluded from the overlap count — it must be in all three.

Verified by DV-01 – DV-07 and demo-check on the seed: maximum shared major garments = 1.

R4.6 Ask Closet routing (31 functions)

The keyword parser maps a message to one of 31 whitelisted functions; the optional LLM adapter may only return one of the same functions with a validated schema. Rules: (1) questions ("Is my blazer clean?") route to read-only functions — they never change state (defect D-09); (2) state changes return an `undoId`; (3) follow-ups use context (`itemIds`, `buy`, `pieceId`) — "add this to my shopping list" after a purchase check; (4) anything else returns the scope message "I can help with your wardrobe, outfits, laundry, shopping, weather and planner."

Group	Functions (names as in FUNCTIONS)
Outfits	recommend · change_piece · style_piece · event_rescue · week_plan · repeat_check · memory · plan_event
Availability	availability (question, read-only) · set_status · wash_done · laundry_list · laundry_risk · borrowed · hide_piece
Shopping	buy_check · add_to_shopping · similar_owned · closet_needs · value
Wardrobe care	forgotten · donate_candidates · keep_piece · rehome_piece · rehome_done · seasonal · health
Planner / travel	plan_check · trip_pack · weather
Scope	help (out-of-scope reply)

The exact list is `FUNCTIONS` in `assistant/keywordParser.js`; AI-40 – AI-47 test 56 phrasings across them.

R4.7 Notification Center

Type	Level (from priority: ≥5 HIGH, ≥3 MEDIUM, else LOW)	Key (de-duplication)	Action
WEATHER_CHANGE	HIGH if the event is today/tomorrow, else MEDIUM	monitor alert key (event + piece)	Adjust outfit
LAUNDRY_WARNING	MEDIUM	laundry-risk key (event)	Open laundry
BORROW_RETURN (overdue)	MEDIUM	borrow:<piece>:<day>	Mark returned
SEASONAL_ROTATION	LOW	season:<season>:<month>	Review
FORGOTTEN_DONATION	LOW	donate:<piece>:<week>	Style it
Existing: AVAILABILITY_CONFLICT, PLAN_REMINDER, LAUNDRY_DAY, WEATHER_ALERT, PACKING, WASH_DONE, FORGOTTEN_PIECE	by priority	unchanged; a due reminder is skipped when the piece already has an overdue notice (D-17)	as before

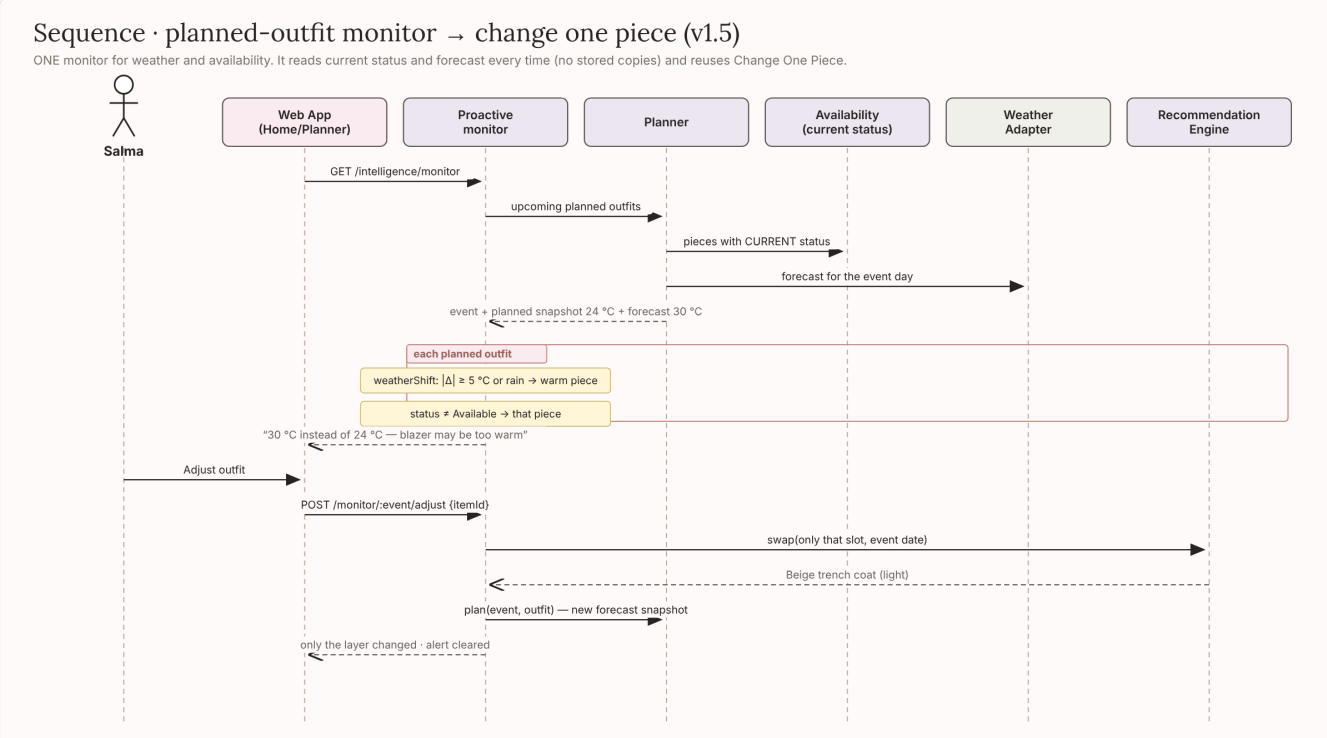


Figure R4-3 (D23). Sequence · planned-outfit monitor → Adjust outfit (only the affected piece changes).

R4.8 Data · migration 006

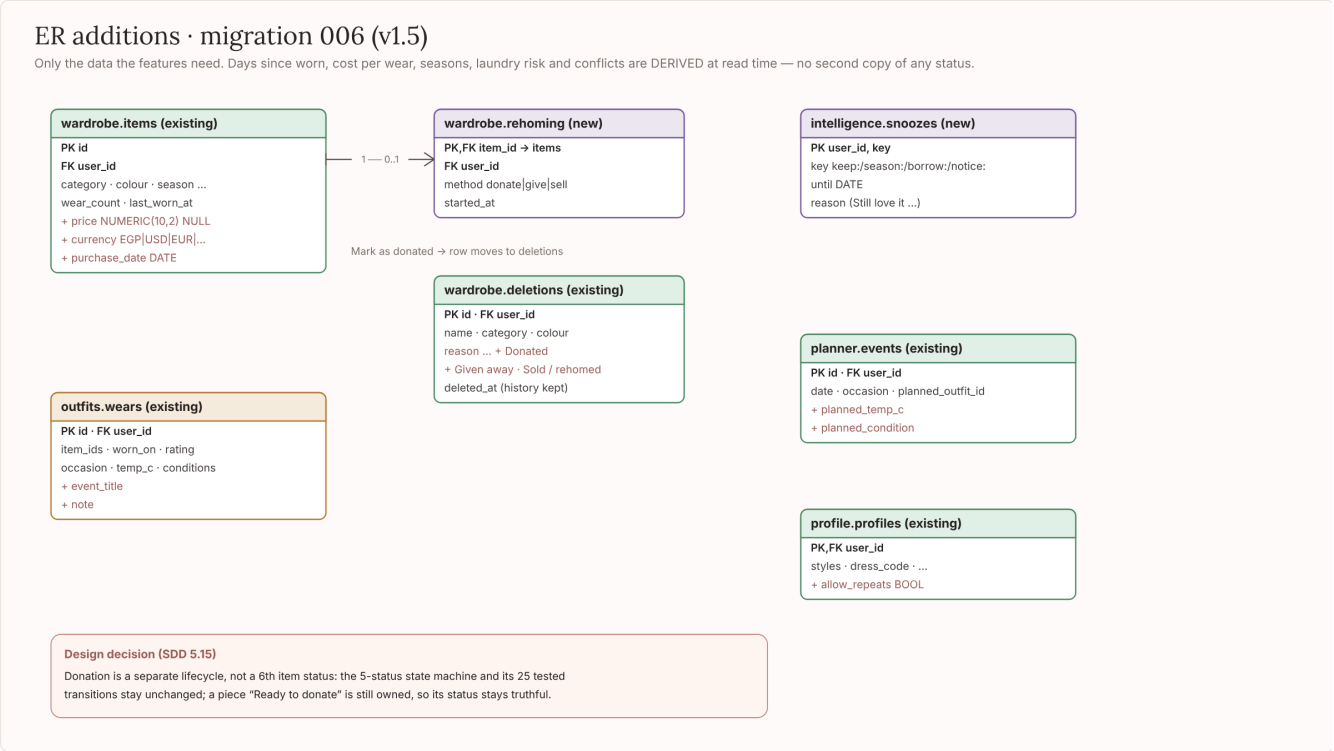


Figure R4-4 (D20). ER additions in v1.5.

Table	Change	Why
wardrobe.items	+ price numeric(10,2) ≥ 0, currency, purchase_date	cost per wear (FR-97)
wardrobe.rehoming (new)	item_id PK, method donate|give|sell, since	donation flow (FR-83)
wardrobe.deletions	reason check + Donated, Given away, Sold / rehomed	exit reasons, no replacement suggested
intelligence.snoozes (new)	user_id, key, until	Keep / Not now / Remind later (FR-82, FR-84, FR-90)
outfits.wears	+ event_title, note	outfit memory (FR-88)
planner.events	+ planned_temp_c, planned_condition	monitor compares with the forecast (FR-89)
profile.profiles	+ allow_repeats	planning preference (FR-86)

Migration 006 was applied to a real PostgreSQL 16 database in the team's test environment; all 180 API tests ran against it.

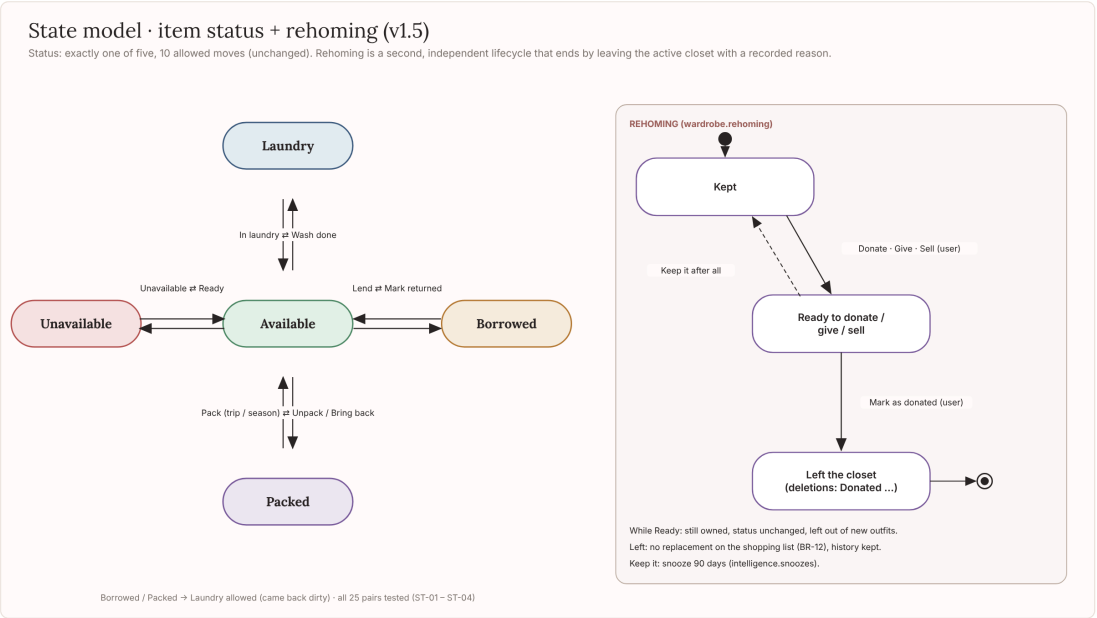


Figure R4-5 (D21). Item status machine (unchanged) and the separate rehoming intention.

R4.9 API additions and changes (build v1.5)

All routes under `/api/v1`; all pass the security pipeline (R4.2). Full table: `04_Design/Closet_API_Table_v1.5.xlsx`.

Method	Route	Purpose	Auth	Request	Response	Errors
GET	`/vocabulary`	Domain vocabulary + STATUS_TRANSITIONS (the web app reads the status rules from here)	public	—	{ CATEGORIES, ..., STATUS_TRANSITIONS }	—
POST	`/items/similar`	Duplicate check before adding a piece	JWT	{ category*, colour*, styles, dressCodes, season, name }	{ count, similar[], message, actions }	400
PATCH	`/items/:id`	Edit a piece — now also price, currency, purchaseDate (null removes)	JWT · owner	{ price ≥ 0 (2 decimals), currency EGP|USD|EUR|GBP|SAR|AED, purchaseDate }	Piece incl. costPerWear, rehome	400 · 404
POST	`/items/:id/rehome`	Mark Ready to donate / give / sell (still owned)	JWT · owner	{ method*: donate|give|sell }	Piece with rehome { method, label, since }	400 · 404
DELETE	`/items/:id/rehome`	Keep it after all	JWT · owner	—	Piece (rehome null)	404
POST	`/items/:id/rehome/done`	Confirm it has left → delete with reason Donated / Given away / Sold / rehomed	JWT · owner	—	{ deleted, reason, effect, addedToShoppingList: null }	404 · 409 NOT_MARKED
POST	`/outfits`	Save a look — now idempotent per set of pieces	JWT · owner of every piece	{ itemIds* (2–4), occasion, favorite, ... }	201 new · 200 existing (same id, favourite kept)	400 · 404
POST	`/outfits/wear`	Wear this — now with eventTitle and note (outfit memory)	JWT · owner	{ itemIds*, rating, occasion, tempC, condition, eventTitle, note }	201 { wearId, day }	400 · 404 · 409 NOT_AVAILABLE
GET	`/intelligence/forgotten`	Long-unworn candidates (style · keep · donate), pieces Ready to leave, stats	JWT	—	{ candidates[], top, pending[], stats, thresholds }	—
POST	`/intelligence/forgotten/:itemId/keep`	Keep it / Not now (snooze)	JWT · owner	{ days 7–365, reason }	{ keptUntil, message }	400 · 404
GET	`/intelligence/seasonal`	Seasonal rotation suggestion	JWT	—	{ season, tempC, pack[], bringBack[], headline }	—
POST	`/intelligence/seasonal`	Pack for season / Bring back / Not now	JWT · owner	{ action*: pack|bring_back|not_now, itemIds* }	{ changed[], skipped[] }	400 · 404 · 409
GET	`/intelligence/laundry-risk`	High-use pieces for an upcoming event are in the laundry	JWT	—	{ warnings[] }	—
POST	`/intelligence/plan-days`	Several days of outfits, laundry-aware	JWT	{ days 1–7, startDate, occasion }	{ days[{ date, occasion, outfit, repeated, note }] }	400
GET	`/intelligence/monitor`	Planned outfits vs CURRENT status and forecast	JWT	—	{ alerts[{ kind, eventId, itemId, message, actions }] }	—
POST	`/intelligence/monitor/:eventId/adjust`	Change only the affected piece (Change One Piece)	JWT · owner	{ itemId* }	{ event, replaced, with, unchanged[] }	404 · 409 NO_REPLACEMENT
POST	`/intelligence/monitor/ignore`	Ignore an alert for 7 days	JWT	{ key* }	{ key, until }	400
GET	`/intelligence/borrowed`	Who has what, since when, overdue	JWT	—	{ borrowed[], overdue[] }	—
POST	`/intelligence/borrowed/:itemId/remind`	Remind me later	JWT · owner	{ days 1–30 }	{ until }	400 · 404
GET	`/intelligence/value`	Cost per wear: best value, least used for its price	JWT	—	{ priced, bestValue[], underused[], note }	—
GET	`/intelligence/memory`	Outfit memory search	JWT	?q, ?best	{ matches[], total }	400

Method	Route	Purpose	Auth	Request	Response	Errors
POST	`/intelligence/repeat`	Repeat insight for a look	JWT	{ itemIds* }	{ insight: null | { kind, message, actions } }	400
GET	`/intelligence/gaps`	Wardrobe gaps with confidence and reasons	JWT	—	{ gaps[{ confidence, priority, reasons, limits }] }	—
GET	`/intelligence/health`	Wardrobe Health — now with explain { raising, lowering, summary }	JWT	—	score, parts, penalties, explain	—
GET	`/notifications`	One Notification Center — now with type, level, action	JWT	—	{ notifications[{ key, type, level, action }], unread }	—
POST	`/shopping/impact`	Purchase simulation (never saved)	JWT	{ category*, colour, styles, dressCodes, season, name }	{ newCombinations, compatiblePieces, recommendation, fillsGap, … }	400
POST	`/packing/plan · /packing/trips`	Trip capsule — now carryOn, laundryAvailable, exclude[]	JWT	+ { carryOn, laundryAvailable, exclude[] }	+ counts { clothing, shoes, extras, combinations }, summary, fitsLimit	400
POST	`/chat`	Ask Closet — 31 functions; context { itemIds, buy, pieceId }	JWT	{ message* ≤ 500, context, pieceId }	{ function, reply, … , undoId }	400 · 404
PUT	`/profile`	Settings — now allowRepeats	JWT	+ { allowRepeats }	Profile	400

R4.10 Use cases and traceability to design

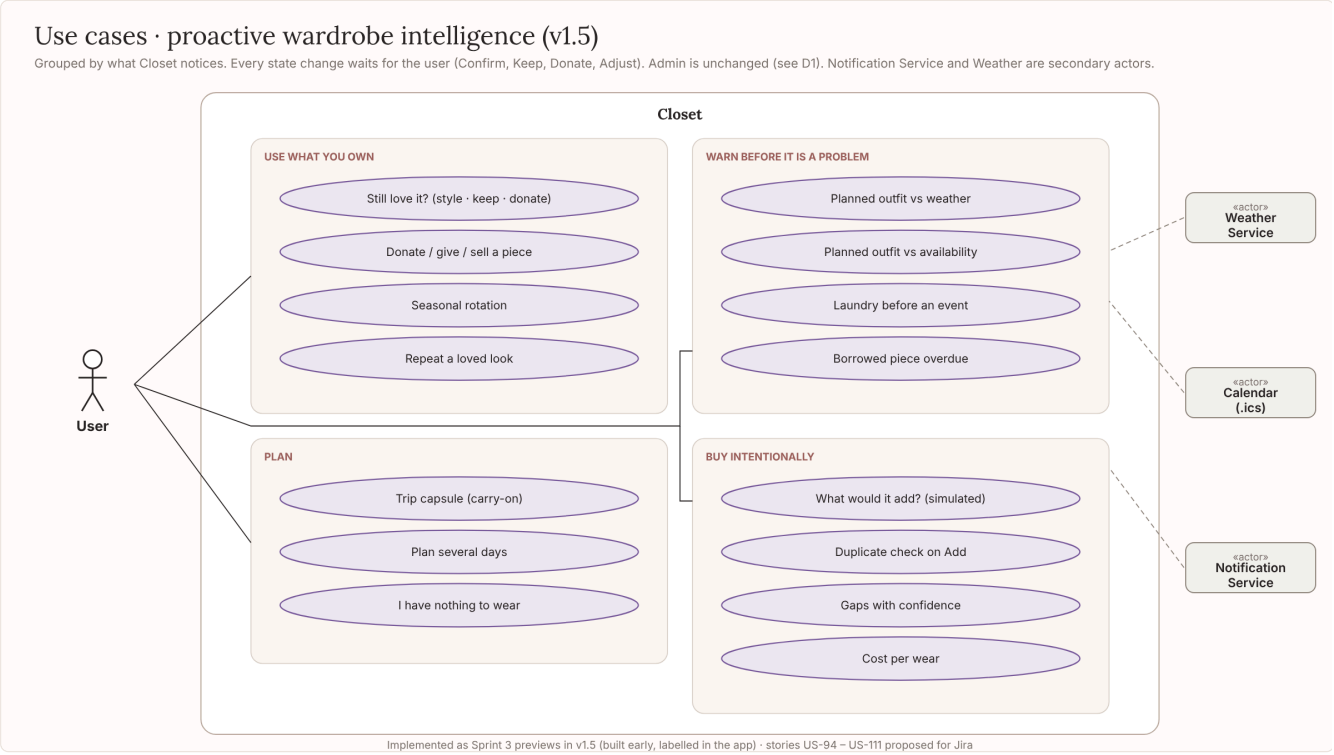


Figure R4-6 (D19). Proactive use cases UC-23 – UC-30. Admin remains in D1.

Req.	Module	API	Data
FR-81	Wardrobe Insights (insights.forgottenCandidates) · Proactive service	GET /intelligence/forgotten	wardrobe.items · availability.item_status · intelligence.snoozes
FR-82	Proactive service · Recommendation Engine	POST /intelligence/forgotten/:itemId/keep · POST /recommendations {fixedId}	intelligence.snoozes
FR-83	Wardrobe · Smart Shopping (ItemDeleted observer)	POST/DELETE /items/:id/rehome · POST /items/:id/rehome/done	wardrobe.rehoming · wardrobe.deletions
FR-84	Wardrobe Insights · Availability state machine	GET/POST /intelligence/seasonal	availability.item_status · intelligence.snoozes
FR-85	Wardrobe Insights (laundryRisk)	GET /intelligence/laundry-risk	availability.item_status · planner.events
FR-86	Wardrobe Insights (planDays) · Recommendation Engine	POST /intelligence/plan-days	—
FR-87	Wardrobe Insights (repeatInsight)	POST /intelligence/repeat	outfits.wears · profile.profiles
FR-88	Outfits · Wardrobe Insights (memoryMatches, memorySignals) · Engine	POST /outfits/wear · GET /intelligence/memory	outfits.wears
FR-89	Proactive monitor · Planner · Recommendation Engine (swap)	GET /intelligence/monitor · POST /intelligence/monitor/:eventId/adjust · POST /intelligence/monitor/ignore	planner.events (planned_temp_c, planned_condition)
FR-90	Proactive service · Availability	GET /intelligence/borrowed · POST /intelligence/borrowed/:itemId/remind · PATCH /items/:id/status	availability.item_status (note, until) · intelligence.snoozes
FR-91	Closet Intelligence (health) · Wardrobe Insights (explainHealth)	GET /intelligence/health	—
FR-92	Wardrobe · Smart Shopping (similarity)	POST /items/similar	—
FR-93	Wardrobe Insights (gapConfidence)	GET /intelligence/gaps	—
FR-94	Smart Shopping · Wardrobe Insights (purchaseImpact) · Engine	POST /shopping/impact	—
FR-95	Packing · Wardrobe Insights (tripCapsule) · Availability	POST /packing/plan · POST /packing/trips · /pack · /unpack	packing.trips
FR-96	Ask Closet · Recommendation Engine · gaps	POST /chat (event_rescue)	—
FR-97	Wardrobe · Wardrobe Insights (costPerWear, valueInsights)	PATCH /items/:id · GET /intelligence/value	wardrobe.items (price, currency, purchase_date)

Req.	Module	API	Data
FR-98	Closet Intelligence (notifications)	GET /notifications · POST /notifications/read	intelligence.notifications_read · intelligence.snoozes
FR-99	Ask Closet (keyword parser / LLM adapter)	POST /chat · POST /chat/undo/:id	assistant.actions
FR-100	Recommendation Engine (pickDiverse)	POST /recommendations	—
FR-101	Outfits (outfitKey, mergeSave)	POST /outfits · PATCH /outfits/:id	outfits.outfits

R4.11 Tradeoffs and limits

- **Rules, not a model.** Proactive features are explainable thresholds. Pro: every notice can say why; testable. Con: thresholds (90 days, 5 °C) are guesses to tune with user data.
- **Composition over new modules.** The Proactive service depends on five module services — a wide fan-in, accepted because it adds no new tables beyond snoozes/rehoming.
- **Weather in the demo is estimated** (WEATHER_MODE=estimate) so the demo is deterministic; live weather depends on the provider key.
- **Try-On is a simulated preview** (overlay on the user's photo), not a generated image — stated in the app.
- **Deployment:** a Render blueprint is configured; the local demo is the verified presentation path.

> The baseline v1.1 revision 3 SDD follows.

Team Closet

Closet — an AI-assisted digital wardrobe

Software Design Document

Version 1.1 · revision 3 (final) · Milestone 2

Name(s)	Habiba Khashaba (requirements, Scrum, documents) · Salma (frontend, demo) · Bashar (profile, planner, gaps, use case diagram, lower-level design, engine) · Karim (backend, database) · Youssef (testing, CI, deployment, component diagram)
Course / section	CSCE 3701 Software Engineering · Fall 2026 · Dr. Sherif Aly
Workstation	github.com/HabibaKhashaba/closet · Jira project SCRUM
Date	10/26/2026
Document version	1.1 (revision 3) · Milestone 2
Project build referenced	Closet application v1.4

TABLE OF CONTENTS

Revision History

1. INTRODUCTION

    1.1 Purpose

    1.2 Scope

    1.3 Overview

    1.4 Reference Material

    1.5 Definitions and Acronyms

2. SYSTEM OVERVIEW

3. SYSTEM ARCHITECTURE

    3.1 Architectural Design

    3.2 Decomposition Description

    3.3 Design Rationale

4. DATA DESIGN

    4.1 Data Description

    4.2 Data Dictionary

5. COMPONENT DESIGN

    5.1 API tier pipeline [Implemented · S1; Audit S2 preview]

    5.2 Identity & Auth [Implemented · S1]

    5.3 Availability — status machine, laundry [Implemented · S1; auto-return and return dates S2 preview]

    5.4 Recommendation Engine — Pipe & Filter, Strategy [Implemented · S2 preview]

    5.5 Closet Intelligence [Implemented · S3 preview; conflicts S2 preview]

    5.6 Smart Shopping and the Shopping List [Implemented · S3 preview; list v1.4]

    5.7 Packing [Implemented · stretch preview]

    5.8 Ask Closet [Implemented · S3 preview]

    5.9 Planner and Calendar Adapter [Implemented · S1; import S2, planned outfits S3 preview]

    5.10 Weather Adapter [Implemented · S2 preview]

    5.11 Error handling [Implemented · S1]

    5.13 Core flow — three looks → change a piece → try on → feedback [Implemented · S2 preview; try-on S3 preview]

    5.14 Deletion-reason handling and learning [Implemented · S3 preview (v1.4)]

    5.15 Ask Closet → shopping list [Implemented · S3 preview (v1.4)]

    5.16 Interaction design of the environments [Implemented · S1 screens; S2/S3 previews labelled]

    5.17 Style a forgotten piece; Mix & Match and Suggest a fix [Implemented · S2/S3 preview (final pass)]

    5.18 Virtual Try-On Preview on the user’s photo, and its privacy [Implemented · S3 preview (final pass); photo-real adapter Planned design]

    5.19 Planned designs (not built)

6. HUMAN INTERFACE DESIGN

    6.1 Overview of User Interface

    6.2 Screen Images

    6.3 Screen Objects and Actions

7. REQUIREMENTS MATRIX

8. APPENDICES

Appendix A – Deployment and testing architecture

Appendix B – Security design

Appendix C – Error handling and validation

Appendix D – Demo data and reset

Appendix E – Design change log

Revision History

Name	Date	Reason for changes	Version
Team Closet	10/13/2026	Milestone 1 baseline: architecture and design inside the combined requirements-and-design specification	1.0
Team Closet	10/25/2026	Milestone 2. Design separated into this SDD (IEEE 1016 template); requirements are in Closet_SRS_v1.1. Adds the authentication and security pipeline, Availability as its own component, the LLM's role, the lower-level design (ER, class, sequence, state), the v1.3 Closet Intelligence design (explainable recommendations, Wardrobe Health, learning, conflicts, notifications), packing, the activity log and the admin health view. Every component is marked Implemented design or Planned design.	1.1
Team Closet	10/26/2026	Revision 2 (instructor feedback): deletion reasons (4) with DeletionPolicy and the ItemDeleted event; shopping list with five sources, labels and similar-owned warning; Ask Closet adds to the list (14 functions); the core flow (three looks → change a piece → try on) and its look store; learning from let-go pieces; interaction design of the dressing room, laundry room and shopping bag (5.13 – 5.16); figures 5-1 – 5-4; try-on release corrected to Sprint 3	1.1 rev. 2
Team Closet	10/26/2026	Revision 3 (final pass): fixed-item path for a forgotten piece (5.17), Mix & Match scoring and the Suggest-a-fix algorithm (5.17), Virtual Try-On Preview on the user's photo with privacy and retention (5.18), look names and the replace tray (5.13), complete ER on one sheet, figures 5-5 – 5-6; build referenced: Closet v1.4	1.1 rev. 3

Implemented vs planned. Each design element in this document is labelled: **[Implemented · S1]** — built, tested and graded in Sprint 1; **[Implemented · S2/S3 preview]** — built and tested early, accepted at its own sprint review; **[Implemented · stretch preview]** — a working concept of a stretch goal; **[Planned design]** — designed, not built.

1. INTRODUCTION

1.1 Purpose

This software design document describes the architecture and system design of **Closet**, an AI-assisted digital wardrobe, release 1.0. It shows how the software is structured to satisfy the requirements in *Closet_SRS_v1.1*: the subsystems and their responsibilities, the interfaces between them, the data model, the algorithms of the main components, and the user interface. It is written for the team's developers and tester (it is the reference for code development), and for the course instructor and TA who review the design.

1.2 Scope

Closet knows which clothes a user owns and which are in the laundry, lent out or packed, and recommends three explained outfits for an occasion using the dress code, the user's style and modesty preference, the weather and the user's history. It includes an AI chat that acts on the closet, an outfit builder, a try-on avatar, a planner with calendar import, wear analytics, Wardrobe Health, smart shopping, notifications, packing and an admin view. Goals: a decision in under a minute every morning, higher wardrobe utilisation, fewer unnecessary purchases, and AI that is transparent and under the user's control.

The design covers the web client, the server (one Node.js process), the PostgreSQL database and the adapters to external services. It does not cover the external services themselves.

1.3 Overview

Section 2 gives the system overview and the product loop. Section 3 is the architecture: the architectural design and component diagram (3.1), the decomposition into modules with the class diagram, interfaces, API and sequence diagrams (3.2), and the design rationale (3.3). Section 4 is the data design (ER diagrams and data dictionary). Section 5 describes each component in pseudocode. Section 6 is the human interface design with screen images. Section 7 is the requirements matrix that traces every SRS requirement to components, data and tests. The appendices hold the deployment and test architecture, the security design, error handling, the demo-data design and the design change log.

1.4 Reference Material

Document	Version	Location
Closet Software Requirements Specification	1.1	Closet_SRS_v1.1.pdf
Closet test report (Milestone 2)	v1.4	Closet_M2_Test_Report_v1.4.pdf
IEEE Std 1016-2009, Software Design Descriptions	2009	course template
R. Pressman, *Software Engineering: A Practitioner's Approach*; course slides on architectures and architectural thinking	—	course material
Source code (server/src, client/src, migrations, tests)	Git history	github.com/HabibaKhashaba/closet

1.5 Definitions and Acronyms

Term	Meaning
Module	A feature area of the server with its own router, service, repository and database schema
Adapter	The only code that talks to one external service; always has a fallback
Pipeline	The ordered request steps in the API tier: security → validation → authentication → access control → audit
Piece	One clothing item (photo + details + status)

Term	Meaning
Status	Available, Laundry, Borrowed, Packed or Unavailable
Rule score	100 − 17 × failed rules (five rules)
Match %	Average of the four part scores colour, style, formality, weather (SRS Appendix A)
Factor	One explanation line of a recommendation, naming one fact the engine used
Pattern	A detected regularity in the user’s feedback, with evidence and confidence
Conflict	A planned piece that will not be available for its event, or a double-booked piece
JWT	JSON Web Token (signed login token) · **CSP** Content Security Policy · **CORS** Cross-Origin Resource Sharing
LLM	Large language model · **PDL** procedural description language (pseudocode) · **ER** entity–relationship
S1, S2, S3	Sprint 1, 2, 3

2. SYSTEM OVERVIEW

Closet is a single-page web application backed by a layered modular monolith. The user photographs pieces and gives their details; the server stores them with an availability status. When the user asks for an outfit, the Recommendation Engine filters the Available pieces, combines them into complete outfits, scores them with transparent rules, ranks them by the user’s taste and feedback, and explains each one with the facts it used. Every reaction (wear, rate, reject with a reason, swap, favourite) is stored as feedback; Closet Intelligence turns feedback into patterns the user can confirm, into a Wardrobe Health score, into conflict warnings for planned outfits, and into notifications.

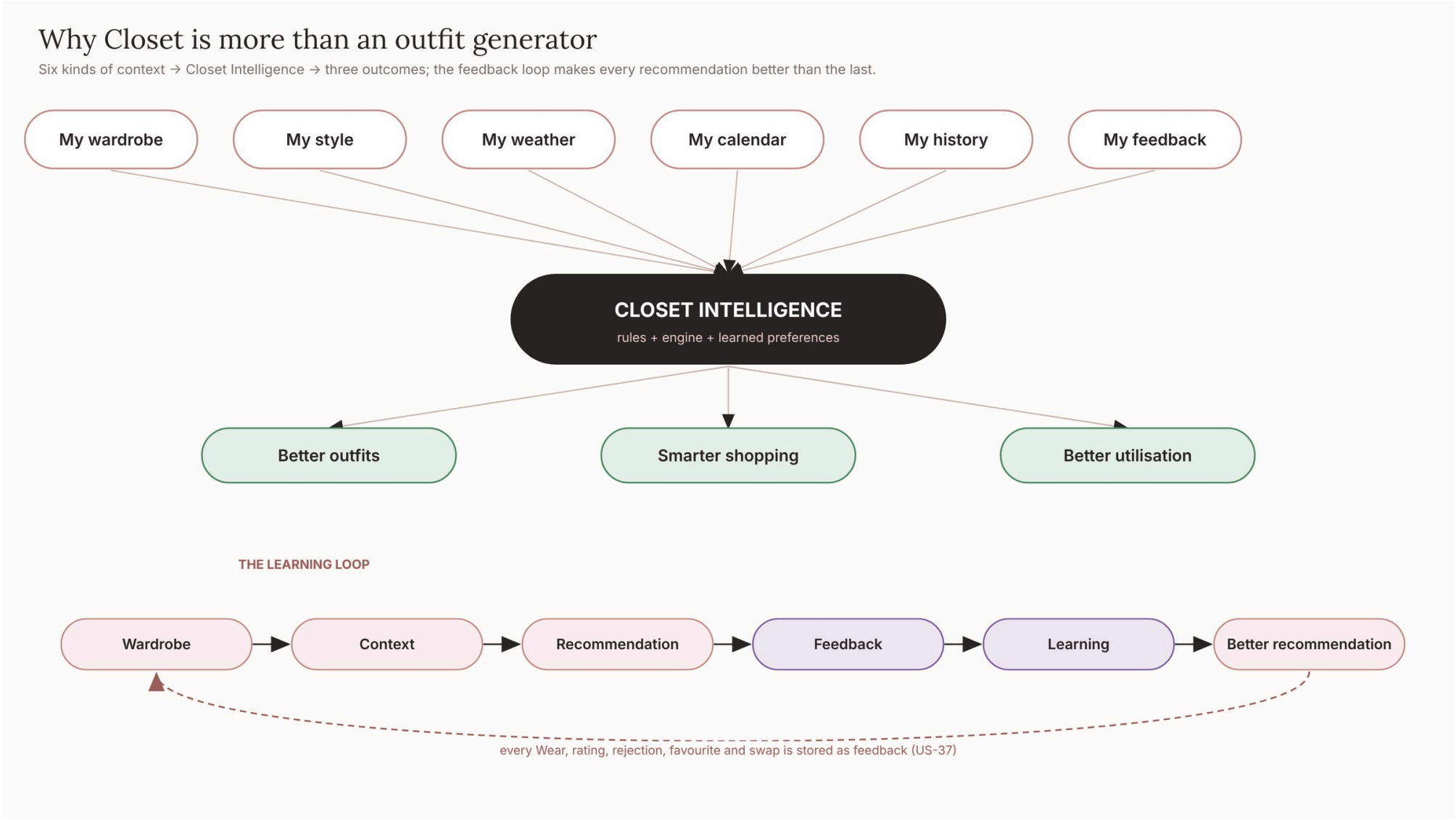


Figure 2-1 – Context and product loop: six kinds of context → Closet Intelligence → three outcomes; feedback closes the loop.

Implementation status at Milestone 2. Sprint 1 — accounts and security, style profile, closet, availability and laundry, planner events and closet gaps — is implemented and graded: 169 automated tests pass. The Sprint 2 and Sprint 3 features and two stretch goals were built early as previews and are labelled as such in the app; with them the suite has 349 passing tests (test report). External integrations that are *not* connected are drawn dashed and marked Planned design: photo-real try-on, push / e-mail notifications, the vision service and live calendar sync.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

Style: layered modular monolith. One backend process is split into modules with explicit interfaces behind one API tier, with the client, application modules, adapters and data in separate tiers. Modules call each other only through provided interfaces or the in-process Event Bus; each module owns its tables.

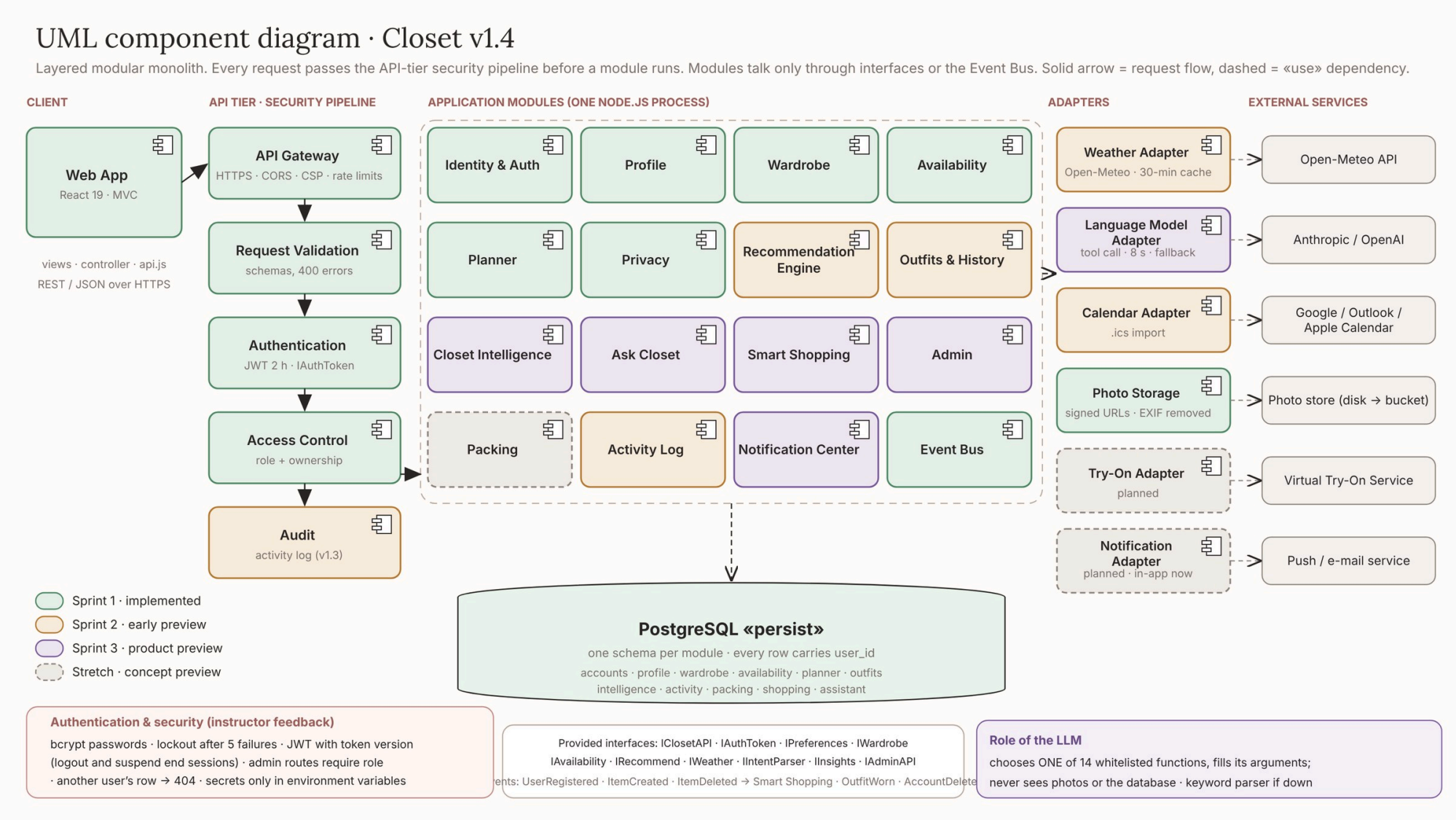


Figure 3-1 – UML component diagram, version 1.4 (prepared by Youssef). Colour = the sprint of each component; dashed = planned design.

How the subsystems collaborate.

1. **Client tier — Web App** [Implemented · S1]. React 19, MVC: `api.js` is the model (every HTTP call), `App.jsx` the controller (routing, session, shared state), screens the views. It never talks to the database or external services.
2. **API tier — security pipeline** [Implemented · S1, Audit S2 preview]. Every request passes, in order: **API Gateway** (HTTPS only, CORS limited to the web app, security headers including CSP, rate limit on sign-in), **Request Validation** (schema per route from the shared vocabulary), **Authentication** (verifies the JWT through IAuthToken: signature, expiry, token version, account status), **Access Control** (role for admin routes; ownership by user id in every repository), and **Audit** (writes successful changes to the user's activity log). Only then is a module called.
3. **Application modules.** Identity & Auth, Profile, Wardrobe, Availability, Planner, Privacy, Event Bus [S1]; Recommendation Engine, Outfits & History, Activity Log [S2 preview]; Closet Intelligence, Ask Closet, Smart Shopping, Notification Center, Admin [S3 preview]; Packing [stretch preview].
4. **Adapters.** Weather (Open-Meteo, 30-minute cache, seasonal-estimate fallback), Language Model (tool call to Anthropic or OpenAI, 8 s timeout, keyword-parser fallback), Calendar (.ics import), Photo Storage (private, signed URLs) are implemented; Try-On, Notification push and Vision adapters are planned design.
5. **Data tier.** PostgreSQL 16 with one schema per module; photos in private storage behind the PhotoStorage interface; secrets in environment variables.

Request journey — "What happens when Salma clicks Add?" React UI → API route (`POST /api/v1/items`) → pipeline (security, validation, authentication, access control) → Wardrobe Service → Photo Storage and Wardrobe Repository → PostgreSQL → Event Bus (`ItemCreated`) → Availability sets the status Available → audit entry "Piece added" → 201 to the client. Figure 3-6 shows it as a sequence diagram.

Subsystems and responsibilities.

Component	Responsibility	Provides	Requires / relations	Status
Web App «MVC»	All screens; flat-lay outfits; Virtual Try-On Preview on the user's photo (browser only) with the SVG avatar as fallback; Notification Center UI	UI	IClosetAPI	Implemented · S1 (screens of later sprints labelled)
API Gateway «Facade»	HTTPS only, CORS, security headers (CSP), sign-in rate limit, routing	IClosetAPI, IAdminAPI	all module routers	Implemented · S1
Request Validation	Schema per route from the shared vocabulary; field errors	—	shared/validate	Implemented · S1
Authentication	Valid, unexpired JWT on every protected route; sets the current user	—	IAuthToken	Implemented · S1
Access Control	Role check (admin); ownership enforced in repositories	—	—	Implemented · S1
Audit	Writes successful changes to the activity log (rule table)	—	Activity Log	Implemented · S2 preview
Identity & Auth	Register, sign in, Google, sign out, bcrypt, lockout, token version	IAuthToken	publishes UserRegistered	Implemented · S1
Profile	Style quiz, preferences, wardrobe rules	IPreferences	subscribes UserRegistered	Implemented · S1 (rules S2 preview)

Component	Responsibility	Provides	Requires / relations	Status
Wardrobe	Pieces, photos, browse/filter, edit, delete with one of 4 reasons, gaps, wear count	IWardrobe	Photo Storage, IAvailability, IPreferences; publishes ItemCreated, ItemDeleted	Implemented · S1
Availability «State»	Owns every status: state machine, laundry basket, Wash done, auto-return, return dates	IAvailability	subscribes ItemCreated; IPreferences	Implemented · S1
Planner	Events, month view, .ics import, calendar markers, planned outfits, reminders	—	IRecommend, IWeather, IAvailability, ICalendar	Implemented · S1 (import S2, plans S3 preview)
Privacy	Delete account (phrase + password)	—	publishes AccountDeleted	Implemented · S1
Recommendation Engine «Pipe & Filter» «Strategy»	Filter → combine → score → rank → explain; swap; style a piece; builder check	IRecommend	IWardrobe, IPreferences, IWeather, Outfits (signals)	Implemented · S2 preview
Outfits & History	Saved / favourite outfits, wear log with occasion and weather, ratings, rejections, hides, insights	IInsights	IWardrobe; publishes OutfitWorn	Implemented · S2 preview
Closet Intelligence	Wardrobe Health, learned patterns (decide / undo), conflicts and fixes	IInsights	IWardrobe, Outfits, IPreferences, Planner, IRecommend	Implemented · S3 preview (conflicts S2)
Notification Center	Computes notices from Planner, Availability, Packing, Analytics, Weather; read state; 3 highlights	IInsights	Planner, Closet Intelligence, Activity Log, INotifier (planned)	Implemented · S3 preview
Ask Closet	Message → one of 14 functions (incl. add to shopping list) → run through module interfaces → reply + Undo	—	IIntentParser, IRecommend, IWardrobe, IAvailability, Planner, Smart Shopping	Implemented · S3 preview
Smart Shopping / Shopping List	Shopping list (sources: Ask Closet, replacement, gap, suggestion, manual) with labels and similar-owned warning; suggestions (gap, basic) with outfits unlocked; “Do I need this?”; similarity; subscribes ItemDeleted → replacement	IShopping	IWardrobe, IPreferences, engine, DeletionPolicy; subscribes ItemDeleted	Implemented · S3 preview
DeletionPolicy «pure functions»	Effect of each deletion reason (BR-10): replacement for Torn / damaged and Became too small; learn + avoid for No longer my style; let-go styles	—	used by Wardrobe, Smart Shopping, Closet Intelligence	Implemented · S3 preview
Packing	Trip plan, packing lists, Mark as packed / Unpack	—	IWeather, engine, Planner, IAvailability	Implemented · stretch preview
Activity Log	Stores and lists the user’s activity	—	—	Implemented · S2 preview
Admin	Accounts, suspend / restore, stats, system health, read-only configuration	IAdminAPI	Access Control	Implemented · S3 preview (health: stretch)
Event Bus «Observer»	In-process publish / subscribe, awaited in order	publish / subscribe	—	Implemented · S1

Component	Responsibility	Provides	Requires / relations	Status
Weather Adapter «Adapter»	Open-Meteo geocoding + forecast, 30-min cache, seasonal estimate fallback	IWeather	Open-Meteo	Implemented · S2 preview
Language Model Adapter «Adapter»	One tool call (Anthropic / OpenAI), 8 s timeout, keyword-parser fallback	IIntentParser	LLM API	Implemented · S3 preview
Calendar Adapter «Adapter»	.ics parsing (UTC → Cairo, all-day, escapes, folded lines)	ICalendar	calendar files	Implemented · S2 preview; live sync Planned design
Photo Storage	Private files, HMAC-signed URLs, type check, EXIF removal	PhotoStorage	disk (bucket later)	Implemented · S1
Try-On Adapter	Photo-real try-on	ITryOn	Virtual Try-On Service	Planned design
Notification Adapter	Push / e-mail delivery of notices	INotifier	Notification Service	Planned design
Vision Adapter	Tag suggestions with confidence	IVisionModel	Vision API	Planned design (stretch)

3.2 Decomposition Description

We use an object-oriented description: each module is decomposed into a **Router** (HTTP binding and validation schemas), a **Service** (business rules, the module's provided interface) and a **Repository** (all SQL, always filtered by `user_id`). Pure domain logic (status machine, recommendation engine, health formula, learning rules, similarity, packing planner, .ics parser) lives in plain functions so every rule is unit-tested.

3.2.1 Class diagram.

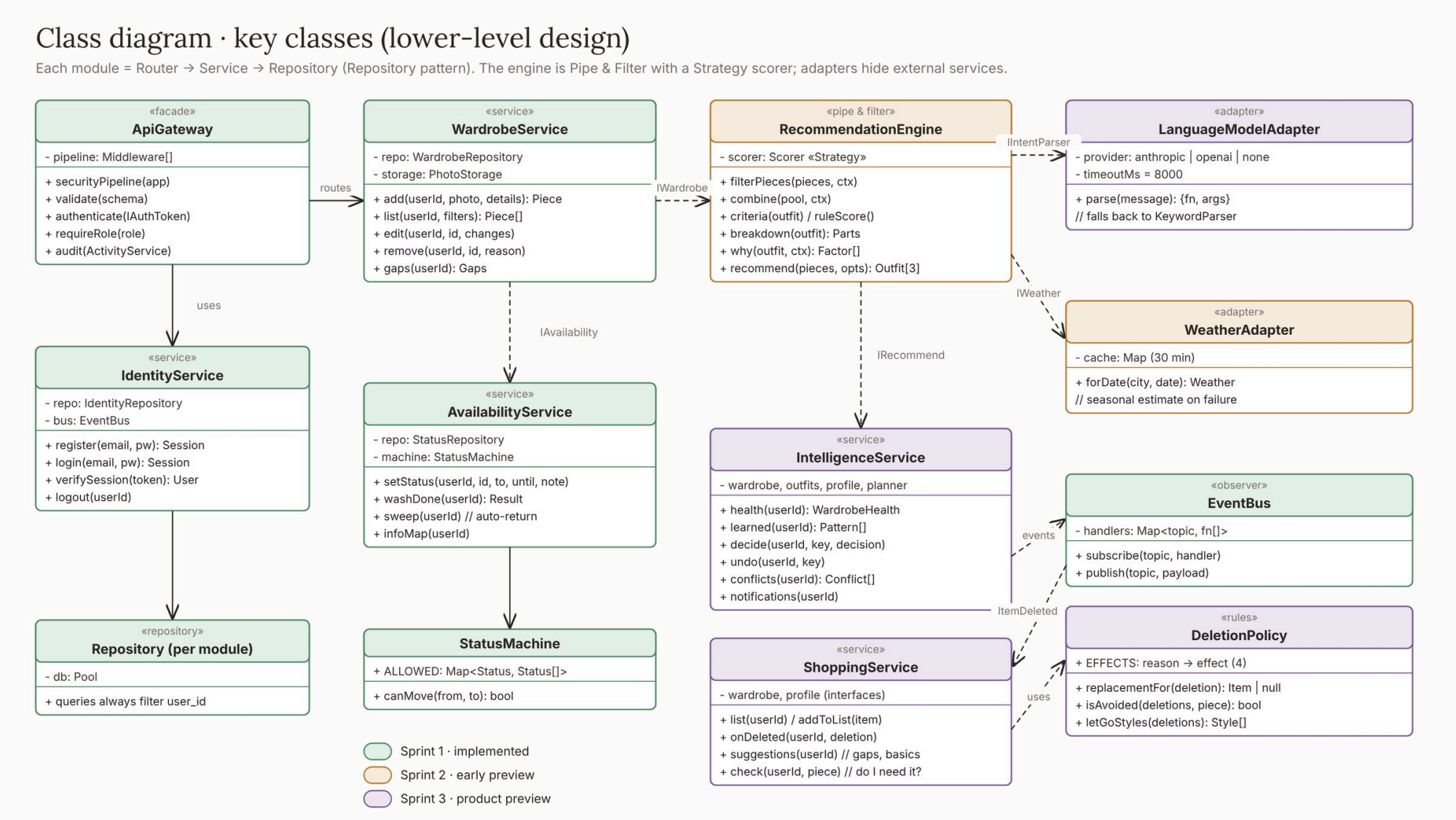


Figure 3-2 – Class diagram of the key classes. Arrows: solid = uses, dashed = depends on an interface.

3.2.2 Interface specifications.

Interface / event	Provided by	Used by	Operations (signature → result)	Status
IClosetAPI	API Gateway	Web App	REST/JSON under /api/v1 (table 3.2.3)	Implemented · S1
IAuthToken	Identity & Auth	Authentication step	verifySession(token) → User {id, email, role} or 401	Implemented · S1
IPreferences	Profile	Recommendation Engine, Smart Shopping, Intelligence	preferences(userId) → {styles, colours, occasions, dressCode, modest, city, laundryDays, repeatDays…}; update(userId, changes)	Implemented · S1
IWardrobe	Wardrobe	Engine, Outfits, Planner, Shopping, Intelligence, Packing, Ask Closet	allPieces(userId) → Piece[] with status; gaps(userId); deletions(userId); recordWear(userId, ids, day)	Implemented · S1
IAvailability	Availability	Wardrobe, Planner, Packing, Ask Closet	setStatus(userId, itemId, to, {until, note}); washDone(userId); infoMap(userId); settings(userId)	Implemented · S1
IRecommend	Recommendation Engine	Planner, Ask Closet, Intelligence	recommend(userId, {occasion, style, level, date, fixedId}) → 3 Outfit; swap(userId, {itemIds, slot, …}); check(userId, {itemIds, occasion})	Implemented · S2 preview
IWeather	Weather Adapter	Engine, Planner, Packing	forDate(city, date) → {tempC, high, low, condition, source}	Implemented · S2 preview
IIntentParser	Language Model Adapter	Ask Closet	parse(message, context) → {fn ∈ 14 functions, args, via}	Implemented · S3 preview
IInsights	Outfits / Closet Intelligence	Web App, Notifications	insights, history, health, learned, conflicts, notifications	Implemented · S2/S3 preview
IShopping	Smart Shopping / Shopping List	Web App, Ask Closet	list(userId) → items with tags, reason, similarOwned, warning; addToList(userId, {title, category, colour, source, reason, replaces}) → {item, added, note}; setListStatus; removeFromList; onDeleted(deletion)	Implemented · S3 preview (v1.4)
IAdminAPI	Admin	Admin screens	users, stats, health, config, setStatus — admin role only	Implemented · S3 preview
ICalendar	Calendar Adapter	Planner	parseIcs(text) → events	Implemented · S2 preview; live sync Planned design
INotifier	Notification Adapter	Notification Center	send(user, notice)	Planned design (in-app only today)
Try-On Preview	Web App (photoTryOn.js)	Try-on screen	storePhoto(file) → data URL kept in this tab; cutout(pieceUrl) → transparent garment; region(piece) → place on the body; removePhoto()	Implemented · S3 preview (final pass)
ITryOn	Try-On Adapter	Web App	render(outfit, photo) → image	Planned design
UserRegistered	Identity & Auth	Profile (creates the default profile)	event	Implemented · S1

Interface / event	Provided by	Used by	Operations (signature → result)	Status
ItemCreated	Wardrobe	Availability (status Available)	event	Implemented · S1
OutfitWorn	Outfits	analytics consumers	event	Implemented · S2 preview
ItemDeleted	Wardrobe	Smart Shopping (replacement for Torn / damaged, Became too small)	event {userId, name, category, colour, styles, reason} → reaction {addedToShoppingList}	Implemented · S3 preview (v1.4)
AccountDeleted	Privacy	every module (purge)	event	Implemented · S1

3.2.3 API design. All endpoints are under `/api/v1`, JSON over HTTPS. “Auth” = bearer JWT required; every authenticated route is scoped to the caller.

Method	Path	Access	Module	Release
POST	`/auth/register`	—	Identity & Auth	S1
POST	`/auth/login`	—	Identity & Auth	S1
POST	`/auth/google`	—	Identity & Auth	S1
POST	`/auth/logout`	Auth	Identity & Auth	S1
GET	`/auth/me`	Auth	Identity & Auth	S1
GET	`/vocabulary · /health`	—	Gateway	S1
GET · PUT	`/profile · /profile/quiz`	Auth	Profile	S1 (rules S2)
GET · POST	`/items`	Auth	Wardrobe	S1
GET · PATCH · DELETE	`/items/:id`	Auth	Wardrobe	S1 (delete S2; v1.4 returns effect + list item)
PATCH	`/items/:id/status`	Auth	Availability	S1
POST · DELETE	`/items/:id/hide`	Auth	Outfits	S2
GET	`/photos/:key?sig`	signed URL	Wardrobe / Photo Storage	S1
GET	`/laundry`	Auth	Availability	S1
POST	`/laundry/wash-done`	Auth	Availability	S1
GET	`/closet/gaps`	Auth	Wardrobe	S1
GET · POST	`/events`	Auth	Planner	S1
PATCH · DELETE	`/events/:id`	Auth	Planner	S1
GET	`/events/calendar · /upcoming · /reminders`	Auth	Planner	S2 / S3
POST	`/events/import`	Auth	Planner + Calendar Adapter	S2
POST	`/events/:id/recommend`	Auth	Planner + Engine	S3

Method	Path	Access	Module	Release
PUT · DELETE	`/events/:id/outfit`	Auth	Planner + Outfits	S3
GET	`/weather?date`	Auth	Weather Adapter	S2
POST	`/recommendations`	Auth	Recommendation Engine	S2
POST	`/recommendations/swap`	Auth	Recommendation Engine	S2
POST	`/recommendations/check`	Auth	Recommendation Engine	S2
GET · POST	`/outfits`	Auth	Outfits	S2
PATCH · DELETE	`/outfits/:id`	Auth	Outfits	S2
POST	`/outfits/wear`	Auth	Outfits	S2
PATCH	`/outfits/wears/:id`	Auth	Outfits	S3
POST	`/outfits/reject`	Auth	Outfits	S2
GET	`/outfits/history`	Auth	Outfits	S2 (v1.3)
GET	`/insights`	Auth	Outfits	S3
GET	`/intelligence/health`	Auth	Closet Intelligence	S3 (v1.3)
GET	`/intelligence/learned`	Auth	Closet Intelligence	S3 (v1.3)
POST · DELETE	`/intelligence/learned/:key`	Auth	Closet Intelligence	S3 (v1.3)
GET	`/intelligence/conflicts`	Auth	Closet Intelligence	S2 (v1.3)
POST	`/intelligence/conflicts/:eventId/fix`	Auth	Closet Intelligence	S2 (v1.3)
GET · POST	`/notifications · /notifications/read`	Auth	Notification Center	S3 (v1.3)
GET · POST	`/chat · /chat/undo/:id`	Auth	Ask Closet	S3
GET · POST	`/shopping · /shopping/check`	Auth	Smart Shopping	S3
PUT · DELETE	`/shopping/:key`	Auth	Smart Shopping	S3
GET · POST	`/shopping/list`	Auth	Shopping List	S3 (v1.4)
PATCH · DELETE	`/shopping/list/:id`	Auth	Shopping List	S3 (v1.4)
POST	`/packing/plan`	Auth	Packing	Stretch (v1.3)
GET · POST	`/packing/trips`	Auth	Packing	Stretch (v1.3)
POST	`/packing/trips/:id/pack · /unpack`	Auth	Packing	Stretch (v1.3)
DELETE	`/packing/trips/:id`	Auth	Packing	Stretch (v1.3)
GET	`/activity`	Auth	Activity Log	S2 (v1.3)
GET	`/admin/users · /stats`	Auth + admin	Admin	S3
PATCH	`/admin/users/:id`	Auth + admin	Admin	S3

Method	Path	Access	Module	Release
GET	`/admin/health · /config`	Auth + admin	Admin	Stretch (v1.3)
DELETE	`/account`	Auth	Privacy	S1

3.2.4 Sequence diagrams.

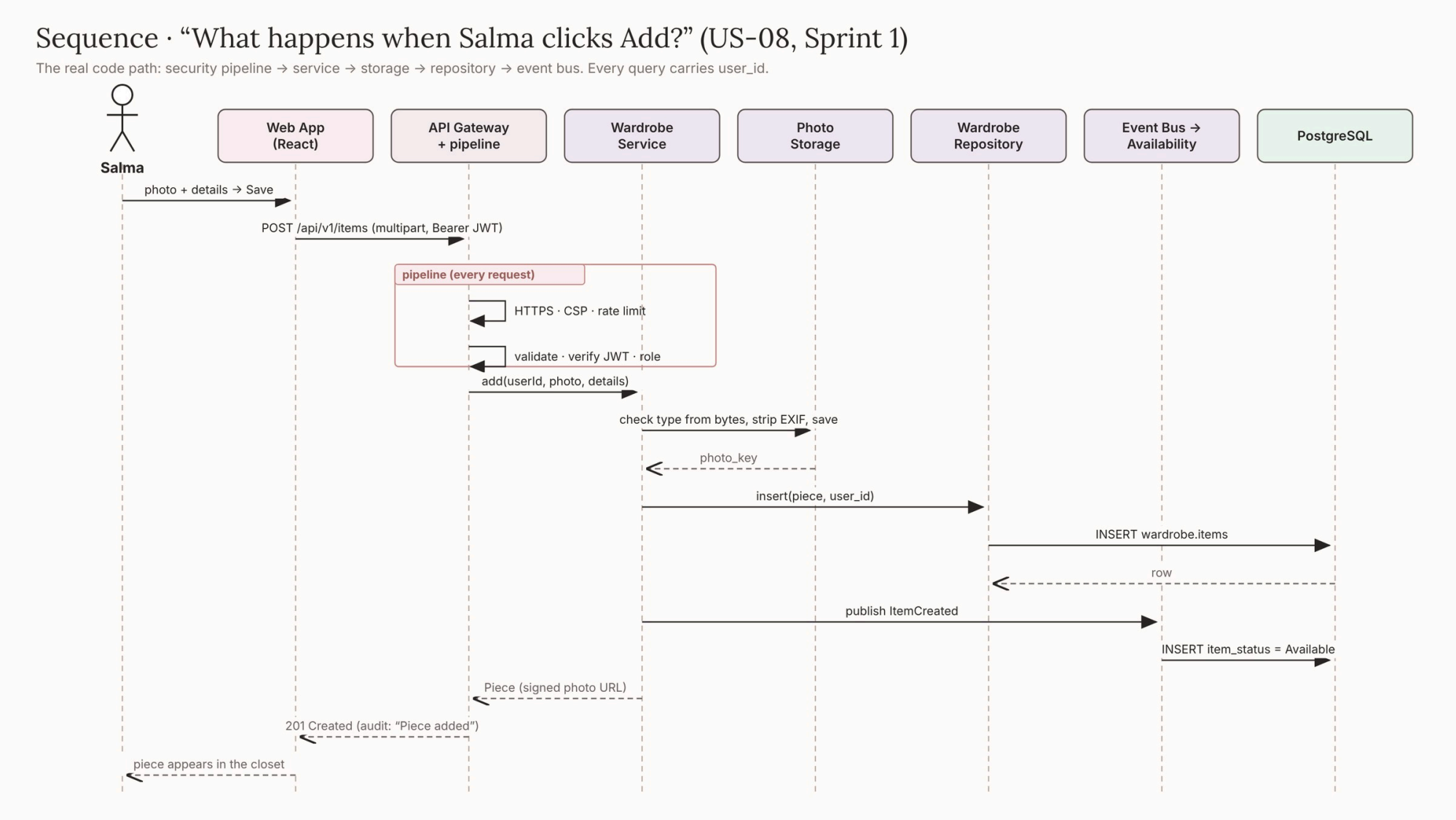


Figure 3-3 – Add a piece (US-08, Sprint 1).

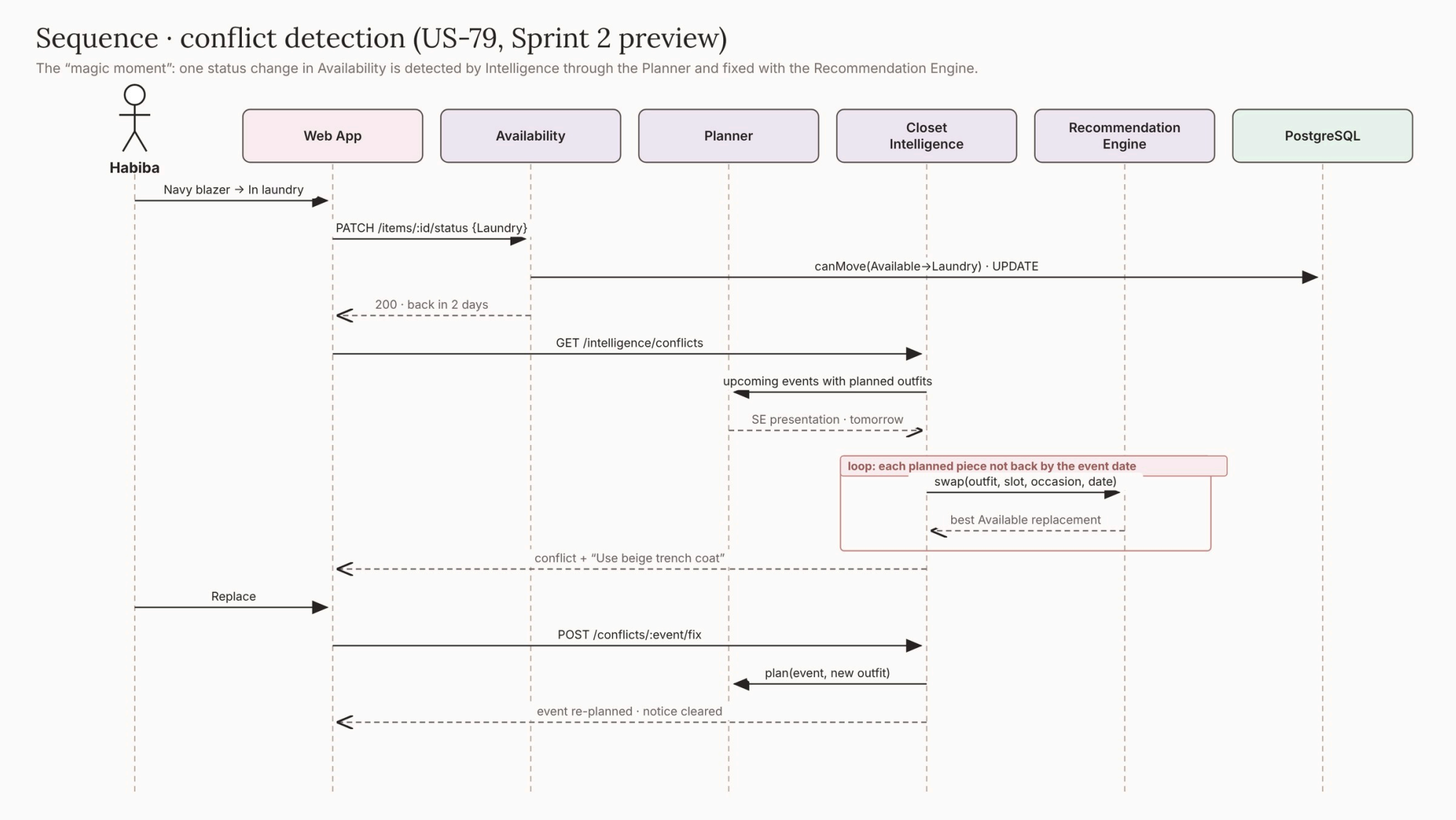


Figure 3-4 – Conflict detection and one-tap replacement (US-79, S2 preview).

Sign in with lockout (UC-01, S1). The gateway applies the sign-in rate limit; validation checks the email and password; Identity & Auth loads the user by email (case-insensitive), refuses a locked or suspended account, compares the bcrypt hash, increments `failed_logins` and sets `locked_until = now + 15 min` on the 5th failure (423), otherwise resets the counter and issues a 2-hour JWT carrying the user id, role and token version. The same 401 message is returned for an unknown email and a wrong password.

3.2.5 State model. Availability implements the status of a piece as a state machine (State pattern): `ALLOWED` lists the 10 legal moves; `setStatus` refuses any other with 409 and writes every move to `status_history`.

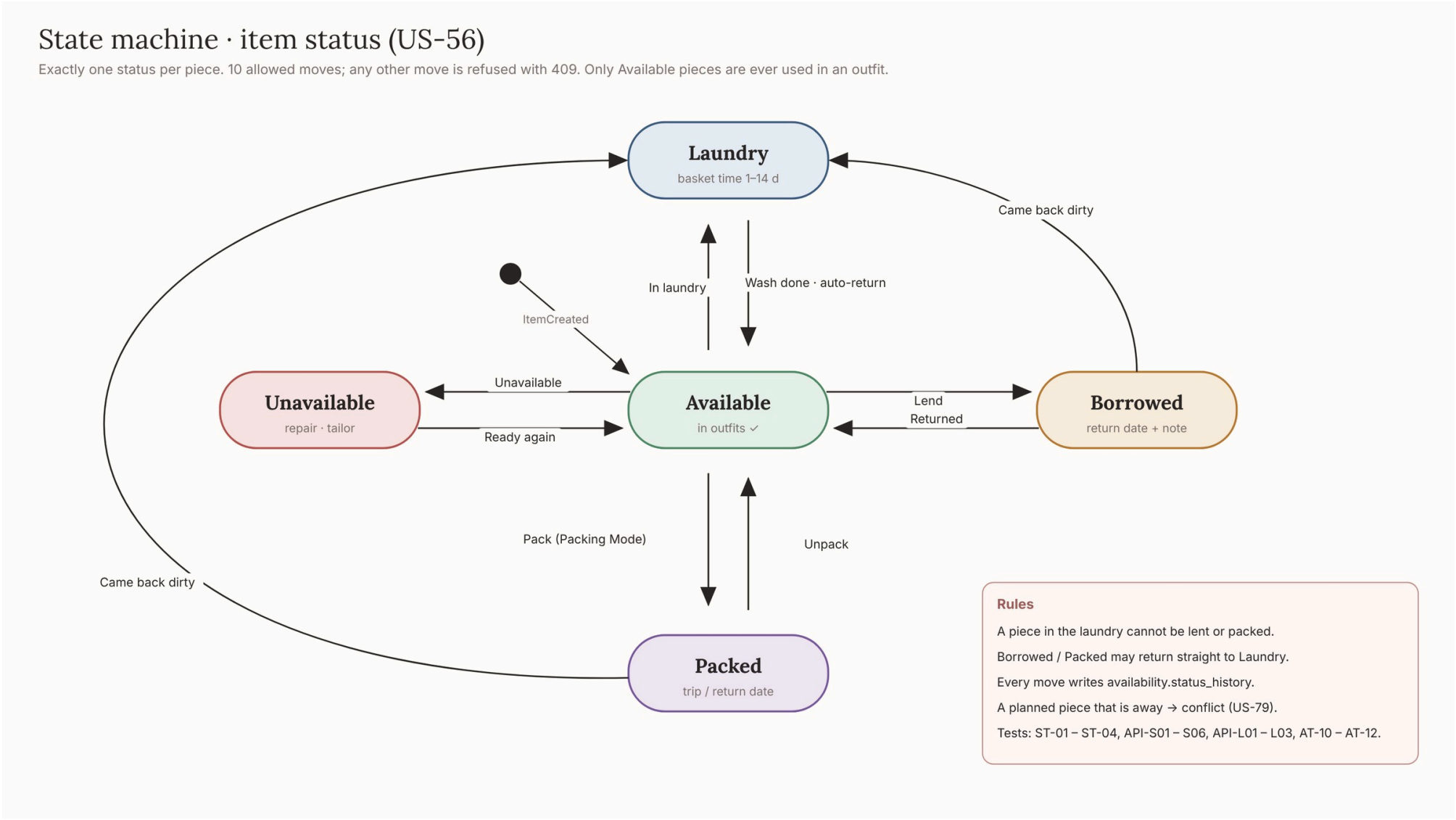


Figure 3-5 – Status state machine.

3.2.6 Design patterns.

Pattern	Where	Quality attribute
Layered	Client · API tier · modules · adapters · data	Maintainability
Facade	API Gateway (one entry point)	Security, simple client
Chain of Responsibility / Interceptor	Security → validation → authentication → access control → audit	Security, separation of concerns, auditability
Repository	One repository per module, all queries filtered by user_id	Testability, security
Pipe and Filter	Recommendation Engine: filter → combine → score → rank → explain	Modifiability, testability
Strategy	Scorer of the engine (rule-based today; a learned scorer can replace it)	Modifiability
State	Availability status machine	Correctness
Observer	Event Bus (UserRegistered, ItemCreated, ItemDeleted, OutfitWorn, AccountDeleted)	Loose coupling
Policy (pure functions)	DeletionPolicy: one table maps each deletion reason to its effect (BR-10)	Testability, single source of truth
Adapter	Weather, Language Model, Calendar, Photo Storage (Try-On, Notification, Vision planned)	Availability, swap providers
MVC	Web App	Maintainability
Command + Undo	Ask Closet actions and learned-preference decisions store the previous state	Safety (NFR-14)

3.3 Design Rationale

Decision	Options considered	Chosen and why	Trade-off accepted
Overall architecture	Microservices · unstructured monolith · modular monolith	**Modular monolith**: one codebase, one deployment and one database for a 5-person, 280-hour project; interfaces keep the boundaries, so a module (most likely Ask Closet) can later become a service	The whole backend scales as one unit
Security placement	Checks inside each module · a pipeline in front of all modules	**Pipeline** in the API tier (instructor feedback: authentication must be evident): every route gets the same checks; modules cannot forget them	One more layer to read when debugging
Ownership	Row-level security in PostgreSQL · repository filters	**Repository filters on user_id** plus 404 for foreign rows: simple, testable, works with one pool	Each repository must be written carefully (covered by tests W16–W18)
Recommendation scoring	Trained model · rules	**Transparent rules + ranking** (Strategy): explainable, testable, no training data; feedback changes ranking, not the rule score	Less “clever” than a model; the Strategy allows replacing it
Explanations	LLM-written text · factors from the engine	**Factors computed from the same facts the engine used**; an LLM could invent reasons	Fixed sentence shapes
Role of the LLM	Free agent with database access · whitelisted tool call	**One of 14 whitelisted functions**, executed through module interfaces, every change undoable; keyword fallback	The chat can only do what a function exists for

Decision	Options considered	Chosen and why	Trade-off accepted
Learning	Silent automatic updates · proposals the user confirms	**Confirm / Not accurate with exact Undo** (BR-6): the user stays in control; evidence and confidence are shown	Learning is slower
Notifications	Push service now · computed in-app notices	**Computed on request** from current state (no stale data, no scheduler); read state stored	No notices outside the app until the Notification Adapter exists
Audit	Calls in every module · interceptor	**Audit step in the pipeline** with a rule table: modules stay unaware; one place to review	Detail is limited to what the request and response contain
Events	Message broker · in-process bus	**In-process Event Bus** (Observer), awaited in order	Not distributed; replace with a broker if split
Deletion effects	if/else in the delete route · a policy table + ItemDeleted event	**DeletionPolicy table + Observer**: Wardrobe does not know about shopping; the reason → effect rule is tested once (DR-01..06)	One more event to trace
Shopping list	Reuse shopping.decisions · own table	**Own table shopping.list_items** with a `source` column, so every item can say where it came from (label) and why	A second shopping table
Core flow state	Server-side session · client look store	**Client look store (sessionStorage)** so the dressing room can flip between the same three looks and return with a changed piece	Lost when the tab is closed (looks can be regenerated)
Try-on	Photo-real service now · client-drawn avatar · preview on the user's own photo in the browser	~~**SVG avatar drawn in the browser** (≤ 1 s, no photo of the user)~~ **Preview on the user's photo, drawn in the browser** (the photo never leaves the device; avatar as fallback; ITryOn kept for a photo-real provider)	~~Stylised, labelled as simulated~~ A simulated overlay (flat garment photos on the body), labelled "Virtual Try-On Preview"; fit adjusted by hand

4. DATA DESIGN

4.1 Data Description

The information domain (users, pieces, statuses, outfits, feedback, events, decisions) is stored in **PostgreSQL 16**, one schema per module, created by four migrations (001 Sprint 1; 002 outfits, planner outfits, chat actions; 003 customization, deletions, shopping; 004 intelligence, history, activity, packing). Every table carries `user_id` with `ON DELETE CASCADE` from `accounts.users`, so deleting an account removes everything (US-06). Arrays (`styles` , `dress_codes` , `item_ids`) keep a piece or outfit in one row; CHECK constraints enforce the vocabulary. Photos are files in private storage referenced by `photo_key` and served through HMAC-signed, expiring URLs with location data removed. Computed values — match %, Wardrobe Health, patterns, conflicts, notifications — are not stored; they are derived on request from the stored facts, so they are never stale. Only *decisions* are stored (learned decisions with the previous values, notices read, shopping decisions).

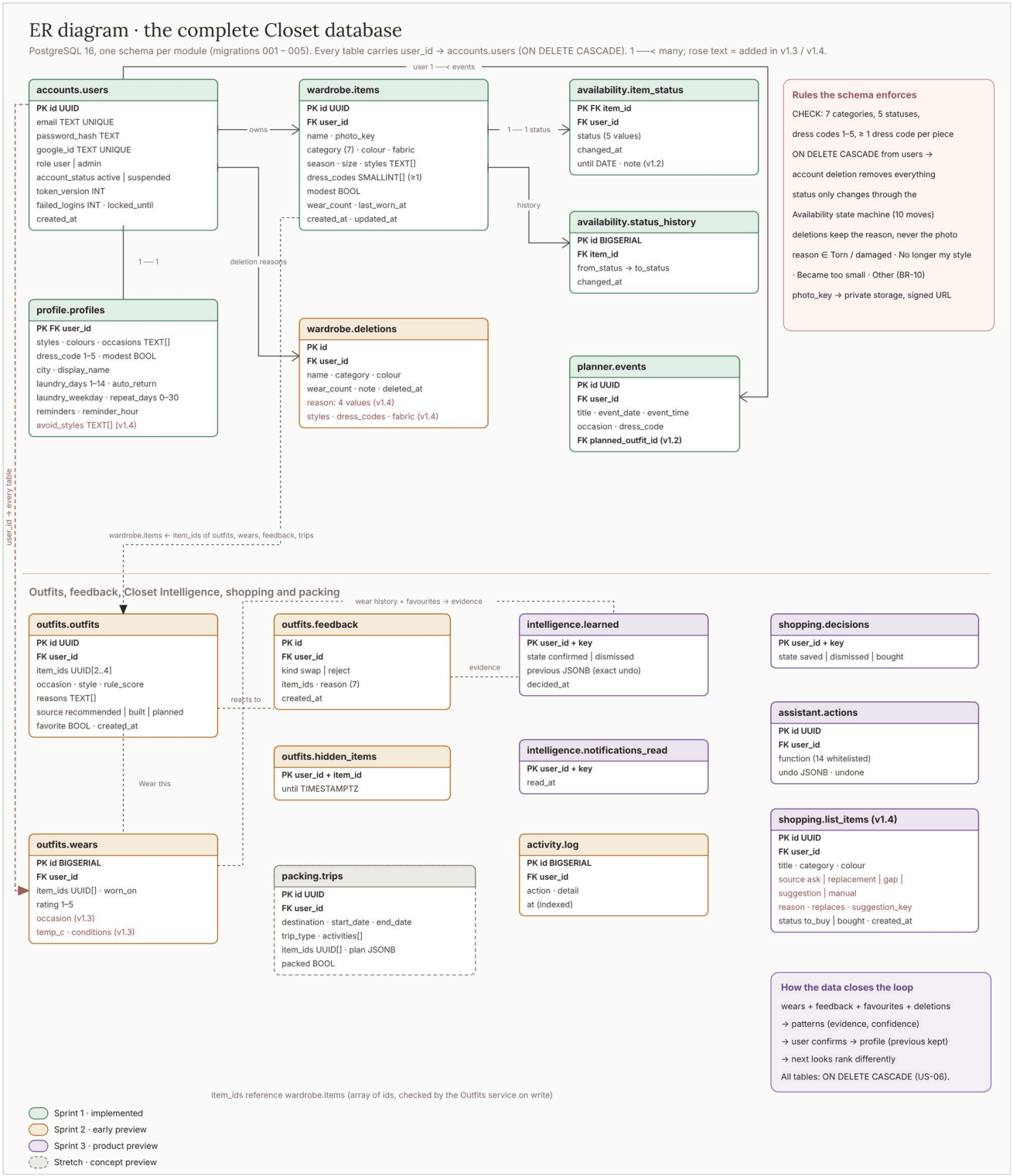


Figure 4-1 – ER diagram: the complete Closet database on one sheet (core data; outfits and feedback; Closet Intelligence; shopping).

4.2 Data Dictionary

Entities (tables) in alphabetical order. Types are PostgreSQL types; PK = primary key, FK = foreign key.

Entity	Attributes (type, constraint)	Description
`accounts.users`	id UUID PK · email TEXT UNIQUE (lower-case) · password_hash TEXT (bcrypt, null for Google-only) · google_id TEXT UNIQUE · role user	admin · account_status active

Entity	Attributes (type, constraint)	Description
`activity.log`	id BIGSERIAL PK · user_id FK · action TEXT · detail TEXT ≤ 160 · at TIMESTAMPTZ (index user_id, at)	Activity / audit entries (v1.3)
`assistant.actions`	id UUID PK · user_id FK · function TEXT (14 whitelisted) · undo JSONB · undone BOOL · created_at	Ask Closet changes with their undo data
`availability.item_status`	item_id UUID PK FK · user_id FK · status (5) · changed_at · until DATE · note TEXT ≤ 60	Current status of each piece
`availability.status_history`	id BIGSERIAL PK · item_id FK · from_status · to_status · changed_at	Every status move
`intelligence.learned`	PK (user_id, key) · state confirmed	dismissed · previous JSONB {values, text, effect} · decided_at
`intelligence.notifications_read`	PK (user_id, key) · read_at	Which notices were read (v1.3)
`outfits.feedback`	id BIGSERIAL PK · user_id FK · kind swap	reject · item_ids UUID[] · reason (7) · created_at
`outfits.hidden_items`	PK (user_id, item_id) · until TIMESTAMPTZ	Pieces hidden for 7 days
`outfits.outfits`	id UUID PK · user_id FK · item_ids UUID[2..4] · occasion · style · rule_score 0–100 · reasons TEXT[] · source recommended	built
`outfits.wears`	id BIGSERIAL PK · user_id FK · item_ids UUID[] · worn_on DATE · rating 1–5 · occasion · temp_c · conditions (v1.3)	Outfit history
`packing.trips`	id UUID PK · user_id FK · destination · start_date · end_date ≥ start · trip_type · activities TEXT[] · item_ids UUID[] · plan JSONB · packed BOOL · created_at	Packing lists (v1.3, stretch)
`planner.events`	id UUID PK · user_id FK · title 1–80 · event_date · event_time · occasion · dress_code 1–5 · planned_outfit_id FK → outfits · created_at	Events
`profile.profiles`	user_id PK FK · styles · colours · occasions TEXT[] · dress_code 1–5 · modest · city · quiz_completed_at · display_name · laundry_days 1–14 · laundry_auto_return · laundry_weekday 0–6 · repeat_days 0–30 · reminders · reminder_hour · avoid_styles TEXT[] (v1.4, learned from let-go pieces) · updated_at	Style profile and wardrobe rules
`shopping.decisions`	PK (user_id, key) · state saved	dismissed
`shopping.list_items`	id UUID PK · user_id FK · title 1–80 · category (7) · colour · source ask	replacement
`wardrobe.deletions`	id BIGSERIAL PK · user_id FK · name · category · colour · wear_count · reason (4: Torn / damaged, No longer my style, Became too small, Other) · styles TEXT[] · dress_codes SMALLINT[] · fabric (v1.4) · note · deleted_at	Why pieces left the closet (no photo)
`wardrobe.items`	id UUID PK · user_id FK · name · photo_key · category (7) · colour · fabric · season · size ≤ 10 · styles TEXT[] · dress_codes SMALLINT[1..5] · modest · wear_count · last_worn_at · created_at · updated_at	A piece

Main objects exchanged through the API (JSON):

Object	Attributes	Produced by
Conflict	key, kind (unavailable / double-booked), eventId, event {title, date, time, occasion}, itemId, piece, status, expectedBack, message, replacement {piece, outfit}	Closet Intelligence
Factor	factor (Occasion, Style, Modesty, Weather, Availability, Wear history, Favourites, Your feedback), text	Recommendation Engine `why()`
Notification	key, kind, priority 1 – 5, title, text, link, date, read, highlight	Notification Center
Outfit (recommended)	itemIds[2 – 4], pieces, title, ruleScore, colour, style, formality, weather, compatibility, criteria[5], reasons[3 – 5], why[], explanation	Recommendation Engine
Pattern	key, kind, text, evidence, count, confidence, change {profile fields}, effect, source	Closet Intelligence `detectPatterns()`
Piece	id, name, category, colour, fabric, season, size, styles[], dressCodes[], modest, status, statusSince, statusUntil, statusNote, expectedBack, wearCount, lastWornAt, photoUrl	Wardrobe
Trip plan	destination, startDate, endDate, tripType, activities[], days[{date, occasion, event, weather, outfit}], itemIds[], counts {pieces, outfits, days, missing}, missing[], warnings[], reuse[]	Packing
WardrobeHealth	score 0 – 100, grade, parts[{key, label, points, max, detail}], penalties[], actions[≤ 3], stats, forgotten[], duplicates[]	Closet Intelligence `wardrobeHealth()`

5. COMPONENT DESIGN

Pseudocode (PDL) of every main component, written from the code. Local data is named where it matters.

5.1 API tier pipeline [Implemented · S1; Audit S2 preview]

```
FOR EACH request to /api/v1/*:
  securityPipeline: IF production AND not HTTPS → redirect; set CSP and security headers;
CORS only for CLIENT_ORIGIN
  IF route is /auth/login|register|google → rateLimit(30 per 15 min per IP)
  validate(schema): FOR EACH field rule → collect "<label> must …" messages; IF any → 400
VALIDATION_FAILED {details}
  authenticate (protected routes): claims ← readToken(bearer)            // HS256, expiry
checked
      user ← identity.verifySession(claims)   // exists, account active, claims.tv =
user.token_version
      IF none → 401  ELSE req.user ← {id, email, role}
  requireRole('admin') on /admin/* : IF req.user.role ≠ admin → 403
  audit: rule ← first RULE matching (method, path); IF rule.piece → name ←
pieceName(user, id) BEFORE the handler
      ON response finish: IF status < 400 → activity.record(user, rule.action, name ·
detail)  // never fails the request
  route to module router; errors → errorHandler (5.11)
```

5.2 Identity & Auth [Implemented · S1]

```
login(email, password):
  user ← repo.findByEmail(lower(trim(email)))
  IF user = null → 401 "Email or password is incorrect"
  IF user.locked_until > now → 423 locked
  IF user.locked_until (expired) → reset failed_logins            // start counting again
  IF user.account_status ≠ active → 403
  IF NOT bcrypt.compare(password, user.password_hash):
      failed_logins ← failed_logins + 1; IF failed_logins ≥ 5 → locked_until ← now + 15
min → 423
      ELSE → 401 (same message as an unknown email)
  reset failed_logins; RETURN {token: jwt(sub = id, role, tv = token_version, 2 h), user}
logout(user): token_version ← token_version + 1          // every older token stops
working
```

5.3 Availability — status machine, laundry [Implemented · S1; auto-return and return dates S2 preview]

```
ALLOWED = { Available: [Laundry, Borrowed, Packed, Unavailable], Laundry: [Available],
            Borrowed: [Available, Laundry], Packed: [Available, Laundry], Unavailable:
[Available] }   // 10 moves
setStatus(user, item, to, until, note):
  current ← repo.get(user, item) OR 404
  IF to ∈ {Available, Laundry} → until, note ← null
  IF until < today → 400
  IF current = to AND no until/note → no change
  IF NOT to ∈ ALLOWED[current] → 409 INVALID_TRANSITION "Allowed: …"
  transaction: UPDATE item_status WHERE status = current (optimistic); INSERT
status_history(current → to)
washDone(user): transaction: ids ← all Laundry pieces; set Available; history rows;
RETURN {returned: |ids|}
sweep(user):  // on every read of statuses, when auto-return is on
  FOR piece IN Laundry WHERE changed_at + laundryDays ≤ now → Available (+ history)
expectedBack(piece) = Laundry ? changed_at + laundryDays : until
```

5.4 Recommendation Engine — Pipe & Filter, Strategy [Implemented · S2 preview]

```
recommend(pieces, {occasion, style, level, prefs, weather, hiddenIds, today, signals, fixedId}):
  level ← level OR OCCASIONS[occasion]
  pool ← filterPieces:  keep p IF status = Available (BR-1) AND p ∉ hidden AND (NOT prefs.modest OR p.modest)
          AND main/accent pieces suit level AND layers suit level or level−1
          AND NOT (Top|Dress|Bottom worn < prefs.repeatDays days ago)      // recycle time
          AND NOT (weather ≥ 26 °C AND warm(p))
  combos ← combine(pool): top × bottom × shoes × (accent | layer if < 19 °C); dress × (layer) × shoes × accent
           // each slot limited to its 10 best-ranked pieces; fixedId → only combos containing it
  FOR o IN combos (minus looks rejected "Not in the mood today" this week):
      crit ← criteria(o): Complete · Colour harmony (≤ 2 accents, no clash) · Dress code · Weather fit · Style (+ modest)
      rule ← 100 − 17 × failed(crit)                                       // Strategy: rule scorer
      rank ← rule + Σ ranker(p) + (accent ? 2 : 0)
         ranker(p) = +4 favourite colour, +3 style match, −8 worn < 3 days, +6 forgotten (≥ 40 days),
                     +2 if ≥ 2 "Too formal" rejections and p suits a lower level (or "Too casual" → higher), +1 in a favourite look
  chosen ← 3 by rank with ≤ 2 shared pieces (first pass also varies the accent)
  FOR o IN chosen: breakdown(o) = {colour, style, formality, weather}; compatibility = mean
                   reasons ← explain(o) (3–5 lines); why ← factors(o); explanation ← sentence(why)
factors(o): Occasion ("Work calls for Business Casual; every main piece is tagged for it"), Style (first main piece
            in a preferred style), Modesty (if on), Weather (temperature, place, condition, layer), Availability
            (how many pieces are away), Wear history (forgotten piece or recycle time), Favourites (piece in a
            favourite look or favourite colour), Your feedback (repeated rejection reason)   // only facts actually used
sentence(why) = "Recommended because " + first three of [Style, Modesty, Weather, Wear history, Your feedback,
                Favourites, Occasion, Availability] joined with ";"
swap(outfit, slot): alternatives ← filterPieces ∩ same category, not already in the outfit; best by (rule, rank)
check(pieces, occasion): criteria, rule, breakdown, one-sentence mismatch, explanation paragraph, unavailable ids
```

Planned design: a learned scorer implementing the same Scorer interface; weather sensitivity of occasion templates used as a weight in the weather part score.

5.5 Closet Intelligence [Implemented · S3 preview; conflicts S2 preview]

```
health(user):                                                  // wardrobeHealth(), pure
  n ← pieces; IF n = 0 → {score 0, empty, action "Add your first pieces"}
  parts: Utilisation 35·worn30/n, Versatility 20·(≥2 dress codes)/n, Balance 20·(1 − gaps/(3·levels)),
         Readiness 15·Available/n, Basics 10·owned/7
  penalties: −min(10, forgotten), −min(10, potential duplicates)
  score ← clamp(Σ parts + Σ penalties, 0, 100); grade ← Excellent ≥ 80 · Good ≥ 65 · Fair ≥ 50 · Needs care
  actions ← up to 3 of: style forgotten pieces; "you may not need another …"; add a missing basic; wash day
detectPatterns(prefs, rejectCounts(60 days), favourites, wears(60 days), pieces):
  "Too formal" ≥ 2 and > "Too casual" and dressCode > 1 → change dressCode − 1   (and the mirror rule)
  a style in ≥ half (and ≥ 2) of favourite looks, missing in prefs → add style
  ≥ 70 % of worn pieces neutral and a worn neutral missing in prefs → add it
  ≥ 2 pieces of a non-neutral colour, none worn for 40 days → remove from favourites (or note only)
  confidence = evidence / (evidence + 2)
decide(user, key, confirm|dismiss): p ← current pattern(key) OR 404
  IF confirm: previous ← prefs[fields of p.change]; profile.update(p.change)
  UPSERT learned(key, state, {values: previous, text, effect})
undo(user, key): row ← DELETE learned(key) OR 404; IF row.state = confirmed → profile.update(row.previous.values)
conflicts(user):
  FOR event IN upcoming 30 days WITH planned outfit, FOR piece IN outfit:
      IF piece.status ≠ Available AND NOT (expectedBack ≤ event.date) → conflict(unavailable, message)
  FOR piece used by two events on one day < 4 h apart → conflict(double-booked)
  replacement ← IRecommend.swap(outfit, slot of piece, event.occasion, event.dressCode, event.date)
fix(user, event, item): c ← current unavailable conflict OR 409; r ← replacement OR 409;
  planner.plan(event, r.outfit)   // only that slot differs
notifications(user):  // computed, never stored
  list ← conflicts(5) + plan-tomorrow(4) + packing within 3 days not packed(4) + borrowed due(3) + rain for an event(3)
         + laundry day tomorrow(2) + Wash done in 24 h(1) + longest forgotten piece(1)   // (priority)
  mark read from notifications_read; sort unread, priority, date; highlight first 3 unread (NFR-13)
```

5.6 Smart Shopping and the Shopping List [Implemented · S3 preview; list v1.4]

The shopping bag is one table, `shopping.list_items`, fed by five sources. Every item says where it came from (label) and why (reason).

```
SOURCE_LABEL = {ask: "Added by Ask Closet", replacement: "Replacement", gap: "Wardrobe gap",
                suggestion: "Smart suggestion", manual: "Added by you"}
list(user): rows ← list_items of user, newest first
  FOR r: similar ← similarOwned(pieces, wantFor(prefs, r.category, r.colour))  (similarity ≥ 70, top 3)
         tags ← [SOURCE_LABEL[r.source]] + ("You already own similar item" IF similar AND r.replaces = null)
         warning ← IF tags has it → "You already own N similar …"   // a replacement never warns about the piece it replaces
addToList(user, {title, category, colour, source, reason, suggestionKey, replaces}):
  IF same title / category / colour already to_buy → RETURN {item, added: false}  // no duplicates
  INSERT list_items;  IF suggestionKey → decisions[key] ← saved
  note ← IF (category, colour) was let go as "No longer my style" → "You let go of a … before"
  RETURN {item, added: true, note}
onDeleted(user, deletion):            // Observer of ItemDeleted (5.14)
  r ← DeletionPolicy.replacementFor(deletion);  IF r → addToList(user, r)   // Torn / damaged, Became too small only
suggestions(user):
  Gap   ← closetGaps at levels the user dresses for; colour = a liked neutral NOT let go of
  Basic ← 7 capsule basics not owned
  FOR s: outfitsUnlocked ← number of rule-score ≥ 83 outfits a hypothetical piece makes with Available pieces
  keep s IF not decided AND outfitsUnlocked > 0 AND NOT isAvoided(deletions, s)
         AND NOT every style of s ∈ prefs.avoidStyles          // learned let-go (5.14)
  sort by priority (gap > basic) then outfitsUnlocked
similarity(owned, want): IF category differs → 0
  25 + (same colour 35 | both neutral 15) + 15·|styles ∩|/|styles ∪| + 15·|levels ∩|/|levels ∪| + (season compatible 10)
check(user, {category, colour, styles, season}): similar ← owned with similarity ≥ 70
  verdict: ≥ 2 similar → Skip · removed as "no longer my style" → Think twice · ≥ 6 outfits → Good buy
           · 1–5 → Optional · 0 → Not yet;  duplicate.message ← "You already own N similar …"
```

5.7 Packing [Implemented · stretch preview]

```
plan(user, {destination, startDate, endDate ≤ start + 13, tripType, activities}):
  FOR d IN trip days: occasion ← planned event that day OR activities[i mod |activities|]
                      weather ← IWeather.forDate(destination, d)
  planTrip: pieces ← Available (wear history ignored); FOR each day: r ← engine.recommend(…, repeatDays = 0)
       choose the look sharing most pieces with the suitcase (re-use), tie → rule score; none → missing day
  warnings: rain and no outer layer → "waterproof outer layer"; min < 19 °C and no layer; max ≥ 26 °C; estimated forecast
pack(user, trip): FOR piece IN trip: Available → setStatus(Packed, until = endDate + 1, note = destination)
                  Packed → keep; other status → skipped with reason;  trip.packed ← true
unpack(user, trip): Packed pieces → Available
```

5.8 Ask Closet [Implemented · S3 preview]

```
chat(user, message, context):
  {fn, args, via} ← IIntentParser.parse(message)      // LLM tool call (14 whitelisted
functions) → else keyword parser
  IF fn ∉ FUNCTIONS → help
  result ← FUNCTIONS[fn](user, args, context)         // only through module interfaces;
never raw SQL
  IF result changed data → INSERT assistant.actions(fn, undo data); result.undoId ← id
  RETURN {reply, function: fn, via, outfits | pieces | shopping, undoId}
undo(user, id): action ← own, not undone → apply action.undo (restore statuses / unhide
…); mark undone
```

5.9 Planner and Calendar Adapter [Implemented · S1; import S2, planned outfits S3 preview]

```
parseIcs(text): unfold lines; FOR VEVENT: SUMMARY (unescape) and DTSTART
  UTC "…Z" → convert to Africa/Cairo; VALUE=DATE → all-day 09:00
import(user, ics): keep future events ≤ 1 year ahead, not already present (title + date);
occasion ← guess(title)
calendar(user, month): events + markers {laundry-back, borrowed-due, packed-back,
unavailable-back} from expectedBack
                       + laundry-day on each matching weekday
```

5.10 Weather Adapter [Implemented · S2 preview]

```
forDate(city, date):
  IF no city → estimate("No city in your profile")
  TRY  geo ← cache[city] (30 min) OR fetch geocoding (4 s) → forecast 16 days (4 s)
       i ← index of date; IF none → estimate("More than 16 days ahead")
       RETURN {tempC (current if today, else max), high, low, condition, place, source:
Open-Meteo}
  CATCH → estimate("Weather service unavailable")    // Cairo monthly average, labelled
"estimate"
```

5.11 Error handling [Implemented · S1]

```
errorHandler(err): AppError → {status, code, message, details}; bad JSON → 400; too
large → 413
                   otherwise → count serverErrors (admin health); 500 "Something went
wrong. Please try again."
```

5.13 Core flow — three looks → change a piece → try on → feedback [Implemented · S2 preview; try-on S3 preview]

The heart of Closet (SRS Figure 2-2, FR-75, FR-76). The Style me screen and the dressing room share one look store, so the user can flip between the same three looks and return with a changed piece.

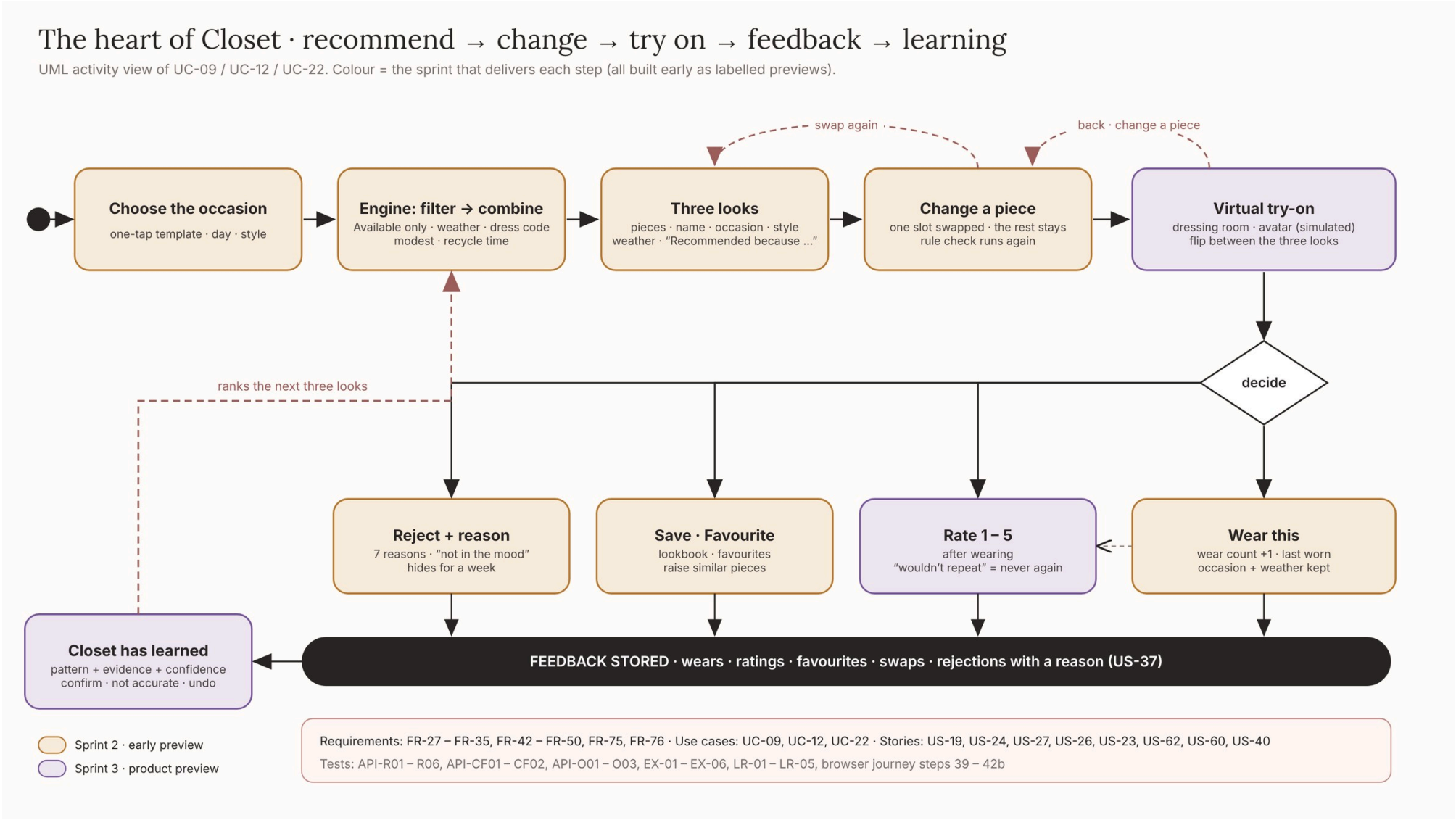


Figure 5-1 – Core flow (activity): occasion → 3 looks → change → try-on → wear / save / favourite / reject / rate.

```
StyleMe.generate(occasion, day, [template]):
  show "Reading your closet… / Checking the weather… / Building from N available pieces…"
(≥ 1.6 s)
  r ← POST /recommendations {occasion, style, level, date}         // 5.4: exactly 3
looks, ≤ 2 shared pieces
  names ← lookNames(looks): occasion word / main style + the noun of the piece that makes
the look
          different (Stripes, Linen, Tailoring…); the first unused combination → never
two alike
  show 3 cards (arriving one by one): name, pieces, occasion, dress code, weather, match
%, one reason
  lookStore.save({occasion, levelName, tempC, result: r, changed: false})   //
sessionStorage "closet.looks"
StyleMe.change(look i, slot):                                      // click the garment
  r ← POST /recommendations/swap {itemIds, slot, occasion, date}
      options ← ≤ 3 Available pieces of that category (exact dress code first; one level
away
                only if fewer than 3), each with its re-scored look
  tray "Replace bottom": each option shows old % → new %
  user picks one → look i ← option.outfit (name kept); the new piece animates in; the
match %
  counts from the old value to the new; toast "X → Y · 89% → 92%. The rest of the look
stays."
  lookStore.save(..., changed: true);  outfits.feedback ← swap (server)
TryOn.open(ids, lookIndex):
  looks ← lookStore.result.outfits with look i's ids (keeps the changed piece)
  IF the user chose a photo → OnPhoto(photo, pieces, fit)            // 5.18, never
uploaded
  ELSE upload card "See this look on you" (or the stylised avatar on request)
  previous / next / dots flip between the 3 looks; "Back to my looks" → /outfits?back=1
restores the store
Feedback (from a card or the dressing room):
  Wear → POST /outfits/wear (occasion, weather) → wear counts; rating 1–5 → PATCH
/outfits/wears/:id
  Save / Favourite → POST /outfits (favourite feeds the ranker +1)
  Not for me → reason (7) → POST /outfits/reject → feedback; "Not in the mood today"
hides for 7 days
```

5.14 Deletion-reason handling and learning [Implemented · S3 preview (v1.4)]

A deleted piece must have one of four reasons. One pure table, **DeletionPolicy**, says what each reason does (SRS BR-10); Wardrobe publishes **ItemDeleted** and Smart Shopping reacts (Observer), so Wardrobe never calls shopping code.

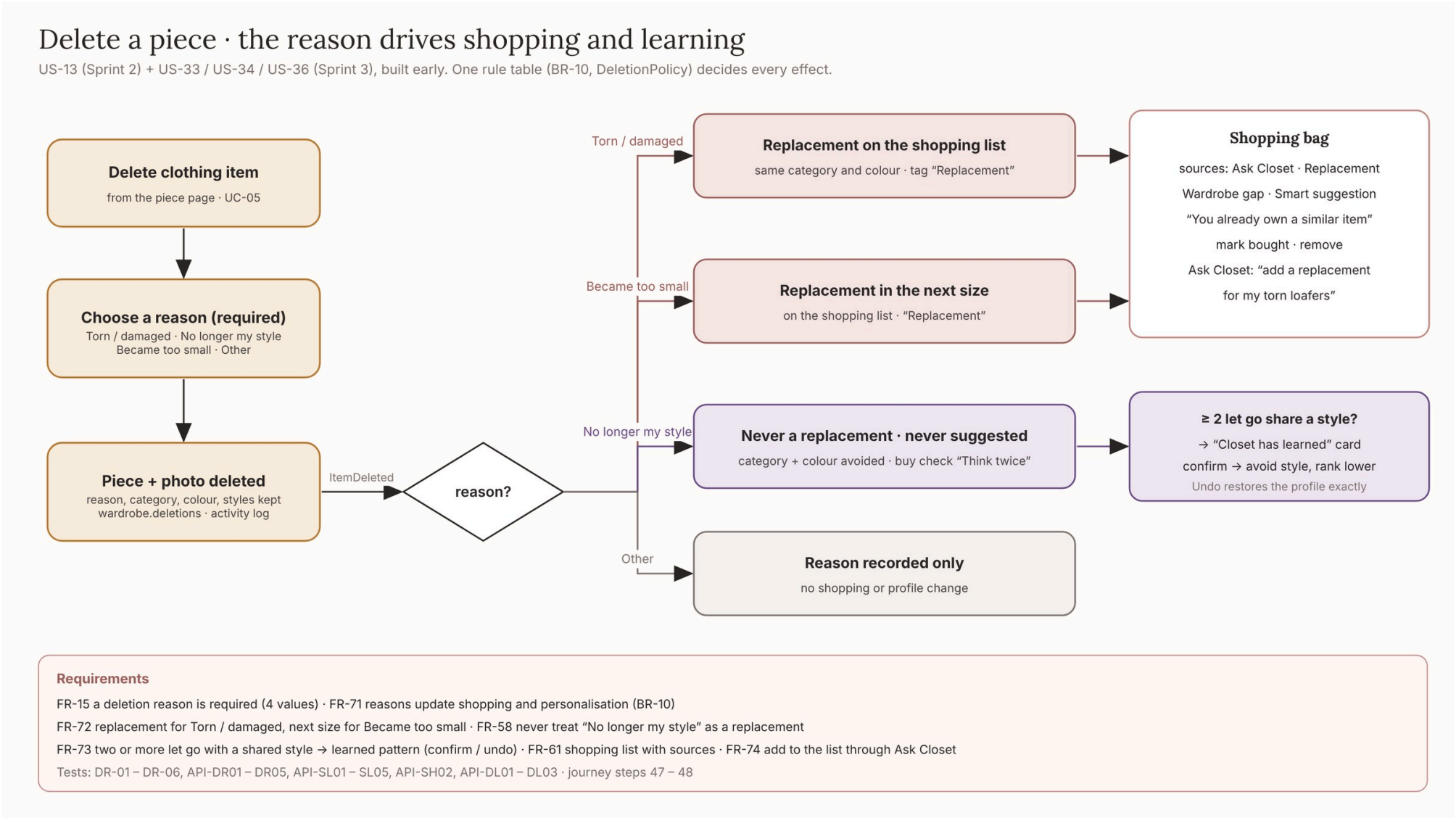


Figure 5-2 – Deletion reasons and their effects.

```
EFFECTS = { "Torn / damaged":     {replacement: yes, learn: no},
            "Became too small":   {replacement: yes (next size), learn: no},
            "No longer my style": {replacement: NO, learn: yes, avoid: yes},
            "Other":              {replacement: no, learn: no} }        // reason kept
only
Wardrobe.deletePiece(user, id, {reason, note}):
  validate reason ∈ DELETE_REASONS (else 400);  piece ← own piece (else 404)
  INSERT deletions(name, category, colour, wear_count, styles, dress_codes, fabric,
reason, note)
  DELETE item + photo
  reactions ← bus.publish(ItemDeleted, {userId, deletion})            // 5.6 onDeleted
  RETURN {deleted, reason, effect: EFFECTS[reason].effect, addedToShoppingList:
reactions.listItem | null}
replacementFor(d): IF EFFECTS[d.reason].replacement →
  {title: Name (+ " · next size"), category, colour, source: replacement, replaces: name,
reason}
isAvoided(deletions, s) ← some "No longer my style" deletion with the same category AND
colour
letGoStyles(deletions) ← styles shared by ≥ 2 "No longer my style" deletions
Learning (5.5): detectPatterns adds "let-go-<style>" (source "Deleted pieces", evidence,
confidence)
  Confirm → profile.avoidStyles += style, style removed from preferred styles; previous
values kept for Undo
  Effect: engine ranker −3 per piece in an avoided style; "Your feedback" factor in the
explanation;
          suggestions in only that style are never shown; buying the same category +
colour shows a note.
```

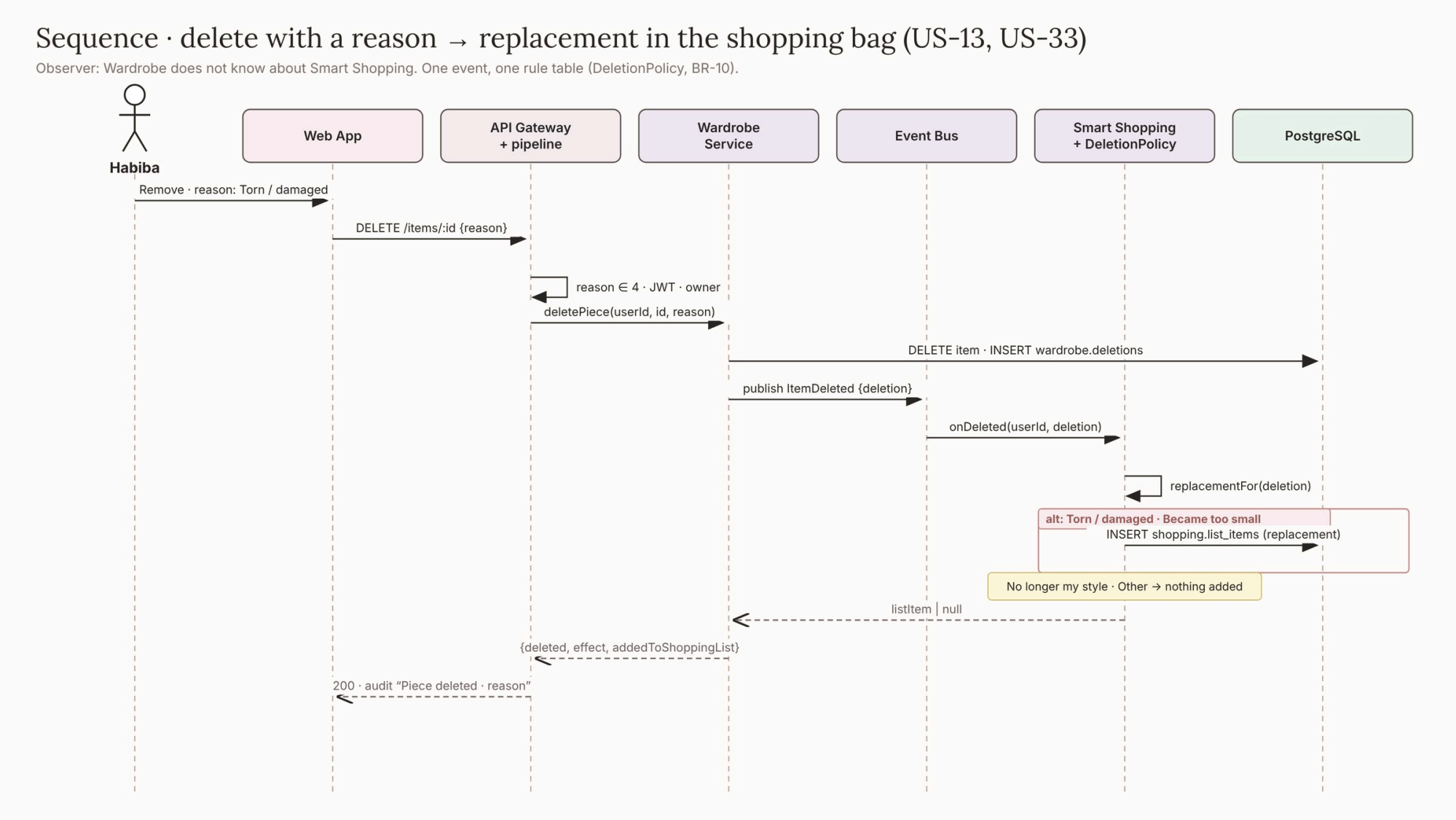


Figure 5-3 – Sequence: delete with reason → ItemDeleted → replacement on the shopping list.

5.15 Ask Closet → shopping list [Implemented · S3 preview (v1.4)]

```
parse("put beige trousers in my shopping list")  → fn add_to_shopping, piece {Bottom, Beige}, word "trousers"
parse("add a replacement for my torn loafers")   → fn add_to_shopping, replacement, damaged, word "loafers"
parse("add this to my shopping list")            → fn add_to_shopping, useContext   // after a buy check
add_to_shopping(user, a, ctx):
  q ← a.piece OR (a.useContext AND ctx.buy)          // ctx.buy is set by buy_check, validated against the vocabulary
  IF a.replacement: p ← owned piece matching colour / category / word → replaces ← its name, reason "torn or damaged"
  IF q = null → ask for the piece
  r ← IShopping.addToList(user, {title, category, colour, source: ask, reason, replaces})
  IF NOT r.added → "already on your shopping list"
  undoId ← assistant.actions(add_to_shopping, {type: shopping, itemId})   // Undo removes the item
  RETURN reply + listItem (labels, warning) + similar owned pieces
```

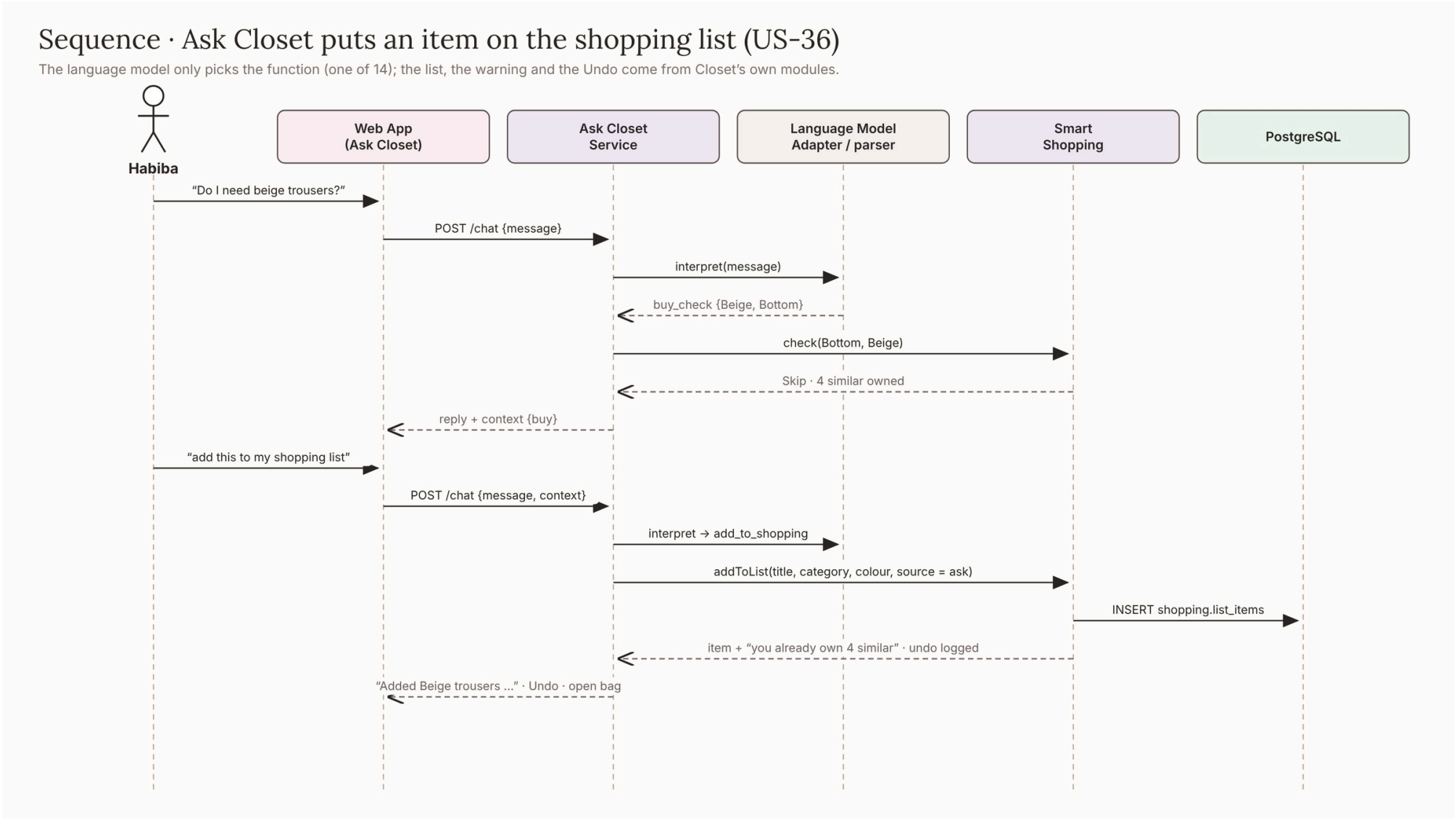


Figure 5-4 – Sequence: Ask Closet adds an item to the shopping list.

5.16 Interaction design of the environments [Implemented · S1 screens; S2/S3 previews labelled]

Environment	Design	Why
Closet — dressing room	Rails per category inside a boutique wall with an LED strip; each piece shows its status chip	One glance shows what is clean and wearable
Laundry room	Drawn utility room (SVG): tiles, window, shelf, washing machine whose drum spins while washing, rail of clean shirts, ironing board with steam; “Wash done” returns pieces	Status changes feel physical and visible
Style me	Step rail (Occasion → Three looks → Change → Try on → Wear · save · rate); occasion tiles; three flip cards — front = the look and actions, back = why (factors, part scores, rule check)	Progressive disclosure: explanation on demand
Dressing room (try-on)	Stage with curtains and spotlight; the user’s photo (or the avatar) centred in an arched vanity mirror with LED bulbs; fitting card with the pieces on hangers; “Simulated try-on” chip	Try-on as a moment, honest about being simulated
Shopping bag	A bag with handles; each item has its label (Added by Ask Closet, Replacement, Wardrobe gap, You already own similar item), reason, bought / remove; suggestions beside it	The list looks intentional; every item explains itself
Closet has learned	Flip card: front “Closet has learned from your …”, back = insight, evidence, confidence, effect, Confirm / Not accurate	Less text until the user asks
Delete a piece	Four reason cards, each stating its effect before the user confirms	The user knows what will happen

5.17 Style a forgotten piece; Mix & Match and Suggest a fix [Implemented · S2/S3 preview (final pass)]

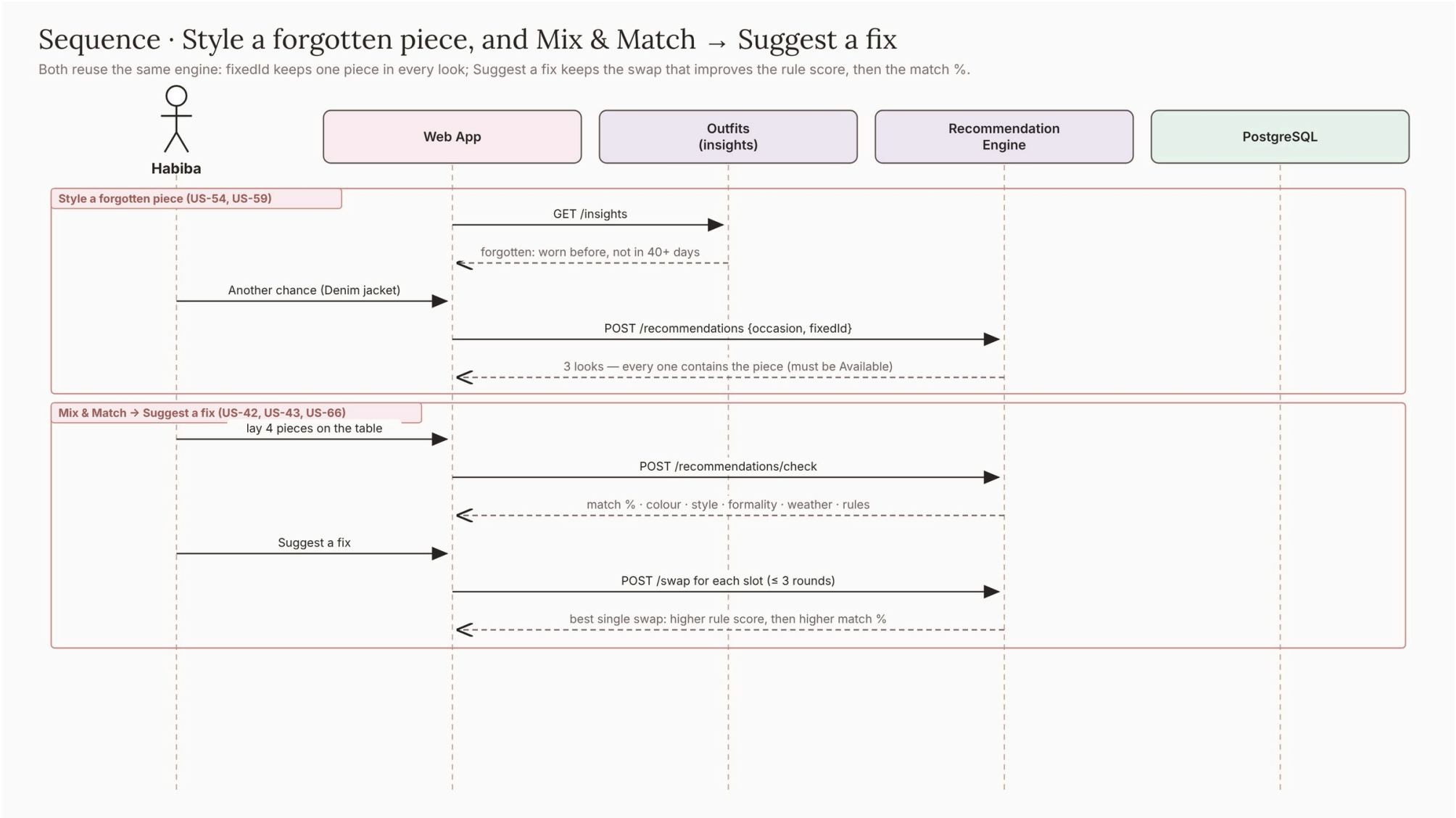


Figure 5-5 – Sequence: style a forgotten piece; Mix & Match → Suggest a fix.

```
 Forgotten pieces (Outfits.insights): p.wearCount > 0 AND days since p.lastWornAt ≥ 40,
sorted by days
recommend(pieces, {occasion, fixedId}):                          // the fixed-item path
  fixed ← piece fixedId;  IF fixed.status ≠ Available → RETURN {outfits: [], note: "not
Available"}
  pool ← filterPieces(...);  IF fixed ∉ pool → pool += fixed     // weather / recycle
time never remove it (D-06)
  combine(pool, fixedId): every slot list that can hold the fixed piece is reduced to
[fixed];
         a fixed outerwear piece may complete top + bottom + shoes (4 pieces)
  keep only combos containing fixed → score → rank → choose 3 → names (5.13)
Mix & Match: check(pieces, occasion) → criteria (5 rules) · rule = 100 − 17 × failed
  parts: colour (clash 40 · > 2 accents 55 · 2 accents 88 · 1 accent 96 · neutral 92),
style (share of
  main pieces in the target styles), formality (exact level 1, one level away 0.5),
weather (warm piece
  ≥ 26 °C, no layer < 19 °C);  match % = mean of the four parts
Suggest a fix (client, ≤ 3 rounds):
  REPEAT: FOR each slot i: r_i ← POST /swap {itemIds, slot: i}
          best ← argmax over i of (r_i.ruleScore, then r_i.compatibility)
          IF best does not beat the current look → STOP
          replace that one piece; record "old → new"
  show "82% → 98% · red cropped tee → ivory silk blouse"
```

5.18 Virtual Try-On Preview on the user's photo, and its privacy [Implemented · S3 preview (final pass); photo-real adapter Planned design]

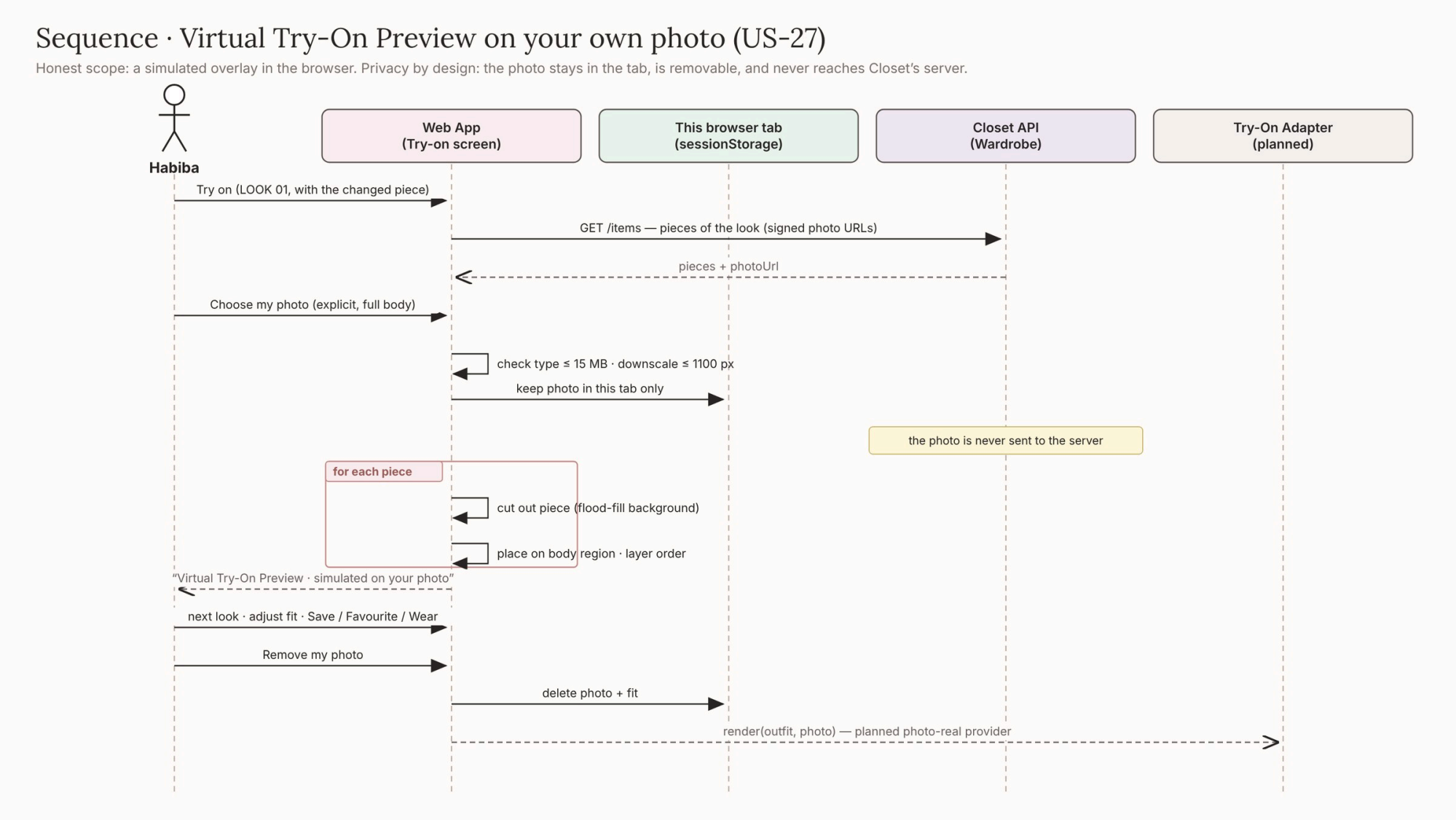


Figure 5-6 – Sequence: Virtual Try-On Preview on the user's own photo (the photo never reaches the server).

```
storePhoto(file):                                   // only after the user explicitly
chooses a photo
  require type ∈ {jpeg, png, webp} AND size ≤ 15 MB
  downscale to ≤ 1100 px tall (canvas) → JPEG data URL →
sessionStorage["closet.tryonPhoto"]
cutout(pieceUrl):                                   // each garment's own photo, cached
per URL
  draw ≤ 520 px · background colour ← mean of the border pixels
  flood-fill from the borders: pixels within distance 34 of the background → transparent
  soften a 1-px edge · crop to the garment
OnPhoto(photo, pieces, fit):
  body box: height 0.9 × size, top 5 % + shift, centre x; shoulder span 0.27 × body
height
  region(piece): top / dress / outerwear / bottom (skirt or trousers) / shoes / bag /
sunglasses,
                 earrings, scarf → top, width, centre, layer order (z), maximum height
  every piece → an <img> at its region; label "Virtual Try-On Preview · simulated on your
photo"
Privacy and retention:
  the photo is never part of any API request (checked by the browser journey) · kept only
in this
  tab's sessionStorage → gone when the tab closes · "Remove my photo" deletes photo and
fit at once
  · not shown to the Admin (it never reaches the server) · no faces are processed or
stored
Planned: ITryOn.render(outfit, photo) behind the Try-On Adapter for a photo-real
provider, with this
preview as the fallback when the provider is unavailable (FR-42).
```

5.19 Planned designs (not built)

Component	Design	Requirement
Notification Adapter	INotifier.send(user, notice) for the 3 highlighted notices a day, at the user’s reminder hour; push or e-mail provider behind the adapter (TBD-4)	US-67, US-74
Try-On Adapter	ITryOn.render(outfit, userPhoto) → image from a photo-real provider (explicit consent, no storage); the browser preview on the user’s photo remains the fallback	US-30
Vision Adapter	IVisionModel.suggest(photo) → tags with confidence; only shown above a threshold; nothing saved without confirmation	US-10
Live calendar sync	ICalendar.sync(user) read-only for the next 30 days through the provider’s API	US-50
Learned scorer	A Scorer trained on the user’s feedback replacing the rule scorer behind the same interface; the rule score stays shown	E7

6. HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

The user reaches Closet in a phone or laptop browser. On wide screens a left navigation groups the product — **Today** (Home), **Wardrobe** (Closet, Laundry), **Style** (Recommendations, Mix & Match, Dressing room, Lookbook), **Assistant** (Ask Closet), **Plan** (Planner, Packing), **Insights** (Analytics & insights, Smart shopping) and **You** (profile and settings, Admin for admins); a coloured dot shows each feature's sprint. On phones a bottom bar holds Home, Outfits, Ask, Closet and More. A bell opens the Notification Center from every screen.

Design rules. One screen = one primary goal. Each screen uses a physical metaphor: the closet as a dressing room with rails per category and an LED strip; laundry as a utility room with a washing machine, a rail and an ironing board and a visible "Wash done" moment; Style me as a step rail with three flip cards (the look in front, the reasons behind); Mix & Match as a styling table with a large verdict; try-on as a stage with curtains and an LED vanity mirror; the shopping list as a shopping bag with labelled items; saved looks as a lookbook; packing as a suitcase; analytics as an editorial page. Outfits are shown as layered flat lays, not grids. Depth is subtle (layered cards, soft shadows, perspective on hover for whole-card links, 0.2 – 0.5 s motion; reduced motion respected). Palette: warm ivory, charcoal, muted rose, taupe, sage for success.

Feedback to the user. Every change confirms with a short message (and Undo where it can be undone); every list has an empty state; loading shows skeletons; errors appear next to the field in plain words; every score is labelled and can be expanded ("how the match % is computed"). Every screen states its sprint ("Sprint 1 · Implemented", "Sprint 2 · Early preview", "Sprint 3 · Product preview", "Stretch · Concept preview").

6.2 Screen Images

Screenshots from the automated browser journey on the seeded demo account (1440 × 900).

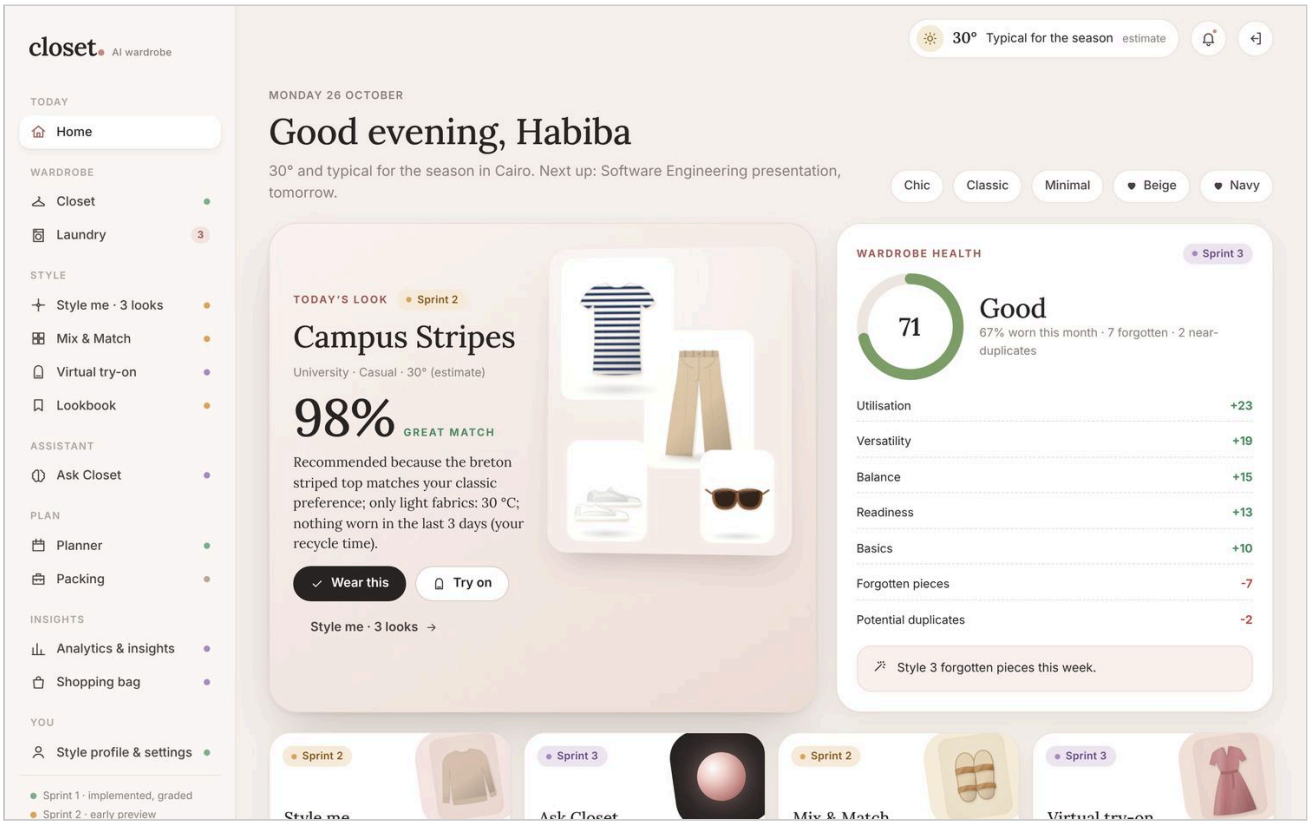


Figure 6-1 – Home — today's look, Wardrobe Health, the forgotten piece (S2/S3 preview).

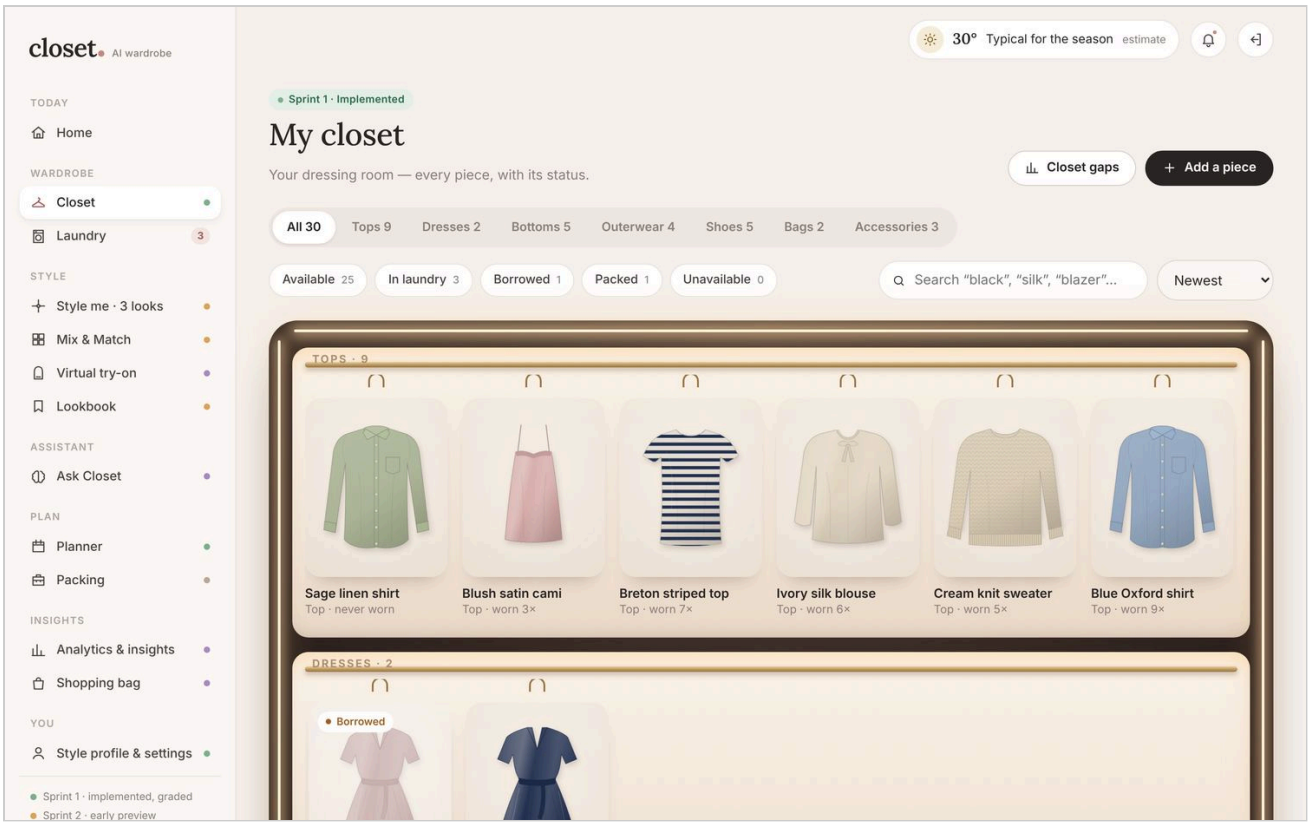


Figure 6-2 – Closet — the dressing room: lit rails, shelves, a status on every piece (S1).

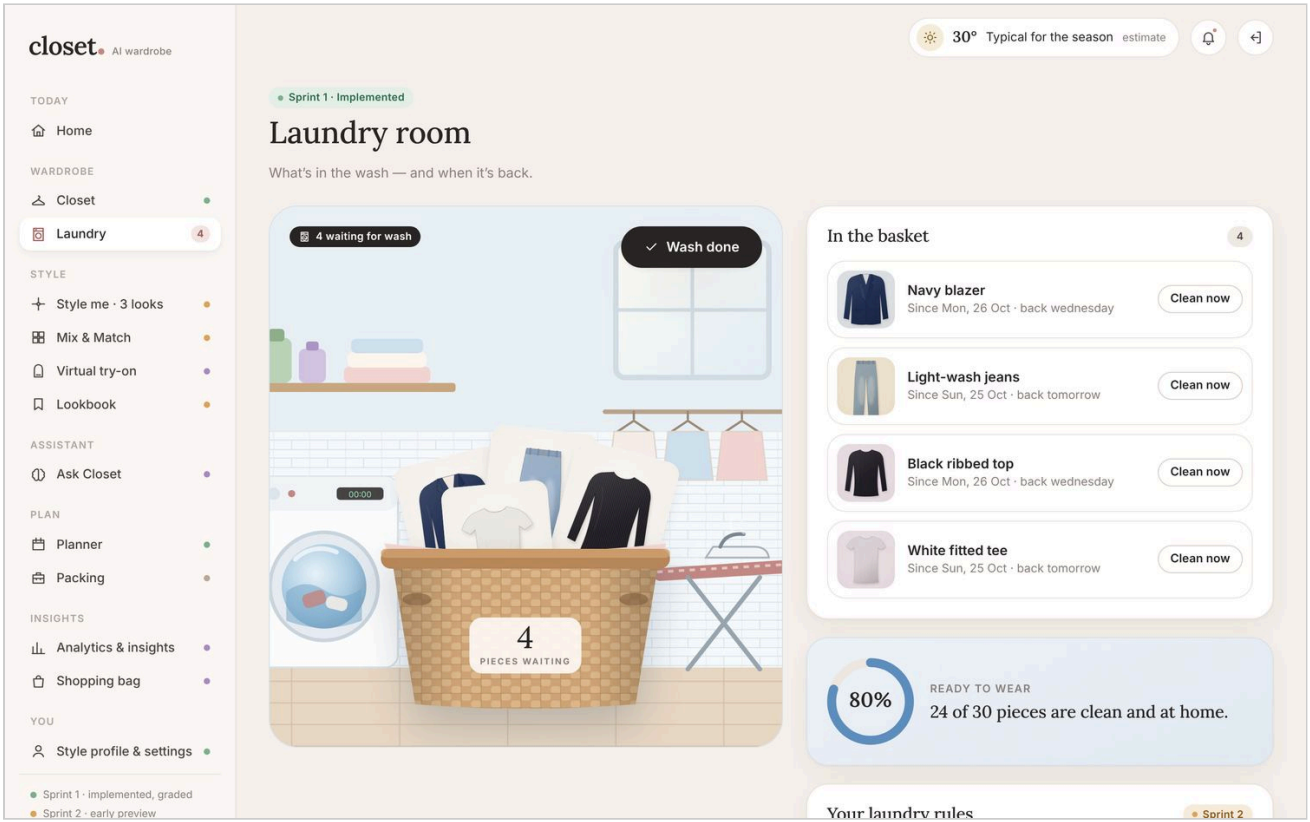


Figure 6-3 – Laundry room — the basket, washing machine and ironing board (S1).

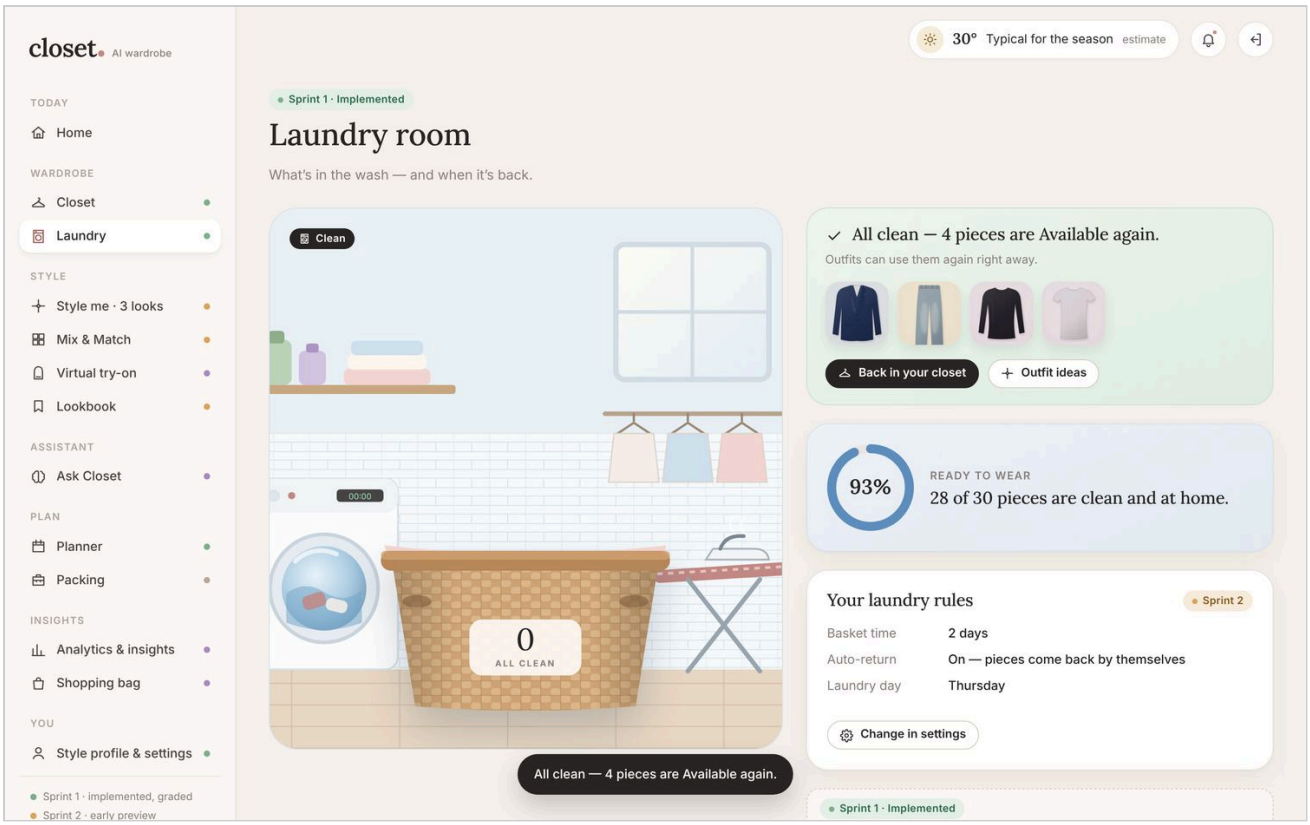


Figure 6-4 – Laundry room — after Wash done (S1).

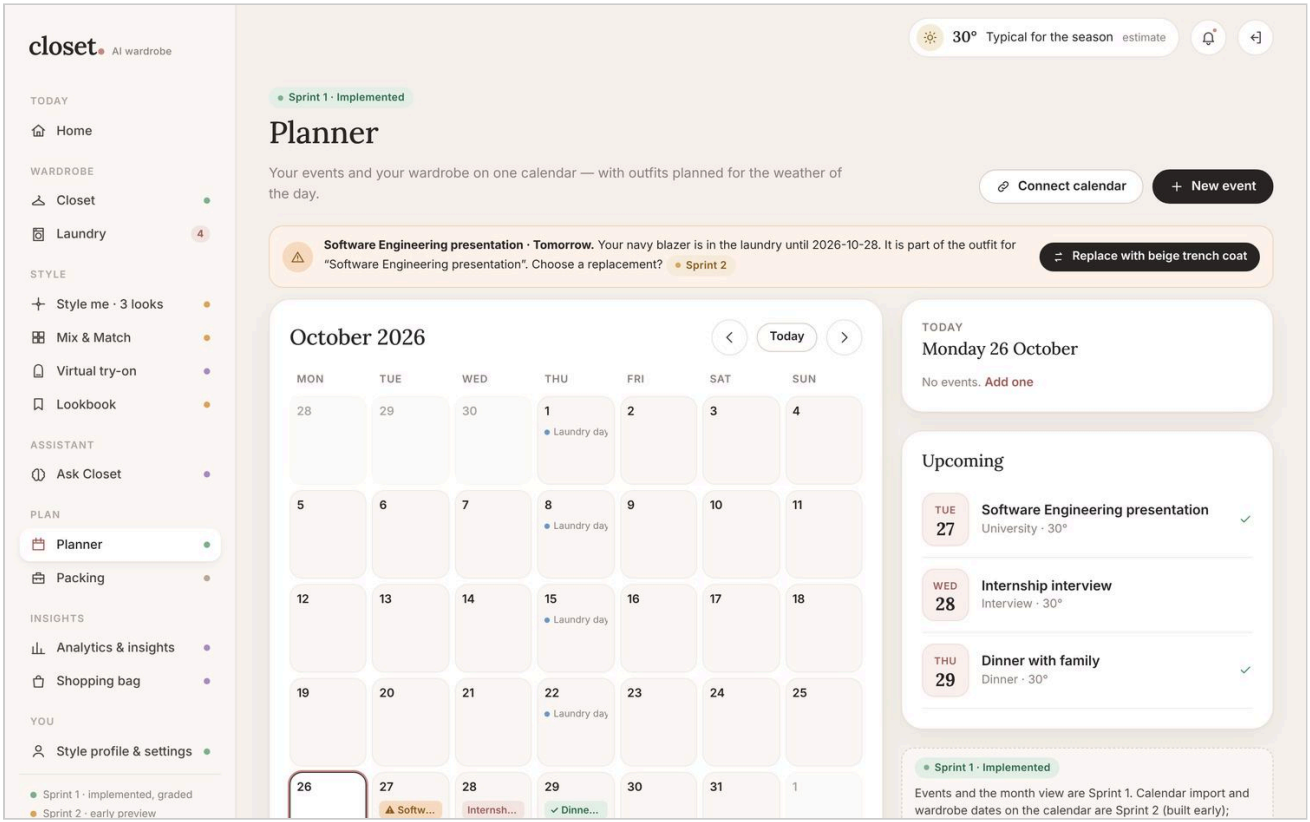


Figure 6-5 – Planner — the planned blazer is in the laundry: one-tap replacement (S2 preview).

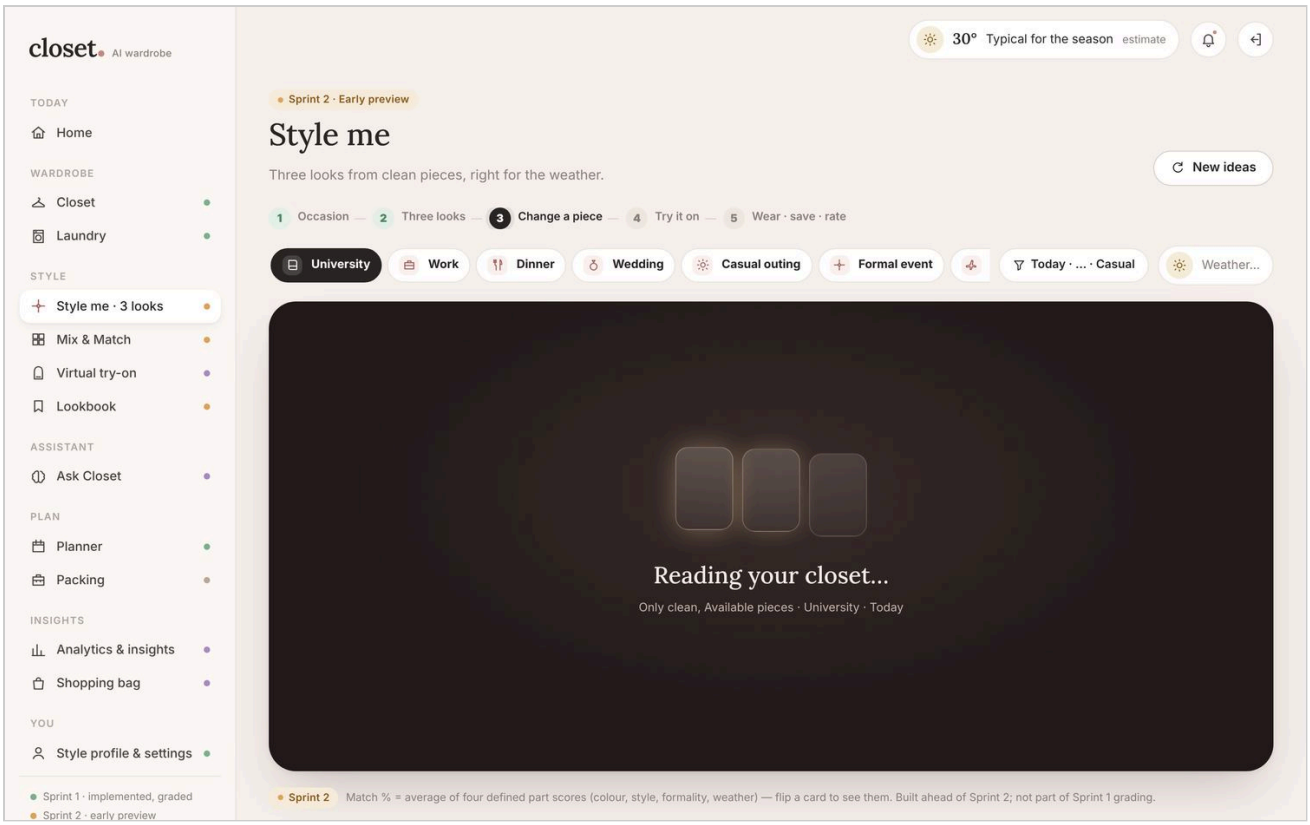


Figure 6-6 – Style me — generating: what Closet reads before the three looks (S2 preview).

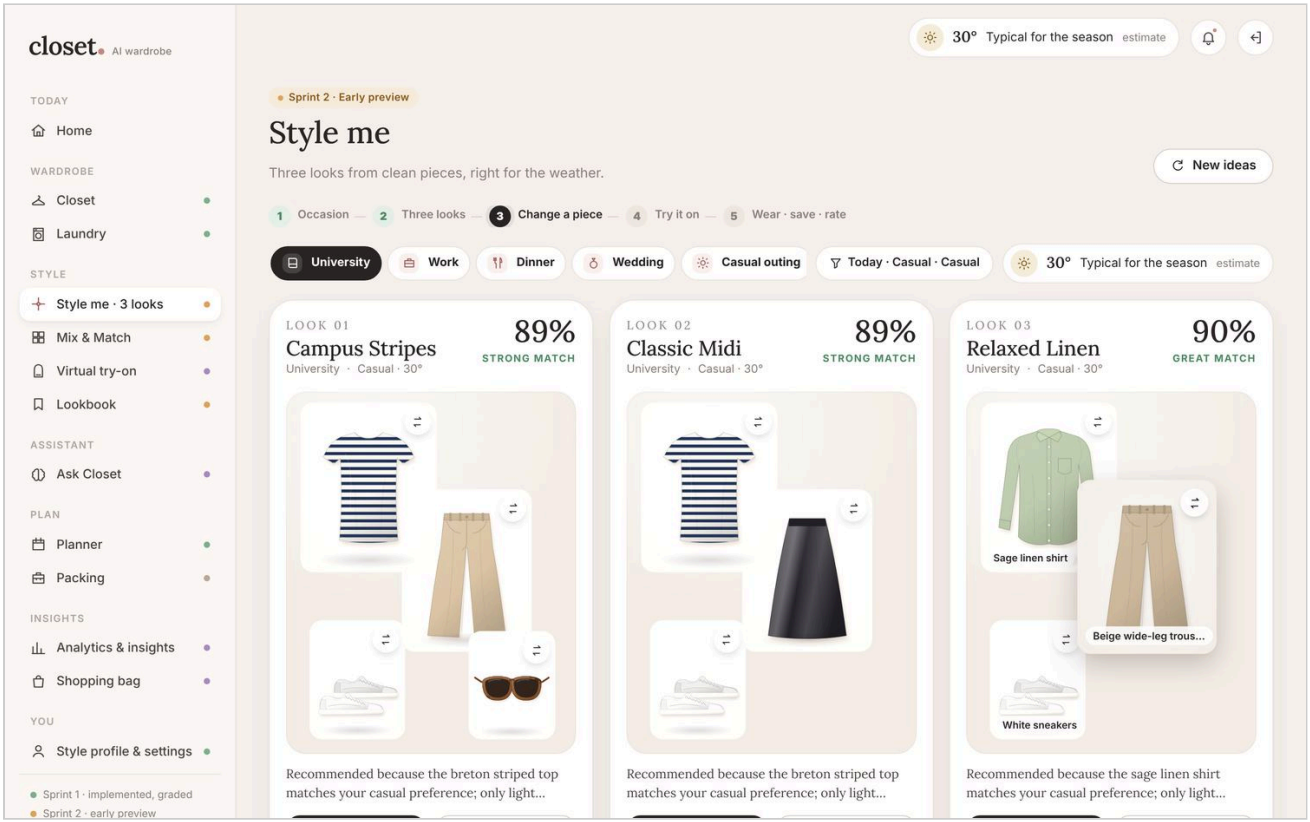


Figure 6-7 – Style me — three looks, each with its own name (S2 preview).

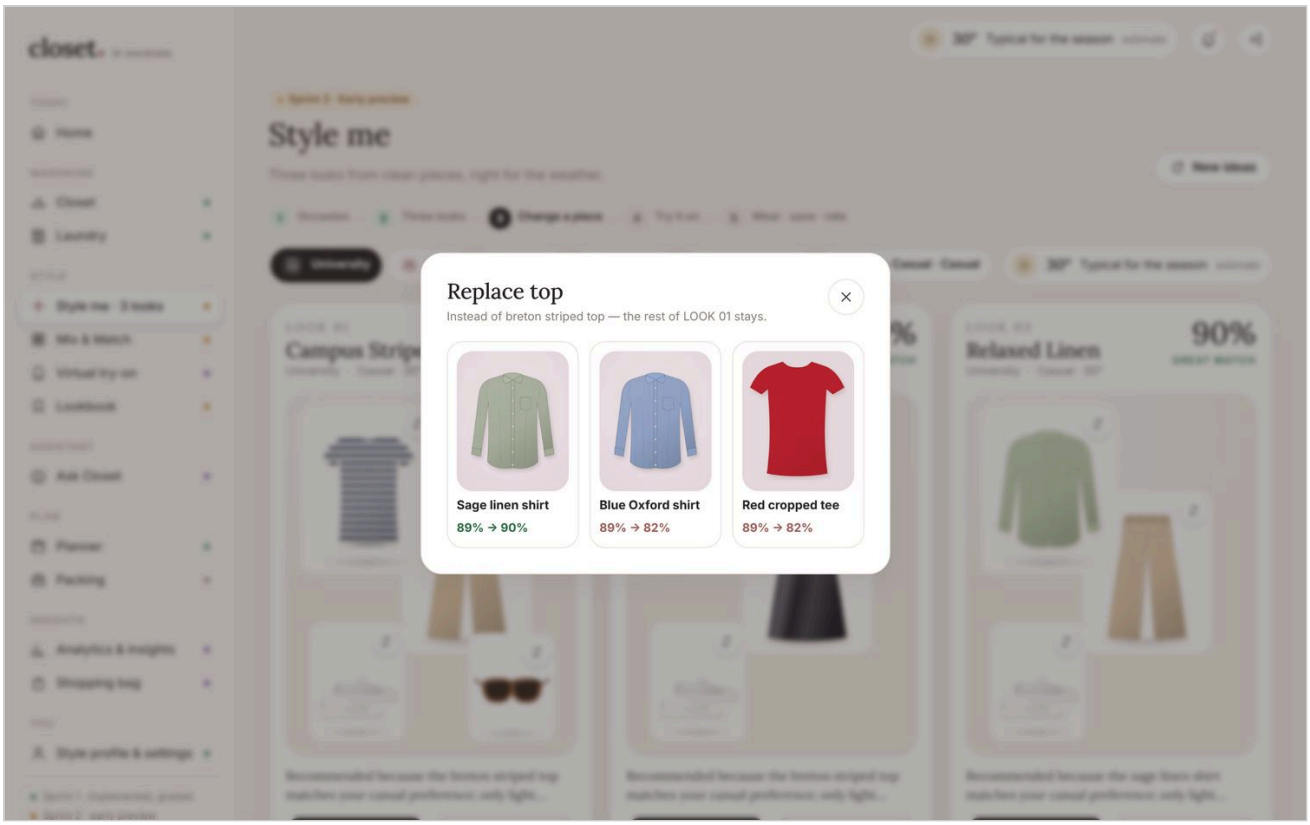


Figure 6-8 – Change one piece — the tray with real alternatives and their new match % (S2 preview).

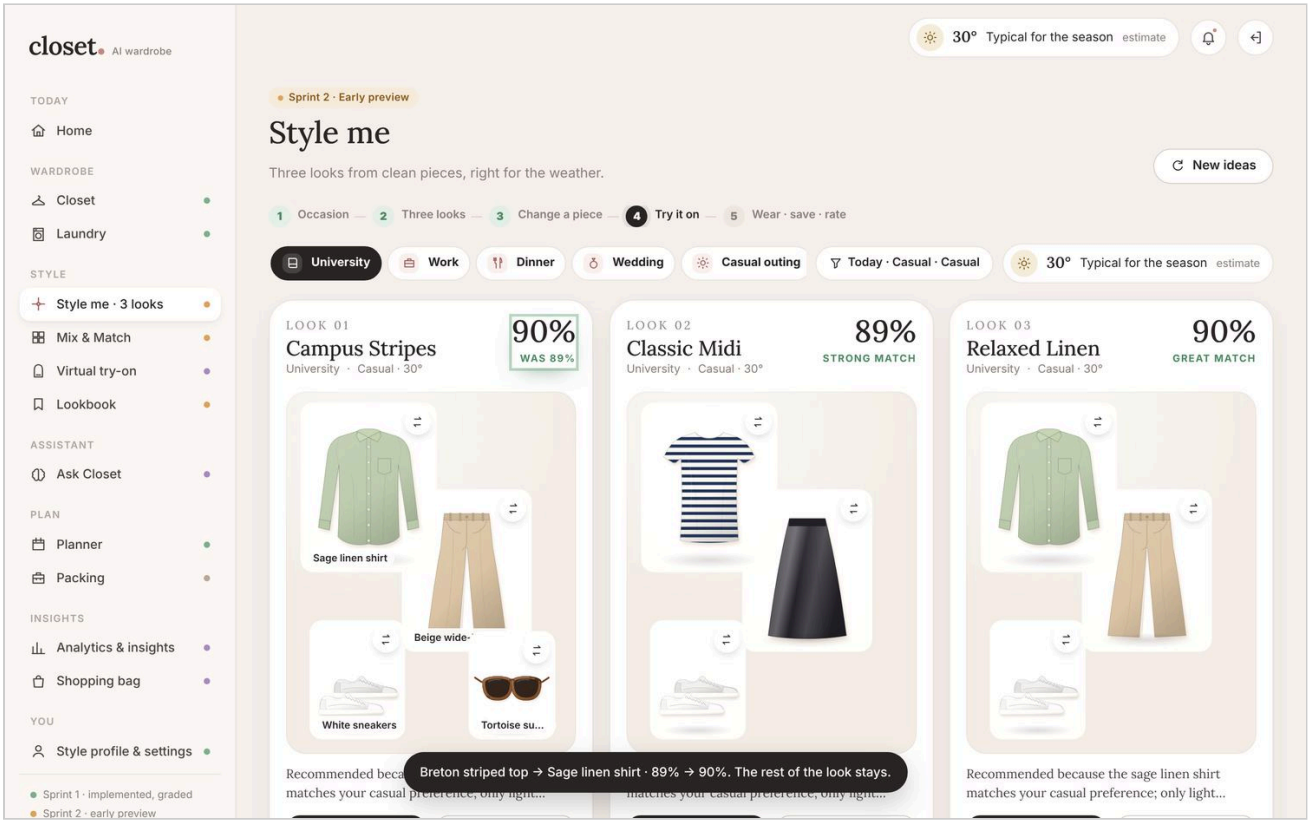


Figure 6-9 – The changed look: the new piece slides in, the score moves (S2 preview).

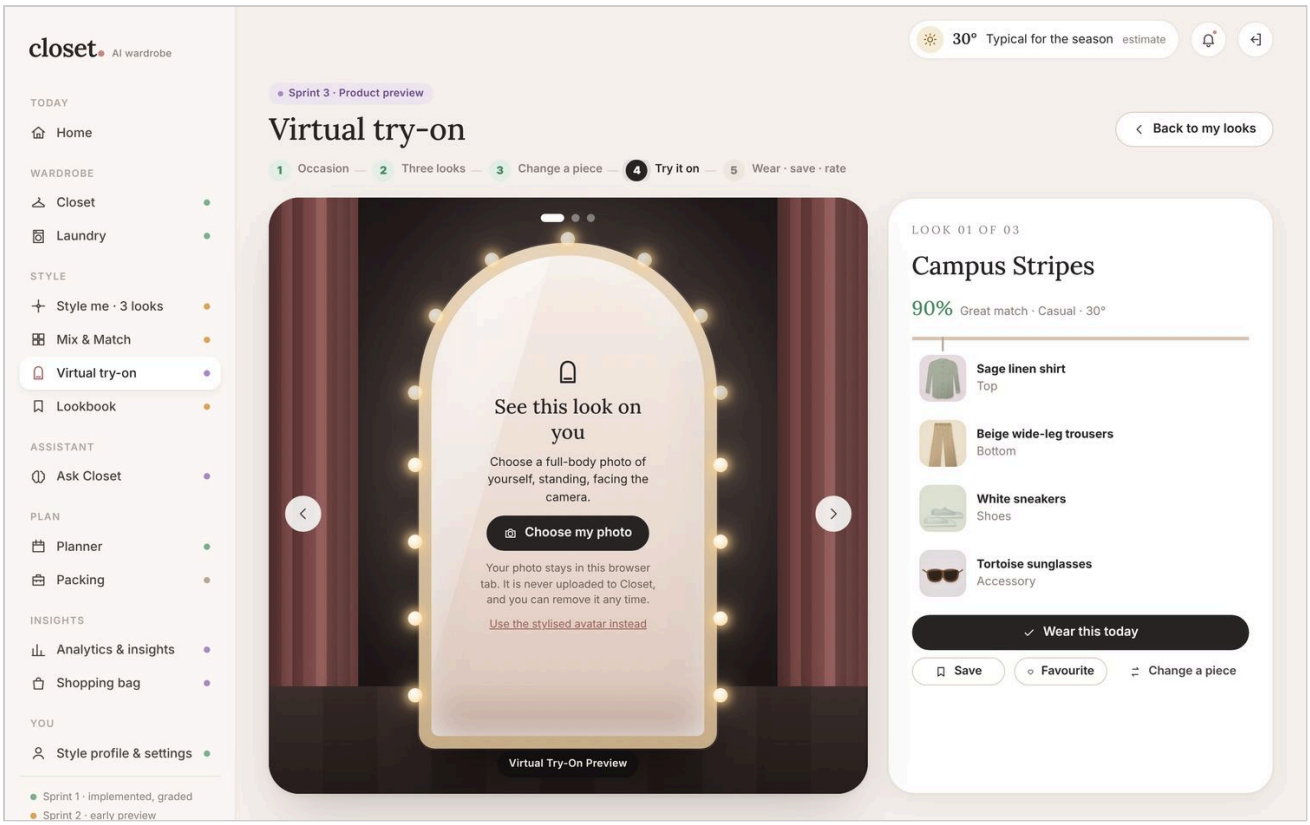


Figure 6-10 – Virtual try-on — choose your photo; it stays in this tab (S3 preview).

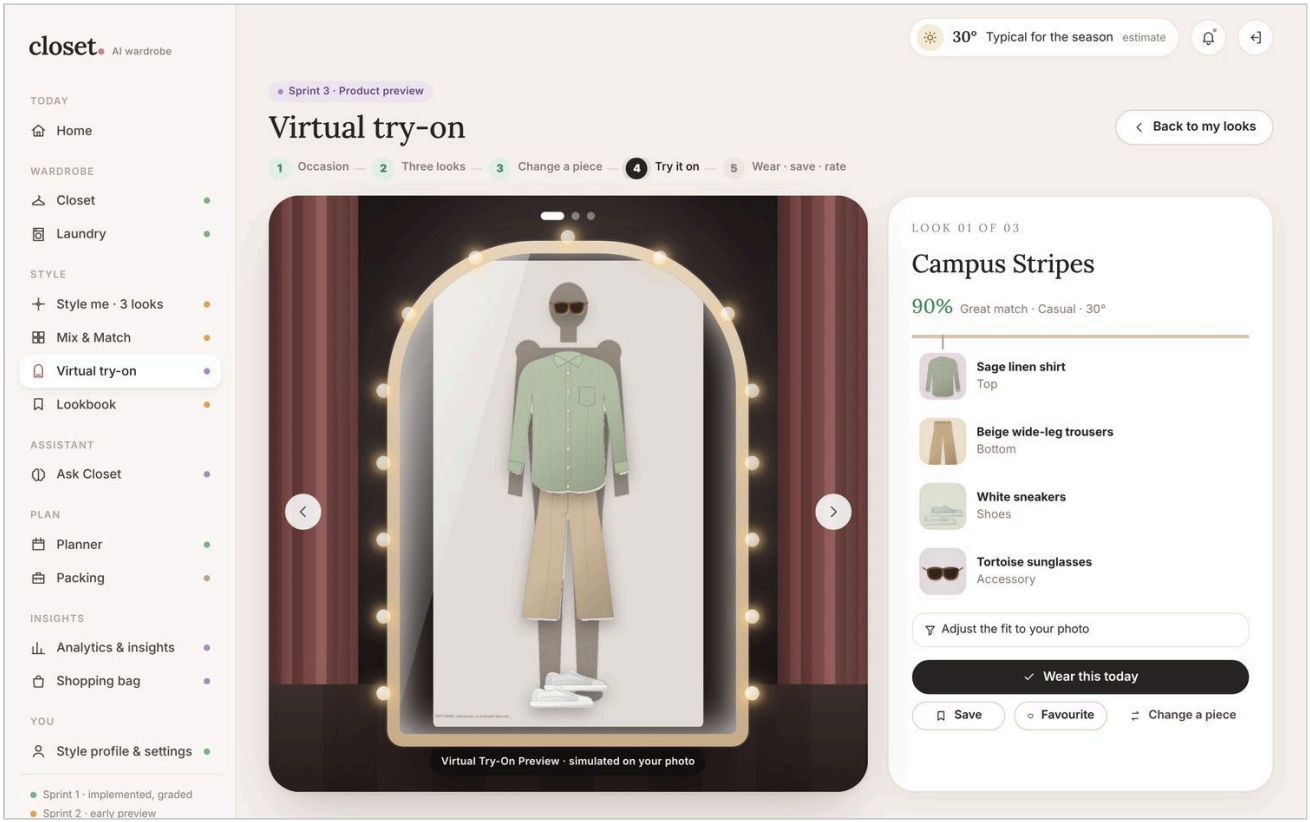


Figure 6-11 – Virtual Try-On Preview on a photo — a neutral test figure in the automated run (S3 preview).

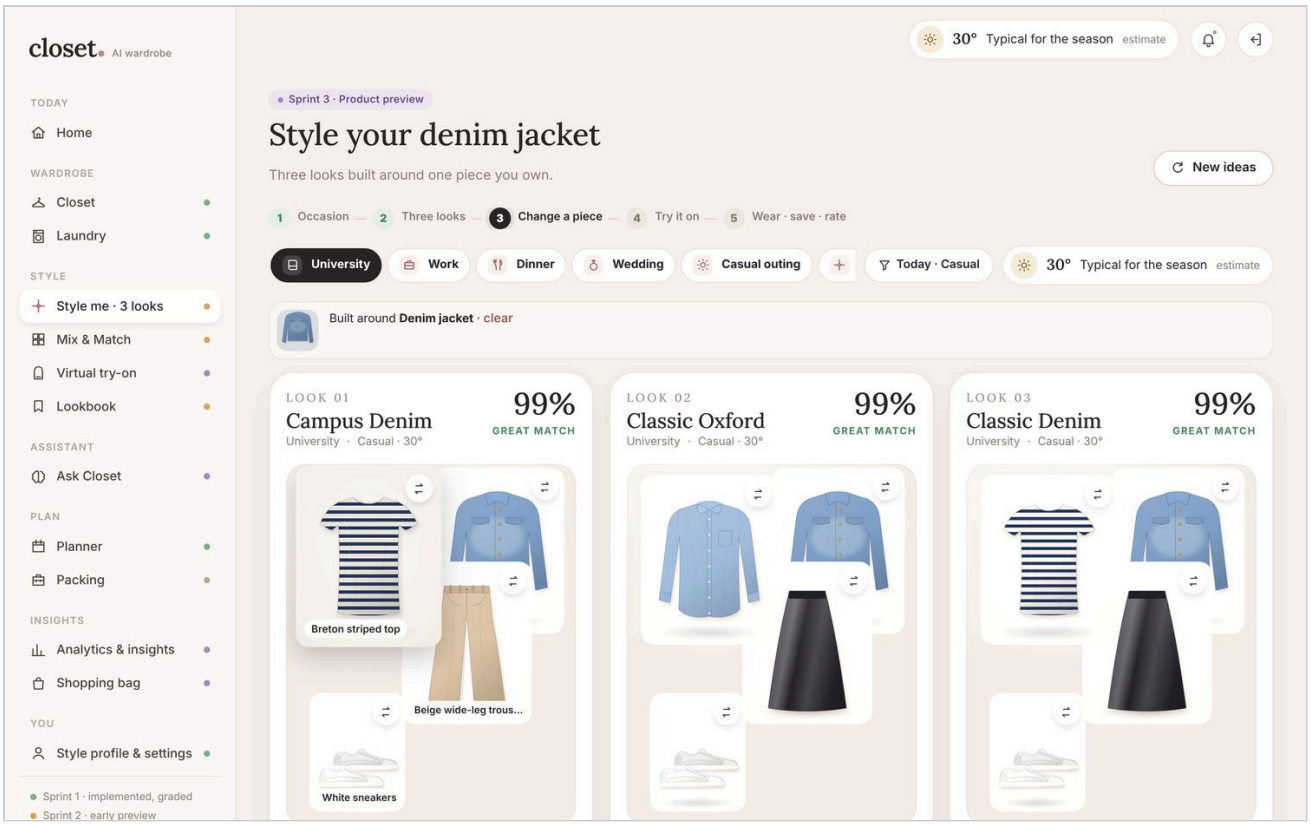


Figure 6-12 – A forgotten denim jacket — three looks that all contain it (S3 preview).

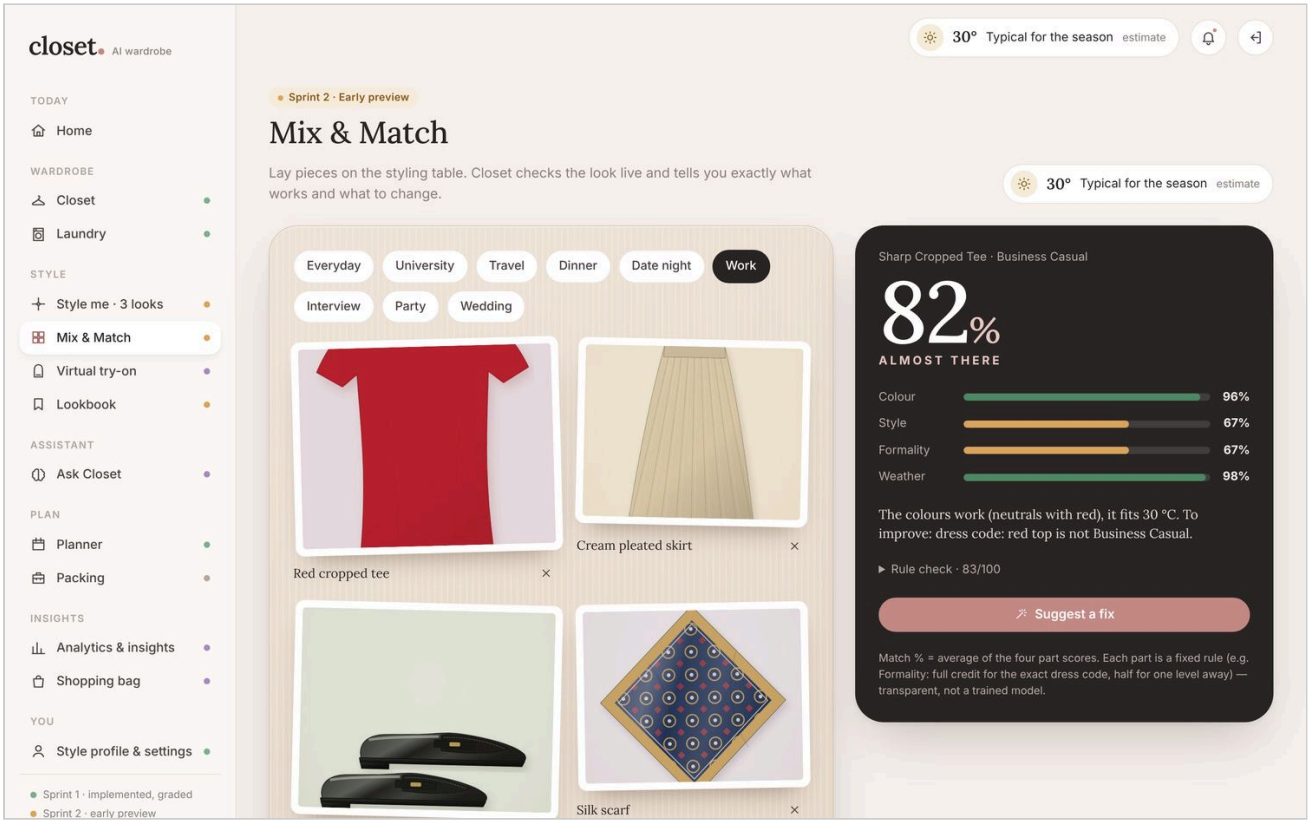


Figure 6-13 – Mix & Match — a weak piece lowers the score (S2 preview).

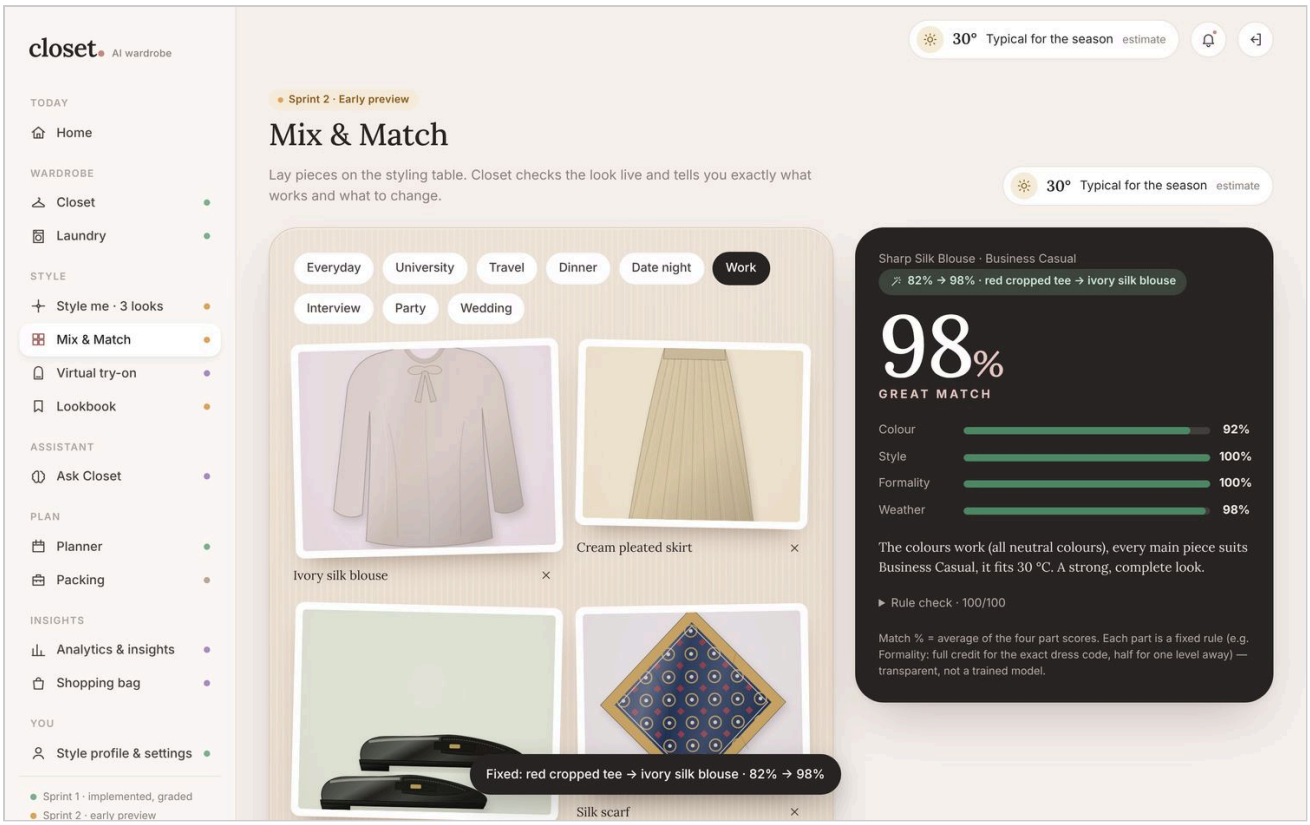


Figure 6-14 – Suggest a fix — 82 % → 98 % (S2 preview).

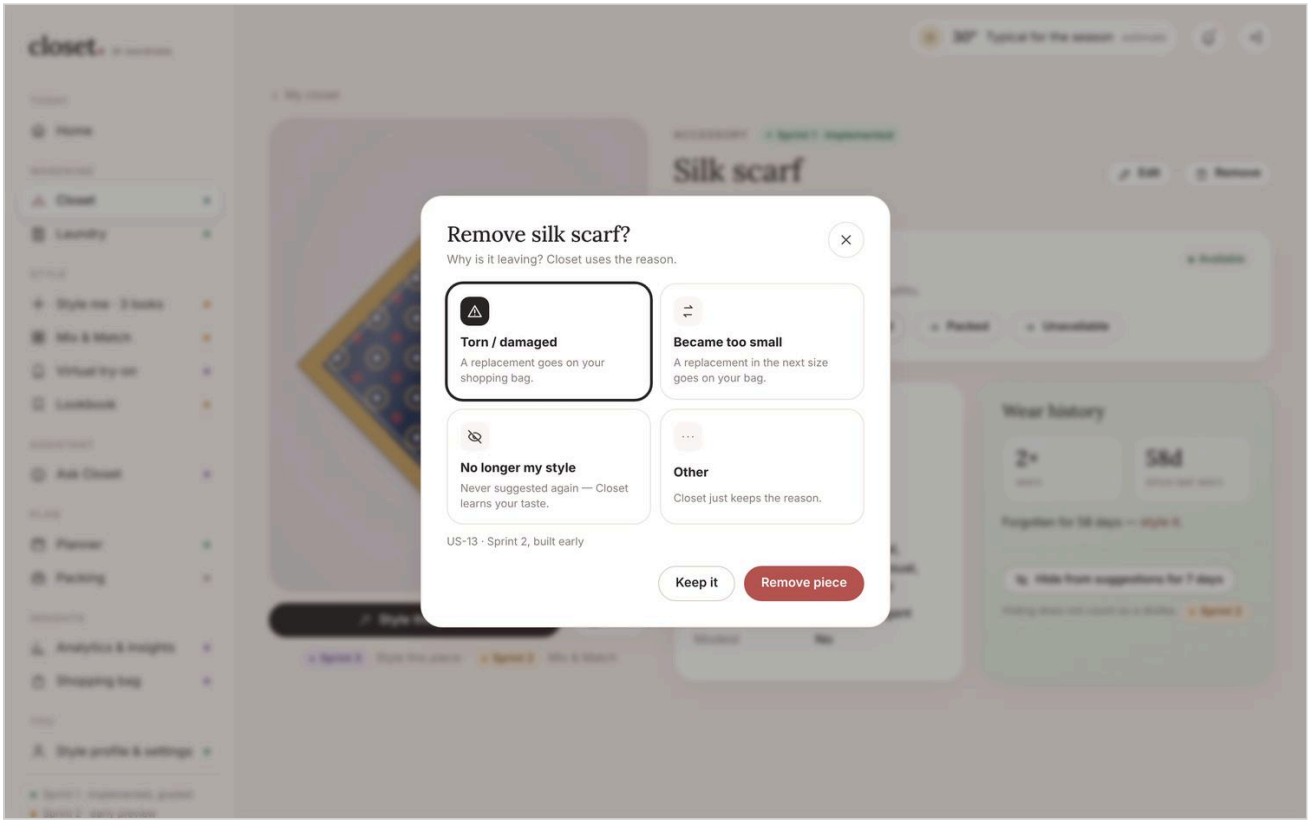


Figure 6-15 – Delete a piece — four reasons, each with its effect (S3 preview).

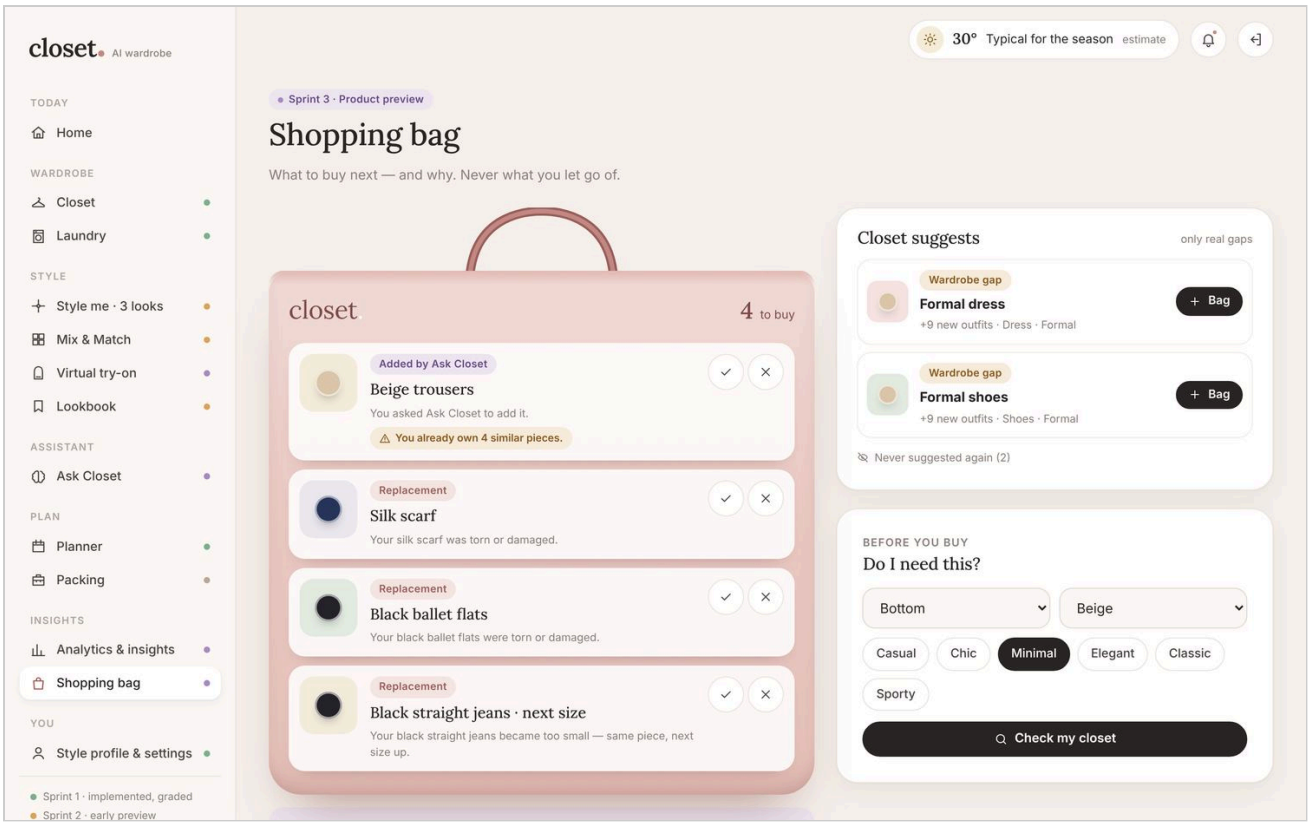


Figure 6-16 – Shopping bag — labelled sources (S3 preview).

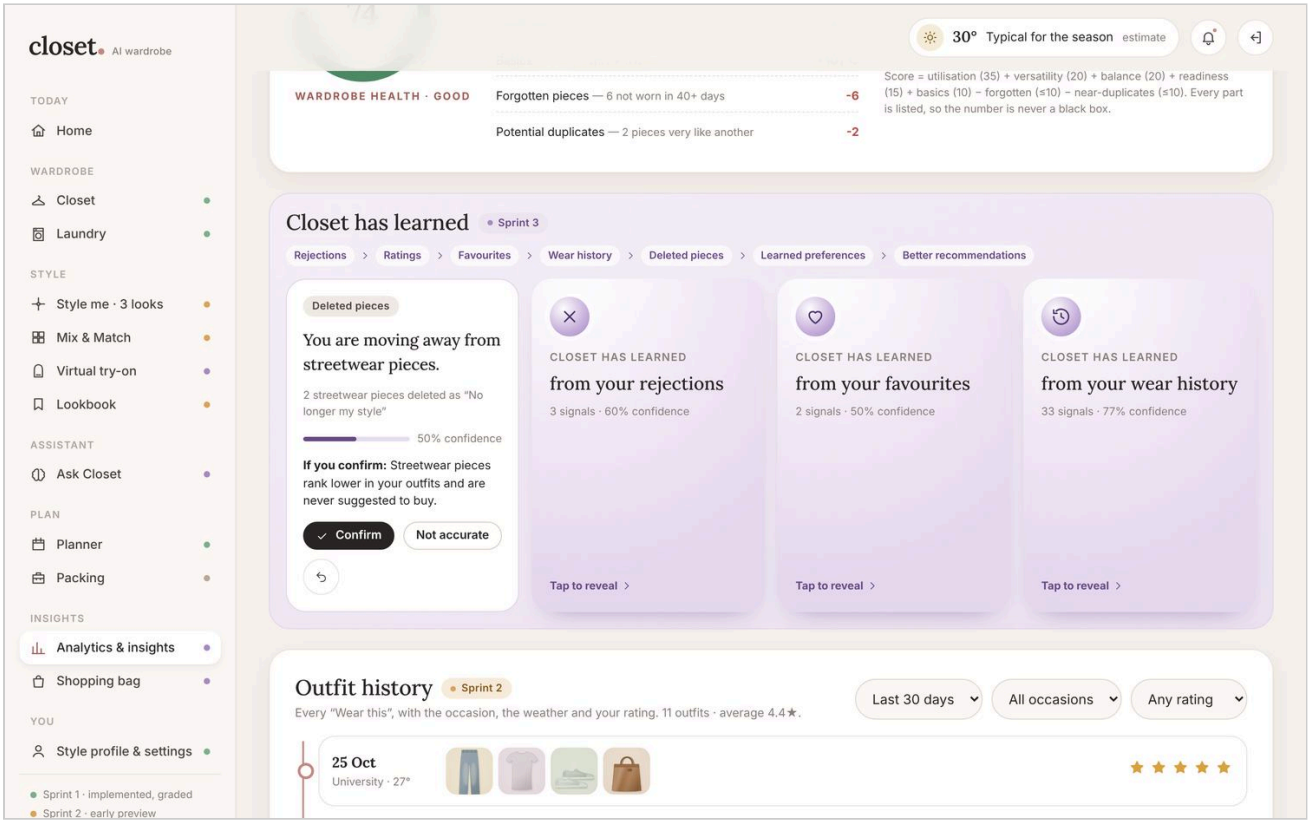


Figure 6-17 – Closet has learned — a flipped card (S3 preview).

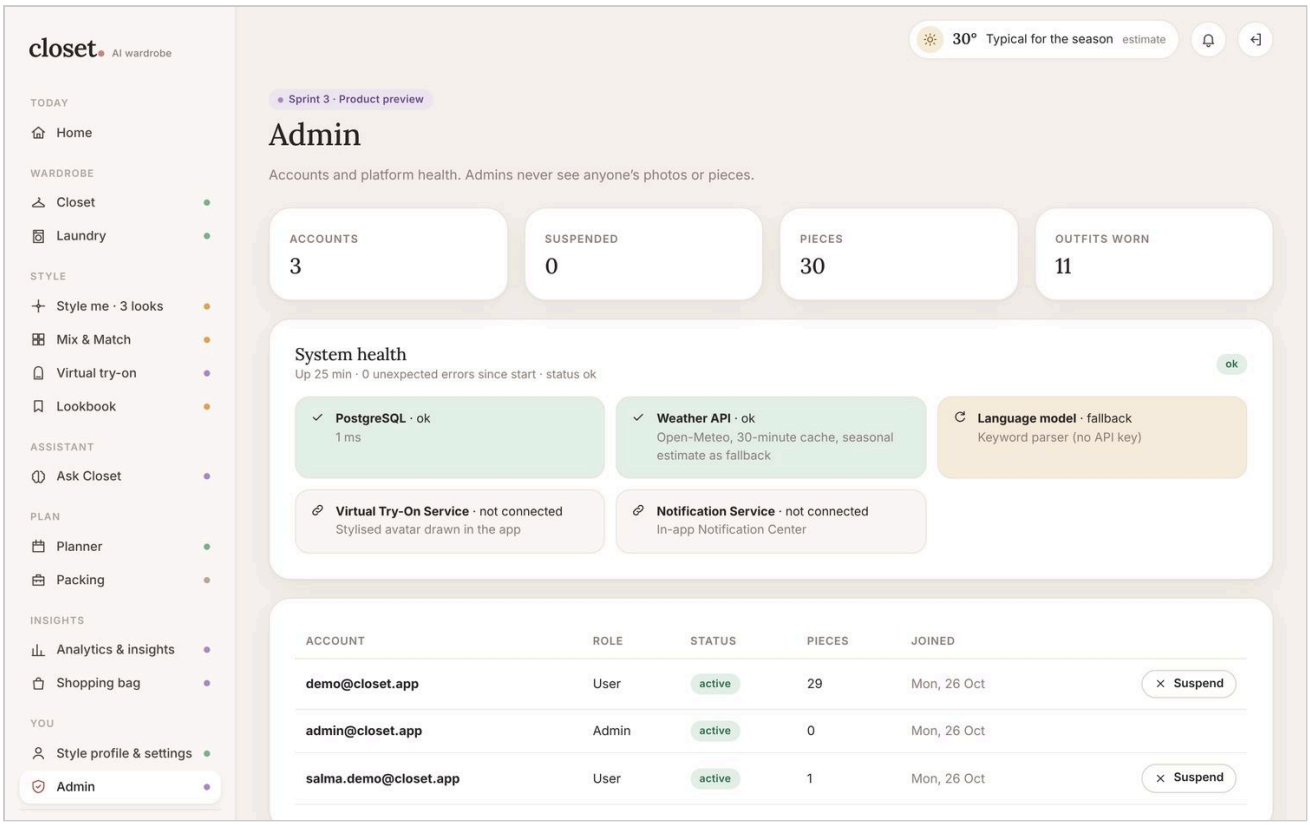


Figure 6-18 – Admin — accounts and system health (S3 / stretch preview).

6.3 Screen Objects and Actions

Screen	Object	Action → result	API
Sign in	Email, password, Sign in, Google	Correct → Home; wrong → one generic message; 5th failure → locked 15 min	POST /auth/login, /auth/google
Home	Today's look (flat lay, match %, "Recommended because …")	Wear this → wear counts + history; Try on → Dressing room; All 3 looks → Recommendations	POST /recommendations, /outfits/wear
Home	Wardrobe Health card	Shows score, parts and one action	GET /intelligence/health
Home	Coming up + conflict banner	"Use beige trench coat" → replaces only that piece	GET /intelligence/conflicts, POST …/fix
Home	Closet has learned card	Confirm → profile changes (Undo in the message); Not accurate → dismissed	POST /intelligence/learned/:key
Closet	Rails per category, filters, search	Open a piece; filters combine with counts	GET /items
Piece	Status buttons, return date, edit, delete with one of 4 reason cards	Status move (409 if not allowed); delete needs a reason; Torn / damaged or Became too small → replacement in the shopping bag (opens it); No longer my style → never suggested again	PATCH /items/:id/status, PATCH/DELETE /items/:id
Laundry room	Washing machine, rail, ironing board, Wash done, Clean now	Wash done → clothes lift out, counter runs down, "All clean — N pieces are Available again."	GET /laundry, POST /laundry/wash-done

Screen	Object	Action → result	API
Style me	Step rail, occasion tiles, day chips, Generate 3 looks, three flip cards	Generate → 3 looks; Change (per piece) → only that slot swapped; Why? → card flips to the factors; Try on → dressing room; Wear / Save / ♡ / Not for me → reason	POST /recommendations, /swap, /outfits, /outfits/wear, /outfits/reject
Mix & Match	Styling table (4 slots), verdict card	Pick pieces → live match %, parts, rules; Suggest a fix	POST /recommendations/check, /swap
Virtual try-on	Stage, LED vanity mirror with ~~the avatar~~ the user's photo wearing the look (avatar as fallback), fitting card, look dots, Choose my photo, Adjust the fit, Remove my photo	Previous / next of the same 3 looks; Wear this today; Save; Favourite; Change a piece → back to the looks (changes kept)	client + POST /outfits, /outfits/wear
Lookbook	Pages per saved look	Wear, Try on, Favourite, Remove	GET/PATCH/DELETE /outfits
Ask Closet	Prompts, chat, Undo	Message → one of 14 functions → answer with pieces, outfits or a bag item; "add this to my shopping list" uses the last buy check; Undo	POST /chat, /chat/undo/:id
Planner	Month calendar, day panel, plan modal, import	Plan outfit; Replace on conflict; import .ics	/events*, /intelligence/conflicts
Packing	Suitcase, trip form, day strip	Plan my suitcase → Add to packing list → Mark as packed / Unpack	/packing/*
Analytics	Editorial numbers, health, learned, history, forgotten	Give it another chance; Confirm / Undo; filter history	GET /insights, /intelligence/*, /outfits/history
Shopping bag	Bag with labelled items, Closet suggests, Do I need this?	✓ Bought / ✕ Remove; "You already own similar item" opens the similar pieces; Bag adds a suggestion; Buy anyway / Add to bag	GET/POST /shopping/list, PATCH/DELETE /shopping/list/:id, GET /shopping, POST /shopping/check
Notifications	Bell, list	Open → go to the source, marked read; Mark all read	GET /notifications, POST /notifications/read
Profile	Style, wardrobe rules, security, activity	Save; sign out; delete account (phrase + password)	PUT /profile, GET /activity, DELETE /account
Admin	Accounts, system health, configuration	Suspend / Restore (sessions end)	/admin/*

7. REQUIREMENTS MATRIX

Every functional requirement of the SRS (FR-01 – FR-80) traced to the components and data structures that satisfy it, the API that exposes it, and the tests that verify it.

FR	Component(s)	Data structures	API	Tests	Design status
FR-01	Identity & Auth, Request Validation	accounts.users, profile.profiles	POST /auth/register, /auth/google	AT-01, API-A01 – A05, API-A16 – A18	Implemented · S1
FR-02	Identity & Auth	accounts.users	POST /auth/login	API-A06, API-A07, UT-S06	Implemented · S1
FR-03	Identity & Auth	accounts.users (failed_logins, locked_until)	POST /auth/login	API-A08 – A10	Implemented · S1
FR-04	Authentication (IAuthToken), Identity & Auth	accounts.users (token_version, account_status)	POST /auth/logout · every protected route	API-A11 – A15, UT-S01 – S05, API-AD03	Implemented · S1
FR-05	Access Control, every Repository	user_id on every table	all /api/v1 routes	API-W16 – W18, API-DL03, API-R06, API-PK03	Implemented · S1
FR-06	Privacy, Event Bus (AccountDeleted)	all tables (CASCADE), photo files	DELETE /account	AT-15, API-D01 – D03	Implemented · S1
FR-07	Profile	profile.profiles	PUT /profile/quiz	AT-02 – AT-04, API-P01 – P03	Implemented · S1
FR-08	Profile	profile.profiles	PUT /profile	API-P04, API-P05	Implemented · S2 preview
FR-09	Profile, Availability	profile.profiles	PUT /profile	API-C01 – C03	Implemented · S2 preview
FR-10	Wardrobe, Photo Storage	wardrobe.items, photo file, availability.item_status	POST /items	AT-05, API-W01 – W08, UT-P01 – P08	Implemented · S1
FR-11	Wardrobe, Request Validation	wardrobe.items	POST /items	AT-06, API-W09, API-W10, UT-V11 – V21	Implemented · S1
FR-12	Web App (Add piece)	—	POST /items	AT-07	Implemented · S1
FR-13	Wardrobe	wardrobe.items, availability.item_status	GET /items	AT-08, API-W11 – W13	Implemented · S1
FR-14	Wardrobe	wardrobe.items	PATCH /items/:id	AT-09, API-W14, API-W15	Implemented · S1
FR-15	Wardrobe, Request Validation	wardrobe.deletions	DELETE /items/:id	API-DL01 – DL03, API-DR05, DR-01	Implemented · S2 preview
FR-16	Availability (State machine)	availability.item_status, status_history	PATCH /items/:id/status	AT-10, API-S01 – S06, ST-01 – ST-04	Implemented · S1
FR-17	Availability	availability.item_status	GET /laundry	AT-11, API-L01	Implemented · S1

FR	Component(s)	Data structures	API	Tests	Design status
FR-18	Availability	availability.item_status, status_history	POST /laundry/wash-done	AT-12, API-L02 – L04	Implemented · S1
FR-19	Availability (sweep)	availability.item_status, profile.profiles	GET /laundry, GET /items	API-C02, API-C03	Implemented · S2 preview
FR-20	Availability	availability.item_status (until, note)	PATCH /items/:id/status	API-C05	Implemented · S2 preview
FR-21	Wardrobe (closetGaps)	wardrobe.items, profile.profiles	GET /closet/gaps	AT-14, API-G01, UT-G01 – G07	Implemented · S1
FR-22	Planner	planner.events	POST/GET/PATCH/DELETE /events	AT-13, API-E01 – E05, UT-V22 – V26	Implemented · S1
FR-23	Planner, Calendar Adapter	planner.events	POST /events/import	API-PL03, CAL-01	Implemented · S2 preview
FR-24	Planner	planner.events, item_status, profiles	GET /events/calendar	API-PL01	Implemented · S2 preview
FR-25	Planner, Recommendation Engine	planner.events, wardrobe.items	POST /events/:id/recommend	API-C06, API-AC09	Implemented · S3 preview
FR-26	Planner, Outfits	planner.events, outfits.outfits	PUT /events/:id/outfit	API-K01, API-AC09	Implemented · S3 preview
FR-27	Recommendation Engine	wardrobe.items, item_status	POST /recommendations	RE-09, API-R01	Implemented · S2 preview
FR-28	Recommendation Engine (filter), Availability	availability.item_status	POST /recommendations	RE-01, EX-04, API-PK02	Implemented · S2 preview
FR-29	Recommendation Engine, Weather Adapter	— (adapter cache)	POST /recommendations, GET /weather	RE-05 – RE-08, WA-01 – WA-05	Implemented · S2 preview
FR-30	Recommendation Engine (filter)	wardrobe.items, profiles	POST /recommendations	RE-02, RE-03, RE-24	Implemented · S2 preview
FR-31	Recommendation Engine (filter)	wardrobe.items (last_worn_at), profiles	POST /recommendations	RE-21 – RE-23, API-C04	Implemented · S2 preview
FR-32	Recommendation Engine (explain)	outfits.feedback, outfits.outfits (favourites)	POST /recommendations	RE-12, EX-01 – EX-03, EX-06, API-EX01, API-EX02	Implemented · S2 preview
FR-33	Recommendation Engine (score)	—	POST /recommendations, /check	RE-10, RE-25 – RE-27, API-R05	Implemented · S2 preview
FR-34	Recommendation Engine (swap), Outfits	outfits.feedback	POST /recommendations/swap	RE-17, RE-18, RE-33, API-R04, API-CF04	Implemented · S2 preview
FR-35	Recommendation Engine	wardrobe.items	POST /recommendations {fixedId}	RE-13, RE-14, RE-31, RE-32, API-R06, API-FP01, API-FP02	Implemented · S2 preview

FR	Component(s)	Data structures	API	Tests	Design status
FR-36	Shared vocabulary, Web App	— (vocabulary)	GET /vocabulary	OT-01, API-OT01	Implemented · S2 preview
FR-37	Closet Intelligence, Planner, Recommendation Engine	planner.events, outfits.outfits, item_status	GET /intelligence/conflicts	API-K01, API-K03	Implemented · S2 preview
FR-38	Closet Intelligence	planner.events, outfits.outfits	GET /intelligence/conflicts	API-K04	Implemented · S2 preview
FR-39	Closet Intelligence, Planner	planner.events, outfits.outfits	POST /intelligence/conflicts/:id/fix	API-K02	Implemented · S2 preview
FR-40	Web App (Mix & Match)	wardrobe.items	GET /items	RE-24, browser journey	Implemented · S2 preview
FR-41	Recommendation Engine (check)	—	POST /recommendations/check	RE-19, RE-20, RE-29, API-R05	Implemented · S2 preview
FR-42	Web App (photo try-on preview, avatar fallback)	—	— (client) · ITryOn planned	TO-01 – TO-03, browser journey (42a, 42, 42b)	Implemented · S3 preview
FR-43	Outfits	outfits.outfits	POST/PATCH/DELETE /outfits	API-O01	Implemented · S2 preview
FR-44	Outfits, Wardrobe, Event Bus (OutfitWorn)	outfits.wears, wardrobe.items	POST /outfits/wear	API-O02, API-OH01	Implemented · S2 preview
FR-45	Outfits	outfits.wears	GET /outfits/history	API-OH01	Implemented · S2 preview
FR-46	Outfits	outfits.wears	PATCH /outfits/wears/:id	API-O02	Implemented · S3 preview
FR-47	Outfits	outfits.feedback, outfits.hidden_items	POST /outfits/reject, /items/:id/hide	API-O03, RE-04, RE-16	Implemented · S2 preview
FR-48	Closet Intelligence (learning)	outfits.feedback, wears, outfits	GET /intelligence/learned	LR-01 – LR-05, API-LR01	Implemented · S3 preview
FR-49	Closet Intelligence, Profile	intelligence.learned, profile.profiles	POST/DELETE /intelligence/learned/:key	API-LR01, API-LR02	Implemented · S3 preview
FR-50	Recommendation Engine (ranker)	outfits.feedback, outfits	POST /recommendations	EX-05, API-EX02, DR-06	Implemented · S3 preview
FR-51	Ask Closet, Language Model Adapter	assistant.actions	POST /chat	AI-01 – AI-16, AI-20 – AI-27, API-AC01 – AC03, AC05 – AC10, DU-03	Implemented · S3 preview
FR-52	Ask Closet	assistant.actions	POST /chat/undo/:id	API-AC04, API-AC08	Implemented · S3 preview
FR-53	Language Model Adapter, Keyword Parser	—	POST /chat	AI-17 – AI-19	Implemented · S3 preview

FR	Component(s)	Data structures	API	Tests	Design status
FR-54	Outfits (insights)	outfits.wears, wardrobe.items	GET /insights	API-O02, API-O04	Implemented · S3 preview
FR-55	Outfits (insights), Recommendation Engine	wardrobe.items	GET /insights	API-O04, RE-14	Implemented · S3 preview
FR-56	Closet Intelligence (health)	wardrobe.items, outfits.wears, item_status	GET /intelligence/health	WH-01 – WH-05, API-H01, API-H02	Implemented · S3 preview
FR-57	Smart Shopping	wardrobe.deletions, items, shopping.decisions	GET /shopping	API-SH01, API-AC05	Implemented · S3 preview
FR-58	Smart Shopping, DeletionPolicy	wardrobe.deletions	GET /shopping, POST /shopping/check	DR-03, API-DR03	Implemented · S3 preview
FR-59	Smart Shopping	wardrobe.items, deletions	POST /shopping/check	API-SH04, API-AC03	Implemented · S3 preview
FR-60	Smart Shopping (similarity)	wardrobe.items	POST /shopping/check	DU-01 – DU-03, API-DU01, API-DU02	Implemented · S3 preview
FR-61	Smart Shopping (shopping list)	shopping.list_items, shopping.decisions	GET/POST /shopping/list · PATCH/DELETE /shopping/list/:id	API-SL04, API-SL05, API-SH03	Implemented · S3 preview
FR-62	Notification Center (Closet Intelligence), Planner, Packing	events, item_status, packing.trips, activity.log, notifications_read	GET /notifications	API-N01, API-N02, API-K03, API-PL02	Implemented · S3 preview
FR-63	Notification Center	intelligence.notifications_read	POST /notifications/read	API-N01	Implemented · S3 preview
FR-64	Packing, Weather Adapter, Recommendation Engine	packing.trips	POST /packing/plan, /packing/trips	PK-01, PK-03, API-PK01	Implemented · stretch preview
FR-65	Packing	—	POST /packing/plan	PK-02	Implemented · stretch preview
FR-66	Packing, Availability	packing.trips, availability.item_status	POST /packing/trips/:id/pack, /unpack	API-PK02, API-PK03	Implemented · stretch preview
FR-67	Admin, Access Control	accounts.users (counts only)	GET /admin/users, PATCH /admin/users/:id	API-AD02 – AD04	Implemented · S3 preview
FR-68	Access Control (requireRole)	accounts.users (role)	/admin/*	API-AD01, API-AD05, UT-S08	Implemented · S3 preview
FR-69	Admin	—	GET /admin/health, /admin/config	API-AD05	Implemented · stretch preview
FR-70	Audit (API Gateway), Activity Log	activity.log	GET /activity	API-N02, API-N03	Implemented · S2 preview

FR	Component(s)	Data structures	API	Tests	Design status
FR-71	Wardrobe → Event Bus (ItemDeleted) → Smart Shopping, DeletionPolicy	wardrobe.deletions, shopping.list_items, profile.profiles (avoid_styles)	DELETE /items/:id	DR-01, DR-02, API-DR01 – DR05	Implemented · S2 preview
FR-72	Smart Shopping (onDeleted), DeletionPolicy	shopping.list_items (source replacement, replaces)	DELETE /items/:id · GET /shopping/list	DR-02, API-DR01, API-DR02, API-SH02	Implemented · S3 preview
FR-73	Closet Intelligence (learning), DeletionPolicy, Profile	wardrobe.deletions (styles), intelligence.learned, profile.profiles (avoid_styles)	GET/POST/DELETE /intelligence/learned	DR-04, DR-05, API-DR04	Implemented · S3 preview
FR-74	Ask Closet → Smart Shopping (shopping list)	shopping.list_items, assistant.actions	POST /chat, POST /chat/undo/:id	SL-01, SL-02, API-SL01 – SL04	Implemented · S3 preview
FR-75	Web App (Style me), Recommendation Engine, Outfits	— (client look store)	POST /recommendations, /recommendations/swap, /outfits, /outfits/wear, /outfits/reject	API-CF01, API-CF02, API-R01, browser journey (39 – 41)	Implemented · S2 preview
FR-76	Web App (dressing room, look store)	— (client sessionStorage closet.looks)	— (client) · POST /outfits, /outfits/wear	browser journey (42_tryon, 42b_tryon_next)	Implemented · S3 preview
FR-77	Admin (no wardrobe routes), Access Control	accounts.users (counts only)	/admin/*	API-AD02	Implemented · S3 preview
FR-78	Recommendation Engine (lookNames)	—	POST /recommendations	RE-30, API-CF03, API-R01, browser journey (40)	Implemented · S2 preview
FR-79	Web App (Mix & Match), Recommendation Engine	— (client loop over /swap)	POST /recommendations/check, /recommendations/swap	API-MM01, browser journey (44a – 44c)	Implemented · S2 preview
FR-80	Web App (photoTryOn: browser only)	— (sessionStorage in the user’s browser only)	— (never sent)	browser journey (42: no request carries the photo)	Implemented · S3 preview

8. APPENDICES

Appendix A – Deployment and testing architecture

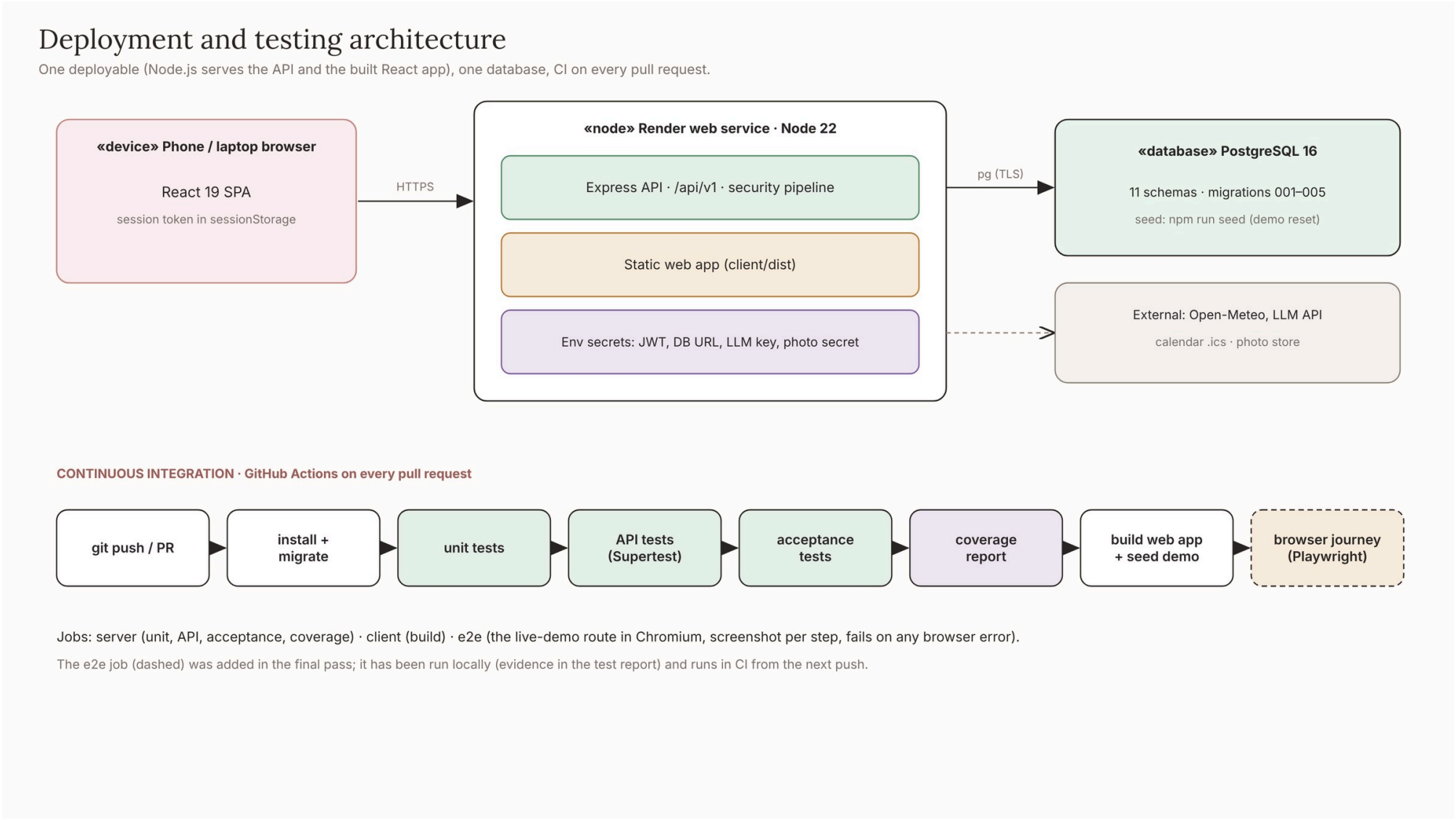


Figure A-1 – Deployment and CI.

Test architecture. Unit tests cover the pure domain functions; API tests run the real Express app against a real PostgreSQL test database that is emptied before every test (Supertest); acceptance tests map one-to-one to the Sprint 1 acceptance criteria; a Playwright browser journey walks the demo path and fails on any console error. External services are replaced by fakes (weather) or disabled (LLM key absent → keyword parser) so tests are deterministic.

Suite	Tests	Result
Sprint 1 — unit (validation, security, status machine, closet gaps)	89	89 pass
Sprint 1 — API (auth, wardrobe, availability, profile, planner, privacy)	65	65 pass
Sprint 1 — acceptance (one per acceptance criterion)	15	15 pass
v1.2 previews — engine, assistant, weather, calendar, preview API	102	102 pass
v1.3 — Closet Intelligence unit + API	44	44 pass
v1.4 — deletion reasons, shopping list, core flow (unit + API)	20	20 pass
Final pass — look names, replace tray, forgotten piece, Mix & Match, try-on placement, repeatable demo	14	14 pass
Total	**349**	**349 pass, 0 fail**

Coverage (V8 provider): lines 97.95 %, branches 84.57 %, functions 94.72 % (statements are not reported separately by this provider). Browser journey: 58 steps, 0 browser errors, three runs from a fresh seed with identical results (run locally; the CI e2e job runs from the next push). Evidence: test_run_final.txt, coverage_final.txt, e2e_final.txt.

Appendix B – Security design

Threat	Control	Where	Test
Password guessing	bcrypt (cost 10); lockout 5 → 15 min; sign-in rate limit 30 / 15 min; generic message	Identity & Auth, API Gateway	API-A07 – A10
Stolen or forged token	HS256 JWT, 2 h, token version (sign-out / suspend increments it), `alg: none` refused	Authentication	UT-S01 – S05, API-A13, API-A14
Reading another user's data	Repository filters by user_id; foreign id → 404	Every repository	API-W16 – W18, API-DL03, API-PK03
Privilege escalation	`requireRole('admin')` on /admin; self-suspend refused	Access Control, Admin	UT-S08, API-AD01, API-AD04, API-AD05
Injection / bad input	Schema validation per route; parameterised SQL only	Request Validation, repositories	UT-V*, API-X03
Photo leaks	Private storage, HMAC-signed expiring URLs, file type from content, EXIF removed	Photo Storage	UT-P01 – P08, API-W02, API-W03, API-W17
Information disclosure	Generic 500 without details; security headers; server not advertised	Error handler, API Gateway	API-X01, API-X05
Repudiation	Activity log of every successful change	Audit step	API-N02, API-N03
LLM misuse	14 whitelisted functions; arguments limited to the vocabulary; no photos or database access; Undo	Language Model Adapter, Ask Closet	AI-17 – AI-19, API-AC10

Appendix C – Error handling and validation

All errors are JSON `{error, message, details[]}`. 400 `VALIDATION_FAILED` lists every field with a plain message; 401 not signed in; 403 wrong role or suspended; 404 not found (also for other users' rows); 409

business-rule conflict (invalid status move, conflict already fixed, piece not available); 413 too large; 415 unsupported photo type; 423 locked; 500 generic. The client maps `details` to the fields and shows the message in a toast; no raw error ever reaches the screen. Adapters never throw to callers: the weather adapter returns a labelled seasonal estimate, the language-model adapter returns the keyword parser's result, and auditing failures never fail a request.

Appendix D – Demo data and reset

`npm run seed` rebuilds the demo account **demo@closet.app / Closet2026!** (display name Habiba) through the real API, so the same validation and events run: 29 illustrated pieces; three pieces in the laundry; a dress lent to Nour (due tomorrow); sandals packed; one piece deleted as "Became too small" after 14 wears; 11 worn outfits in 30 days with occasions, weather and ratings; four rejections (three "Too formal") so "Closet has learned" has evidence; four saved looks (two favourites, one built); four events — two with planned outfits, the next one tomorrow with the navy blazer (the conflict demo); a Paris packing list; and admin@closet.app. Running the seed again resets the demo exactly.

Appendix E – Design change log

Design changes since the Milestone 1 baseline (requirement changes are in the SRS, Appendix D).

ID	Change	Section
C-02	Security pipeline in the API tier: gateway → validation → authentication → access control (instructor feedback)	3.1
C-09	PhotoStorage interface, signed URLs, EXIF removal	3.1, 4.1
C-11	Events UserRegistered, ItemCreated; interfaces IAuthToken, IAdminAPI	3.2.2
C-12	Lower-level design: ER, class, sequence, state diagrams, API table	3.2, 4
C-13	Status state machine with 10 moves	3.2.5
C-22	Availability drawn as its own component; LLM role note; Try-On, Calendar, Notification adapters	3.1
C-25	Smart Shopping separated from Ask Closet	3.1
C-41	Closet Intelligence module: Wardrobe Health, learned patterns with exact Undo, conflict detection and fix	3.1, 5.5
C-42	Engine explain stage: factors and "Recommended because" sentence; feedback-aware ranker (rule score unchanged)	5.4
C-43	Audit step (Interceptor) in the pipeline + Activity Log module; migration 004	3.1, 5.1, 4
C-44	Notification Center computed from module state; read table; 3 highlights	5.5
C-45	Packing module (stretch) using Weather, Engine, Planner, Availability	5.7
C-46	Similarity score for duplicate warnings in Smart Shopping	5.6
C-47	Outfit history: occasion and weather stored with each wear	4.2
C-48	Admin health and read-only configuration endpoints	3.2.3
C-49	Design moved out of the combined specification into this SDD (IEEE 1016); diagrams v1.3 drawn for slides and print	all
C-58	DeletionPolicy (pure table of reason → effect) and the ItemDeleted event (Observer); deletions keep styles, dress codes, fabric	3.2.2, 5.14
C-59	Shopping list: table shopping.list_items with source and replaces; labels; similar-owned warning; IShopping; 4 endpoints	4.2, 5.6
C-60	Ask Closet function add_to_shopping (14 functions); buy check passes its piece as context; Undo removes the item	5.15

ID	Change	Section
C-61	Learned let-go style: profile.avoid_styles; ranker −3; explanation factor; suggestions filtered; migration 005	4.2, 5.5, 5.14
C-62	Core flow: Style me step rail, three flip cards, client look store shared with the dressing room	5.13, 6
C-63	Interaction design of the dressing room, laundry room, shopping bag and learned flip card; try-on release corrected to Sprint 3	5.16, 6
C-68	Fixed-item path: the chosen piece is always in the pool when Available; a fixed outerwear piece completes top + bottom looks (defect D-06)	5.17
C-69	Look names (lookNames) and the replace tray: swap returns up to 3 options with re-scored looks	5.13, 3.2.3
C-70	Virtual Try-On Preview on the user’s photo in the browser (photoTryOn.js) with privacy and retention; ITryOn kept as the planned provider adapter	3.2.2, 3.3, 5.18
C-71	Complete ER on one sheet; sequences D16 (try-on photo) and D17 (forgotten piece, Suggest a fix); CI e2e job in the deployment diagram	4, 5.17, 5.18